Lecture Notes in Data Mining

Introduction to Data Mining

2.1 Introduction to Data Mining

In the age of information, an enormous amount of data is available in different industries and organizations. The availability of this massive data is of no use unless it is transformed into valuable information. Otherwise, we are sinking in data, but starving for knowledge. The solution to this problem is data mining which is the extraction of useful information from the huge amount of data that is available.

Data mining is defined as follows:

'Data mining is a collection of techniques for efficient automated discovery of previously unknown, valid, novel, useful and understandable patterns in large databases. The patterns must be actionable so they may be used in an enterprise's decision making.'

From this definition, the important take aways are:

- Data mining is a process of automated discovery of previously unknown patterns in large volumes of data.
- This large volume of data is usually the historical data of an organization known as the data warehouse.
- Data mining deals with large volumes of data, in Gigabytes or Terabytes of data and sometimes as much as Zetabytes of data (in case of big data).

- Patterns mus be valid, novel, useful and understandable.
- Data mining allows businesses to determine historical patterns to predict future behaviour.
- Although data mining is possible with smaller amounts of data, the bigger the data the better the accuracy in prediction.
- There is considerable hype about data mining at present, and the Gartner Group has listed data mining as one of the top ten technologies to watch.

2.2 Need of Data Mining

Data mining is a recent buzz word in the field of Computer Science. It is a computing process that uses intelligent mathematical algorithms to extract the relevant data and computes the probability of future actions. It is also known as Knowledge Discovery in Data (KDD).

Although data mining has existed for 50 years, it has become a very important technology only recently due to the Internet boom. Database systems are being used since the 1960s in the Western countries (perhaps, since the 1980s in India). These systems have generated mountains of data. There is a broad agreement among all sources that the size of the digital universe will double every two years at least. Therefore, there will be a 50-fold growth from 2010 to 2020.

Every minute, huge data is generated over Internet as depicted in Figure 2.1. The analysis of this huge amount of data is an important task and it is performed with the help of data mining.

Some facts about this rapid growth of data are as follows.

- 48 hours of new videos are uploaded by YouTube users every minute of the day.
- 571 new websites are created every minute of the day.
- 90% of world's data has been produced in the last two years.
- The production of data will be 44 times greater in 2020 than it was in 2010.
- Brands and organizations on Facebook receive 34,722 'Likes' every minute of the day.
- 100 terabytes of data is uploaded daily to Facebook.
- In early 2012, there were more than 465 million accounts and 175 million tweets were shared by people on Twitter every day, according to Twitter's own research.
- Every month, 30 billion pieces of content are shared by people on Facebook.
- By late 2011, approximately 1.8 zetta byte of data had been created in that year alone, according the report published by IDC Digital Universe.

The analysis of such huge amounts of data is a challenging task that requires suitable analytics to sieve it for the bits that will be useful for business purposes. From the start, data mining was designed to find data patterns from data warehouses and data mining algorithms are tuned so that they can handle large volumes of data. The boom in big data volumes has led to great interest in data mining all over the world.

The other allied reasons for popularity of data mining are as follows.

- Growth in generation and storage of corporate data
- Need for sophisticated decision making
- Evolution of technology
- Availability of much cheaper storage, easier data collection and better database management for data analysis and understanding
- Point of sale terminals and bar codes on many products, railway bookings, educational institutions, mobile phones, electronic gadgets, e-commerce, etc., all generate data.
- Great volumes of data generated with the recent prevalence of Internet banking, ATMs, credit and debit cards; medical data, hospitals; automatic toll collection on toll roads, growing air travel; passports, visas, etc.
- Decline in the costs of hard drives
- Growth in worldwide disk capacities

Thus, the need for analyzing and synthesizing information is growing in the fiercely competitive business environment of today.

2.3 What Can Data Mining Do and Not Do?

Data mining is a powerful tool that helps to determine the relationships and patterns within data. However, it does not work by itself and does not eliminate the requirement for understanding data, analytical methods and to know business. Data mining extracts hidden information from the data, but it is not able to assess the value of the information.

One should know the important patterns and relationships to work with data over time. In addition to discovering new patterns, data mining can also highlight other empirical observations that are not instantly visible through simple observation.

It is important to note that the relationships or patterns predicted through data mining are not necessarily causes for an action or behavior. For example, data mining may help in determining that males with income between Rs 20,000 and Rs 75,000 who contribute to certain journals or magazines, may be expected to purchase such-and-such a product. This information can be helpful in developing a marketing strategy. However, it is not necessary that the population identified through data mining will purchase the product simply because they belong to this category.

2.4 Data Mining Applications

The applications of data mining exist in almost every field. Some of the important applications of data mining are in finance, telecom, insurance and retail sectors include loan/credit card approval, fraud detection, market segmentation, trend analysis, better marketing, market basket analysis, customer churn and web site design and promotion. These are discussed below.

Loan/Credit card approvals

Banks are able to assess the credit worthiness of their customers by mining a customer's historical records of business transactions. So, credit agencies and banks collect a lot of customer' behavioural data from many sources. This information is used to predict the chances of a customer paying back a loan.

Market segmentation

A huge amount of data about customers is available from purchase records. This data is very useful to segment customers on the basis of their purchase history. Let us suppose a mega store sells multiple items ranging from grocery to books, electronics, clothing et al. That store now plans to launch a sale on books. Instead of sending SMS texts to all the customers it is logical and more efficient to send the SMS only to those customers who have demonstrated interest in buying books, i.e., those who had earlier purchased books from the store. In this case, the segmentation of customers based on their historical purchases will help to send the message to those who may find it relevant. It will also give a list of people who are prospects for the product.

Fraud detection

Fraud detection is a very challenging task because it's difficult to define the characteristics for detection of fraud. Fraud detection can be performed by analyzing the patterns or relationships that deviate from an expected norm. With the help of data mining, we can mine the data of an organization to know about outliers as they may be possible locations for fraud.

Better marketing

Usually all online sale web portals provide recommendations to their users based on their previous purchase choices, and purchases made by customers of similar profile. Such recommendations are generated through data mining and help to achieve more sales with better marketing of their products. For example, amazon.com uses associations and provides recommendations to customers on the basis of past purchases and what other customers are purchasing. To take another example, a shoe store can use data mining to identify the right shoes to stock in the right store on the basis of shoe sizes of the customers in the region.

Trend analysis

In a large company, not all trends are always visible to the management. It is then useful to use data mining software that will help identify trends. Trends may be long term trends, cyclic trends or seasonal trends.

Market basket analysis

Market basket analysis is useful in designing store layouts or in deciding which items to put on sale. It aims to find what the customers buy and what they buy together.

Customer churn

If an organization knows its customers better then it can retain them longer. In businesses like telecommunications, companies very hard to retain their good customers and to perhaps persuade good customers of their competitors to switch to them. In such an environment, businesses want to rate customers (whether they are worthwhile to be retained), why customers switch over and what makes customers loyal. With the help of this information some businesses may wish to get rid of customers that cost more than they are worth, e.g., credit card holders that don't use the card; bank customers with very small amounts of money in their accounts.

Website design

A web site is effective only if the visitors easily find what they are looking for. Data mining can help discover affinity of visitors to pages and the site layout may be modified based on data generated from their web clicks.

Corporate analysis and risk management

Data mining can be used to perform cash flow analysis to plan finance and estimate assets. The analysis of data can be performed by comparing and summarizing the spending and planning of resources. It helps to analyze market directions and monitor competitors to analyze the competition.

Thus, one can conclude that the applications of data mining are numerous and it is worthwhile for an organization to invest in data mining for generating better business revenue. This is the reason that many organizations are investing in this field and there is a huge demand for data mining experts in the world.

2.5 Data Mining Process

Data mining process consists of six phases, as illustrated in Figure 2.2.

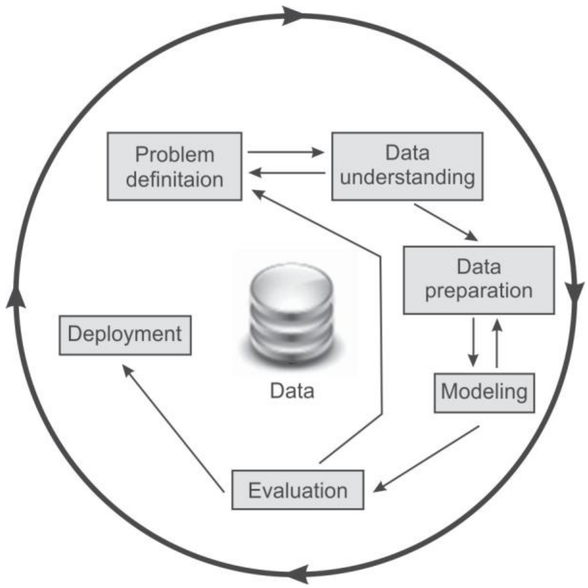


Figure 2.2 Data mining process

A brief description about these phases is as follows.

Problem definition phase

The main focus of the first phase of a data mining process is to understand the requirements and objectives of such a project. Once the project has been specified, it can be formulated as a data mining problem. After this, a preliminary implementation plan can be developed.

Let us consider a business problem such as 'How can I sell more of my product to customers?' This business problem can be translated into a data mining problem such as 'Which customers are most likely to buy the product?' A model that predicts the frequent customers of a product must be built on the previous records of customers' data. Before building the model, the data must be assembled that consists of relationships between customers who have purchased the product and customers who have not purchased the product. The attributes of customers might include age, years of residence, number of children, owners/renters, and so on.

Data understanding phase

The next phase of the data mining process starts with the data collection. In this phase, data is collected from the available sources and in order to make data collection proper, some important activities such as data loading and data integration are performed. After this, the data is analyzed closely to determine whether the data will address the business problem or not. Therefore, additional data can be added or removed to solve the problem effectively. At this stage missing data is also identified. For example, if we require the AGE attribute for a record then column DATE_OF_BIRTH can be changed to AGE. We can also consider another example in which average income can be inserted if value of column INCOME is null. Moreover, new computed attributes can be added in the data in order to obtain better focused information. For example, a new attribute such as 'Number of Times Amount Purchased Exceeds Rs. 500 in a 12 month time period.' can be created instead of using the purchase amount.

Data preparation phase

This phase generally consumes about 90% of the time of a project. Once available data sources are identified, they need to be selected, cleaned, constructed and formatted into the desired form for further processing.

Modeling phase

In this phase, different data mining algorithms are applied to build models. Appropriate data mining algorithms are selected and applied on given data to achieve the objectives of proposed solution.

Evaluation phase

In the evaluation phase, the model results are evaluated to determine whether it satisfies the originally stated business goal or not. For this the given data is divided into training and testing datasets. The models are trained on training data and tested on testing data. If the accuracy of models on testing data is not adequate then one goes back to the previous phases to fine tune those areas that may be the reasons for low accuracy. Having achieved a satisfactory level of accuracy, the process shifts to the deployment phase.

Deployment phase

In the deployment phase, insights and valuable information derived from data need to be presented in such a way that stakeholders can use it when they want to. On the basis of requirements of the project, the deployment phase can be simple (just creating a report) or complex (requiring further iterative data mining processing). In this phase, Dashboards or Graphical User Interfaces are built to solve all the requirements of stakeholders.

2.6 Data Mining Techniques

Data mining can be classified into four major techniques as given below.

- Predictive modeling
- Database segmentation
- Link analysis
- Deviation detection

To briefly discuss each technique:

2.6.1 Predictive modeling

Predictive modeling is based on predicting the outcome of an event. It is designed on a pattern similar to the human learning experience in using observations to form a model of the important characteristics of some task. It is developed using a supervised learning approach, where we have some labeled data and we use this data to predict the outcome of unknown instances. It can be of two types, i.e., classification or regression as discussed in Chapter 1.

Some of the applications of predictive modeling are: predicting the outcome of an event, predicting the sale price of a property, predicting placement of students, predicting the score of any team during a cricket match and so on.

2.6.2 Database segmentation

Database segmentation is based on the concept of clustering of data and it falls under unsupervised learning, where data is not labeled. This data is segmented into groups or clusters based on its features or attributes. Segmentation is creating a group of similar records that share a number of properties.

Applications of database segmentation include customer segmentation, customer churn, direct marketing, and cross-selling.

2.6.3 Link analysis

Link analysis aims to establish links, called associations, between the individual record, or sets of records, in a database. There are three specialisations of link analysis.

- Associations discovery
- Sequential pattern discovery
- Similar time sequence discovery

Associations discovery locates items that imply the presence of other items in the same event. There are association rules which are used to define association. For example, 'when a customer rents property for more than two years and is more than 25 years old, in 40% of cases, the customer will buy a property. This association happens in 35% of all customers who rent properties'.

Sequential pattern discovery finds patterns between events such that the presence of one set of items is followed by another set of items in a database of events over a period of time. For example, this approach can be used to understand long-term customer buying behavior.

Time sequence discovery is used to determine whether links exist between two sets of data that are time-dependent. For example, within three months of buying property, new home owners will purchase goods such as cookers, freezers, and washing machines.

Applications of link analysis include market basket analysis, recommendation system, direct marketing, and stock price movement.

2.6.4 Deviation detection

Deviation detection is a relatively new technique in terms of commercially available data mining tools. It is based on identifying the outliers in the database, which indicates deviation from some previously known expectation and norm. This operation can be performed using statistics and visualization techniques.

Applications of deviation detection include fraud detection in the use of credit cards and insurance claims, quality control, and defects tracing.

2.7 Difference between Data Mining and Machine Learning

Data mining refers to extracting knowledge from a large amount of data, and it is a process to discover various types of patterns that are inherent in the data and which are accurate, new and useful. It is an iterative process and is used to uncover previously unknown trends and patterns in vast amount of data in order to support decision making.

Data mining is the subset of business analytics; it is similar to experimental research. The origins of data mining are databases and statistics. Two components are required to implement data mining techniques: the first is the database and the second is machine learning.

Data mining as a process includes data understanding, data preparation and data modeling; while machine learning takes the processed data as input and performs predictions by applying algorithms. Thus, data mining requires involvement of human beings to clean and prepare the data and to understand the patterns. While in machine learning human effort is involved only to define an algorithm, after which the algorithm takes over operations. Tabular comparison of data mining and machine learning is given in Table 2.1.

Table 2.1 Tabular comparison of data mining and machine learning

Basic for comparison	*Data mining*	*Machine learning*
Meaning	It involves extracting useful knowledge from a large amount of data.	It introduces new algorithm from data as well as past experience.
History	Introduced in 1930 it was initially called knowledge discovery in databases.	It was introduced in 1959.
Responsibility	Data mining is used to examine patterns in existing data. This can then be used to set rules.	Machine learning teaches the computer to learn and understand the given rules.
Nature	It involves human involvement and intervention.	It is automated, once designed it is self-implementing and no or very little human effort is required.

In conclusion the analysis of huge amounts of data generated over Internet or by traditional database management systems is an important task. This analysis of data has the huge potential to improve business returns. Data mining and machine learning play vital roles in understanding and analyzing data. Thus, if data is considered as oil, data mining and machine learning are equivalent to modern day oil refineries that make this data more useful and insightful.

Data Preprocessing

4.1 Need for Data Preprocessing

For any data analyst, one of the most important concerns is 'Data'. In fact, the representation and quality of data which is being used for carrying out an analysis is the first and foremost concern to be addressed by any analyst. In the context of data mining and machine learning, 'Garbage in, Garbage out', is a popular saying while working with large quantities of data.

Commonly, we end up having lot of noisy data; as an example, income: -400 i.e. negative income. Sometimes, we may have unrealistic and impossible combinations of data, for example, in a record, Gender-Male may be entered as Pregnant-Yes. Obviously absurd! because males do not get pregnant. We also suffer due to missing values and other data anomalies. Analyzing such sets of data, that have not been screened before analysis can cause misleading results. Hence, data preprocessing is the first step for any data mining process.

Data preprocessing is a data mining technique that involves transformation of raw data into an understandable format, because real world data can often be incomplete, inconsistent or even erroneous in nature. Data preprocessing resolves such issues. Data preprocessing ensures that further data mining process are free from errors. It is a prerequisite preparation for data mining, it prepares raw data for the core processes.

The University Management System Example

For any University Management System, a correct set of information about their students or vendors is of utmost importance in order to contact them. Hence, accurate and up to date student information is always maintained by a university. Correspondence sent to wrong address would, for instance, lead to a bad impression about the respective university.

Millions of Customer Support Centres across the globe also maintain correct and consistent data about their customers. Imagine a case where a call centre executive is not able to identify the client or customer that he is handling, through his phone number. Such scenarios suggest how reputation is at stake when it comes to accuracy of data. But, on the other hand obtaining the correct details of students or customers is a very challenging task. As most of the data is entered by data entry operators who are not paid well and consequently poorly motivated, these operators make a lot of mistakes while feeding the data into the systems, leading to data which is full of errors. Mistakes like same customer or student's entry might be visible twice in the system. These and other mistakes, are quite common due to data entry negligence. Hence, identification and correction of the data entry done by these operators is one of the basic challenges.

As example, let us consider the records of a vendor providing services to a university stored in different forms in different tables.

Table 4.1 Vendor's record extracted from the first source system

Supplier Name	Jugnoo Food Limited
Address	86 Gandhi Road
City	Indore
State	Madhya Pradesh
PIN	452001
Mobile Number	0731-7766028
Fax	0731-77766022
Email	info@jugnoo.co.in
Owner	Samitra nandan Singh
Last updated	7/12/2017

Table 4.2 Vendor's record extracted from the second source system by Supplier ID

Supplier ID	23234
Business name	JF Limited
Address	855 Gandhi Road
City	Indore
State	Madhya Pradesh
PIN	452001
Telephone	0731-77766028
Fax	0731-77766022
Email	info@jugnoo.co.in
Owner	Samitra N. Singh
Last updated	7/06/2018

Table 4.3 Vendor's record extracted from the third source system

Business name	Jugnoo Food Limited
Address	86 Gandhi Road
City	Indore
State	MP
PIN	Null
Telephone	0731-7766028
Fax	0731-7766022
Email	info@jugnoo.co.in
Owner	Samitra nandan Singh
Postal Address	PO Box 124
Last updated	7/12/2017

There are many errors and missing values in the records extracted from three different source systems. Compare with what the record should look like after it has gone through a preprocessing process.

Table 4.4 Vendor's record after pre-processing

Supplier ID	23234
Business Name	Jugnoo Food Ltd.
Address	86 Gandhi Road
City	Indore
State	Madhya Pradesh
PIN	452001
Postal address	PO Box 124
Telephone	0731-7766028
Fax	0731-7766022
Owner	Samitra Nandan Singh
Last updated	7/06/2018

Table 4.4 is the best information which we could derive from Tables 4.1, 4.2 and 4.3. There is still the possibility that it might contain errors, to remove which further information is required.

Data preprocessing is part of a very complex phase known as ETL (Extraction, Transformation and Loading). It involves extraction of data from multiple sources of data, transforming it to a standardized format and loading it to the data mining system for analysis.

4.2 Data Preprocessing Methods

Raw data is highly vulnerable to missing values, noise and inconsistency and the quality of data affects the data mining results. So, there is a need to improve the quality of data, in order to improve mining results. For achieving better results, raw data is pre-processed so as to enhance its quality and make it error free. This eases the mining process.

As depicted in Figure 4.1 the various stages in which data preprocessing is performed.

- Data Cleaning
- Data Integration
- Data Transformation
- Data Reduction

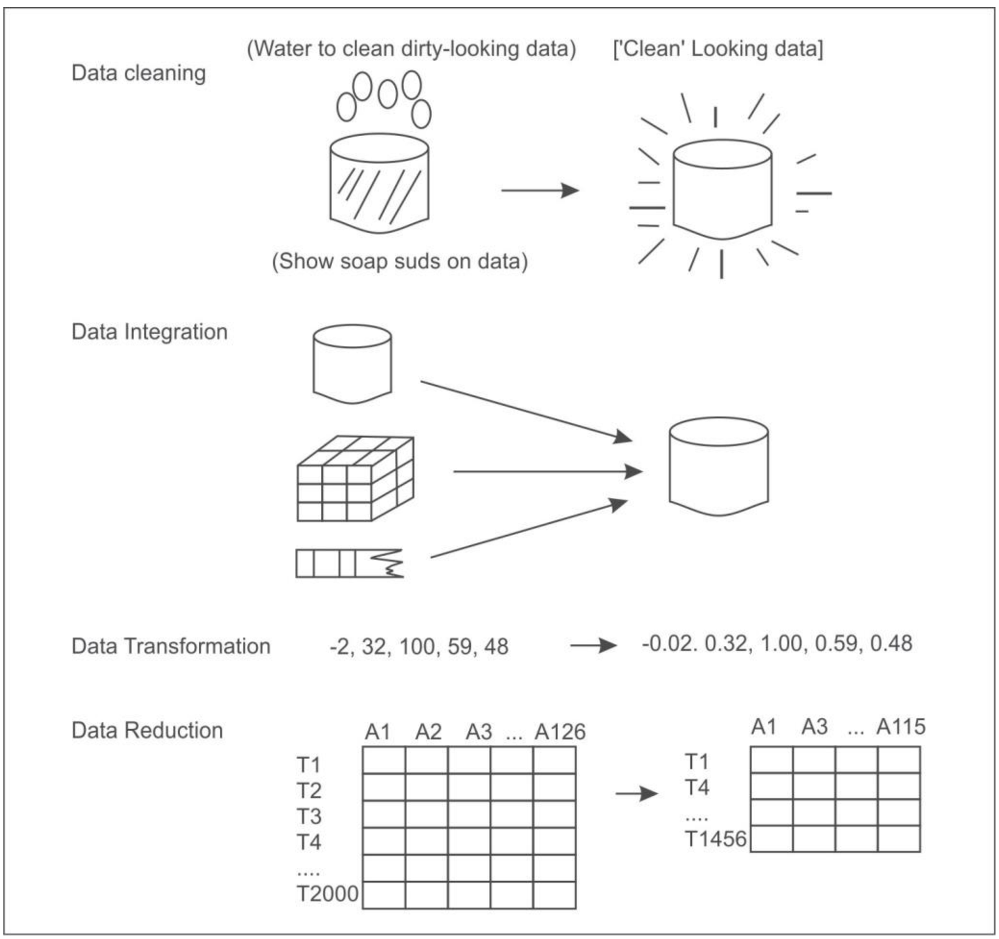


Figure 4.1 Various stages of preprocessing

Let's discuss the details of each phase one by one.

4.2.1 Data cleaning

First the raw data or noisy data goes through the process of cleansing. In data cleansing missing values are filled, noisy data is smoothened, inconsistencies are resolved, and outliers are identified and removed in order to clean the data. To elaborate further:

Handling missing values

It is often found that many of the database tuples or records do not have any recorded values for some attributes. Such cases of missing values are filled by different methods, as described below.

i. *Fill in the missing value manually:* Naturally, manually filling each missing value is laborious and time consuming and so it is practical only when the missing values are few in number. There are other methods to deal with the problem of missing values when the dataset is very large or when the missing values are very many.
ii. *Use of some global constant in place of missing value:* In this method, missing values are replaced by some global label such as 'Unkown' or -∞. Although one of the easiest approaches to deal with the missing values, it should be avoided when mining program presents a pattern due to repetitive occurrences of global labels such as 'Unknown'. Hence, this method should be used with caution.
iii. *Use the attribute mean to fill in the missing value:* Fill in the missing values for each attribute with the mean of other data values of the same attribute. This is a better way to handle missing values in a dataset.
iv. *Use some other value which is high in probability to fill in the missing value:* Another efficient method is to fill in the missing values with values determined by tools such as Bayesian Formalism or Decision Tree Induction or other inference based tools. This is one of the best methods, as it uses most of the information already present to predict the missing values, although it is not biased like previous methods. The only difficulty with this method is complexity in performing the analysis.
v. *Ignore the tuple:* If the tuple contains more than one missing value and all other methods are not applicable, then the best strategy to cope with missing values is to ignore the whole tuple. This is commonly used if the class label goes missing or the tuple contains missing values for most of the attributes. This method should not be used, if the percentage of values that are missing per attribute varies significantly.

So, we can conclude that use the attribute mean to fill in the missing value is most common technique used by most data mining tools to handle the missing values. However, one can always use knowledge of probability to fill these values.

Handling noisy data

Most data mining algorithms are affected adversely due to noisy data. The ***noise*** can be defined as unwanted variance or some random error that occurred in a measurable variable. Noise is removed from the data by the method of 'smoothing'. The methods used for data smoothing are as follows:

i. *Binning methods*

The Binning method is used to divide the values of an attribute into bins or buckets. It is commonly used to convert one type of attribute to another type. For example, it may be necessary to convert a real-valued numeric attribute like temperature to a nominal attribute with values cold, cool, warm, and hot before its processing. This is also called 'discretizing' a numeric attribute. There are two types of discretization, namely, ***equal interval and equal frequency***.

In equal interval binning, we calculate a bin size and then put the samples into the appropriate bin. In equal frequency binning, we allow the bin sizes to vary, with our goal being to choose bin sizes so that every bin has about the same number of samples in it. The idea is that if each bin has the same number of samples, no bin, or sample, will have greater or lesser impact on the results of data mining.

To understand this process, consider a dataset of the marks of 50 students. . The process divides this dataset on the basis of their marks into, for this example, 10 bins.

In case of equal interval binning, we will create bins from 0-10, 10-20, 20-30, 30-40, 40-50, 50-60, 60-70, 70-80, 80-90, 90-100. If most students commonly have marks between 60 to 80, some bins may be full and most bins may have very few entries e.g., 0-10, 10-20, 90-100.

Thus, it might be better to divide this dataset on the equal frequency basis. It means that with the same 50 students in class and we want to put these into 10 bins on the basis of their marks then instead of creating the bins for marks like 0-10, 10-20 and so on, here we will first sort the records of students on the basis of their marks in descending order (or ascending order as we prefer). The first 5 students having highest marks will put into one bin and next 5 students on the basis of their marks will put into another and so on. If our boundary students have same marks then bin range can be shifted to accommodate students with the same marks into one common bin. For example, let us suppose that after arranging the data in descending order of marks and we found that marks of 5th and 6th students are same of 85. Then we cannot put one student in one bin and other in a different bin because both have the same marks. So, we either shift our bin range may be up (i.e., 86 in this case) to accommodate the first 4 students in one bin and next 5 into another. Similarly, we can shift our bin range down to accommodate first 6 students in one bin (i.e., 85 in this case so that 5th and 6th student falls in same bin) and next 5 into another. Thus, in this case of equal frequency most of bins will have a count of approximately 5, while in case of equal interval some bins will be heavily loaded while most will be lightly loaded.

Thus, the idea of having same number of samples in each bin works better as no bin, or sample, will have greater or lesser impact on the results of data mining.

ii. *Clustering or outlier analysis*

Clustering or outlier analysis is a method that allows detection of outliers by clustering. In clustering, values which are common or similar are organized into groups or 'clusters', and those values which lie outside these clusters are termed as outliers or noise.

iii. *Regression*

Regression is another such method which allows data smoothing by fitting it to some function. For example, Linear Regression is one of the most used methods that aims at finding the most suitable line to fit values of two variables or attributes (i.e., best fit). The primary purpose of this is to predict the value of other variable using the first one. Similarly, Multiple Regression is used when more than two variables are involved. Regression allows data fitting which in turn removes noise from data and hence smoothens the dataset using mathematical equations.

iv. *Combined computer and human inspection*

Using both computers and human inspection one can detect suspicious values and outliers.

Handling of inconsistent data

Many times data inconsistencies are encountered when data is recorded during some transaction. Such inconsistencies can be manually removed by using external references. As an example: errors that have been made at the time of data entry be corrected manually by performing a paper trace operation.

4.2.2 Data integration

A most necessary step to be taken during data analysis is Data Integration. Data integration is a process which combines data from a plethora of sources (such as multiple databases, f lat files or data cubes) into a unified data store.

The example of the University Database system discussed under section 4.1 refers to the issues of data integration.

During data integration, a number of tricky issues have to be considered. For example, how does the data analyst or the analyzing machine be sure that student_id of one database and student_number of another database refer to the same entity? This is referred to as the problem of entity identification. Solution to the problem lies with the term 'metadata'. Databases and data warehouses consist of metadata, which is data about data. This metadata is taken as a reference and referred by the data analyst to avoid errors during the process of data integration.

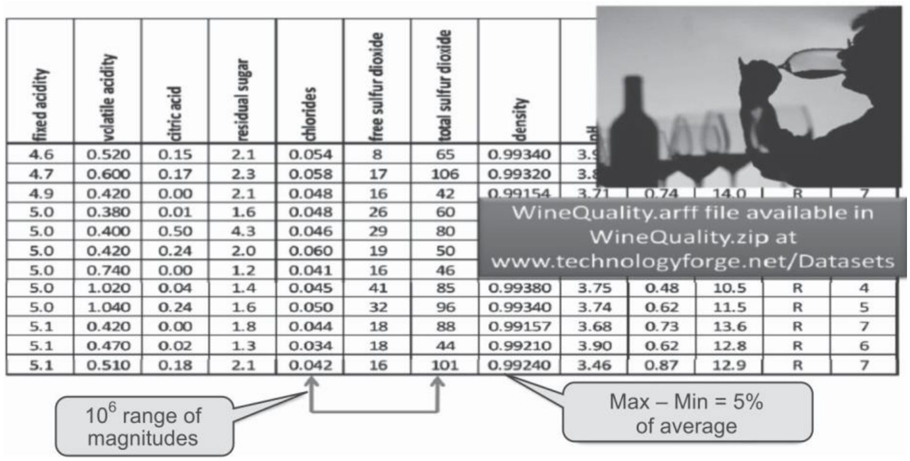
Another such issue which may be caused due to schema integration is redundancy. In the language of database, an attribute is said to be redundant if it is derivable from some other table (of the same database). Mistakes in attribute naming can also lead to data redundancies in the resulting dataset. We use a number of tools to perform data integration from different sources into one unified schema.

4.2.3 Data transformation

When the value of one attribute is small as compared to other attributes, then that attribute will not have much influence on mining of information, since the values of this attribute were smaller than other attributes and the variation within the attribute will also be small.

Thus, data transformation is a process in which data is consolidated or transformed into some other standard forms which are better suited for data mining.

For example, the dataset given in Figure 4.2 is for the chemical composition of wine samples. Note that the values for different attributes cover a range of six orders of magnitude. It turns out that data mining algorithms struggle with numeric attributes that exhibit such ranges of values.



fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH				
4.6	0.520	0.15	2.1	0.054	8	65	0.99340	[illegible]				
4.7	0.600	0.17	2.3	0.058	17	106	0.99320	[illegible]				
4.9	0.420	0.00	2.1	0.048	16	42	0.99154	3.71	0.74	14.0	R	7
5.0	0.380	0.01	1.6	0.048	26	60						
5.0	0.400	0.50	4.3	0.046	29	80						
5.0	0.420	0.24	2.0	0.060	19	50						
5.0	0.740	0.00	1.2	0.041	16	46						
5.0	1.020	0.04	1.4	0.045	41	85	0.99380	3.75	0.48	10.5	R	4
5.0	1.040	0.24	1.6	0.050	32	96	0.99340	3.74	0.62	11.5	R	5
5.1	0.420	0.00	1.8	0.044	18	88	0.99157	3.68	0.73	13.6	R	7
5.1	0.470	0.02	1.3	0.034	18	44	0.99210	3.90	0.62	12.8	R	6
5.1	0.510	0.18	2.1	0.042	16	101	0.99240	3.46	0.87	12.9	R	7

Figure 4.2 Chemical composition of wine samples

All attributes should be transformed to a similar scale for clustering to be effective unless we wish to give more weight to some attributes that are comparatively large in scale. Commonly, we use two techniques to convert the attributes: Normalization and Standardization are the most popular and widely used data transformation methods.

Normalization

In case of normalization, all the attributes are converted to a normalized score or to a range (0, 1). The problem of normalization is an outlier. If there is an outlier, it will tend to crunch all of the other values down toward the value of zero. In order to understand this, let's suppose the range of students' marks is 35 to 45 out of 100. Then 35 will be considered as 0 and 45 as 1, and students will be distributed between 0 to 1 depending upon their marks. But if there is one student having marks 90, then it will act as an outlier and in this case, 35 will be considered as 0 and 90 as 1. Now, it will crunch most of the values down toward the value of zero. In this scenario, the solution is standardization.

Standardization

In case of standardization, the values are all spread out so that we have a standard deviation of 1.

Generally, there is no rule for when to use normalization versus standardization. However, if your data has outliers, use standardization, otherwise use normalization. Using standardization tends to make the remaining values for all of the other attributes fall into similar ranges since all attributes will have the same standard deviation of 1.

4.2.4 Data reduction

It is often seen that when the complex data analysis and mining processes are carried out over humongous datasets, they take such a long time that the whole data mining or analysis process becomes unviable. Data reduction techniques come to the rescue in such situations. Using data reduction techniques a dataset can be represented in a reduced manner without actually compromising the integrity of original data.

Data reduction is all about reducing the dimensions (referring to the total number of attributes) or reducing the volume. Moreover, mining when carried out on reduced datasets often results in better accuracy and proves to be more efficient. There are many methods to reduce large datasets to yield useful knowledge. A few among them are:

i. *Dimension reduction*

In data warehousing, 'dimension' equips us with structured labeling information. But not all dimensions (attributes) are necessary at a time. Dimension reduction uses algorithm such as Principal Component Analysis (PCA) and others. With the usage of such algorithms one can detect and remove redundant and weakly relevant, attributes or dimensions.

ii. *Numerosity reduction*

It is a technique which is used to choose smaller forms of data representation for reducing the dataset volume.

iii. *Data compression*

We can also use data compression techniques to reduce the dataset size. These techniques are classified as lossy and loseless compression techniques where some encoding mechanisms (e.g. Huffman coding) are used.

Some examples of business situations where the classification technique is applied are:

- To analyze the credit history of bank customers to identify if it would be risky or safe to grant them loans.
- To analyze the purchase history of a shopping mall's customers to predict whether they will buy a certain product or not.

In first example, the system will predict a discrete value representing either risky or safe, while in second example, the system will predict yes or no.

Some more examples to distinguish the concept of regression from classification are:

- To predict how much a given customer will spend during a sale.
- To predict the salary-package of a student that he/she may get during his/her placement.

In these two examples, there is a prediction of continuous numeric value. Therefore, both are regression problems.

In this chapter, we will discuss the basic approaches to perform the classification of data.

5.1 Types of Classification

Classification is defined as two types. These are:

- Posteriori classification
- Priori classification

5.1.1 Posteriori classification

The word '*Posteriori*' means something derived by reasoning from the observed facts. It is a supervised machine learning approach, where the target classes are already known, i.e., training data is already labeled with actual answers.

5.1.2 Priori classification

The word '*Priori*' means something derived by reasoning from self-evident propositions. It is an unsupervised machine learning approach, where the target classes are not given. The question is 'Is it possible to make predictions, if labeled data is not available?'

The answer is yes. If data is not labeled, then we can use clustering (unsupervised technique) to divide unlabeled data into clusters. Then these clusters can be assigned some names or labels and can be further used to apply classification to make predictions based on this dataset. Thus, although data is not labeled, we can still make predictions based on data by first applying clustering followed by classification. This approach is known as Priori classification.

In this chapter, we will learn posteriori classification, a supervised machine learning approach to predict the outcome when training dataset is labeled.

5.2 Input and Output Attributes

In any machine learning approach the input data is very important. Data contains two types of attributes, namely, input attributes and output attributes. The class attribute that represents the output of all other attributes is known as an output attribute or dependent attribute, while all other attributes are known as input attributes or independent attributes. For example, the dataset for iris plant's f lowers given in Figure 5.1 has Sepal length, Sepal width, Petal length and Petal width as input attributes and species as an output attribute.

Class

Input Attributes				Output Attribute
Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5	3.6	1.4	0.2	setosa

Numerical Nominal

Figure 5.1 Input and output attributes

The attributes can be of different types. The attributes having numbers are called numerical attributes while attributes whose domain is not numerical are known as nominal or categorical attributes. Here, input attributes are of numerical type while species, i.e., the output attribute is of nominal type (as shown in Figure 5.1).

During classification, it is important to have a database of sufficient size for training the model accurately. The next section covers how a classification model is built.

5.3 Working of Classification

Classification is a two-step process. The first step is training the model and the second step is testing the model for accuracy. In the first step, a classifier is built based on the training data obtained by analyzing database tuples and their associated class labels. By analyzing training data, the system learns and creates some rules for prediction. In the second step, these prediction rules are tested on some unknown instances, i.e., test data. In this step, rules are used to make the predictions about the output attribute or class. In this step, the predictive accuracy of the classifier is calculated. The system performs in an iterative manner to improve its accuracy, i.e., if accuracy is not good on test data, the system will reframe its prediction rules until it gets optimized accuracy on test data as shown in Figure 5.2.

The test data is randomly selected from the full dataset. The tuples of the test data are independent of the training tuples. This means that the system has not been exposed to testing data during the training phase. The accuracy of a classifier on a given test data is defined as the percentage of test data tuples that are correctly classified by the classifier. The associated class label of each test tuple is compared with the class prediction made by the classifier for that particular tuple. If the accuracy of the classifier is satisfactory then it can be used to classify future data tuples with unknown class labels.

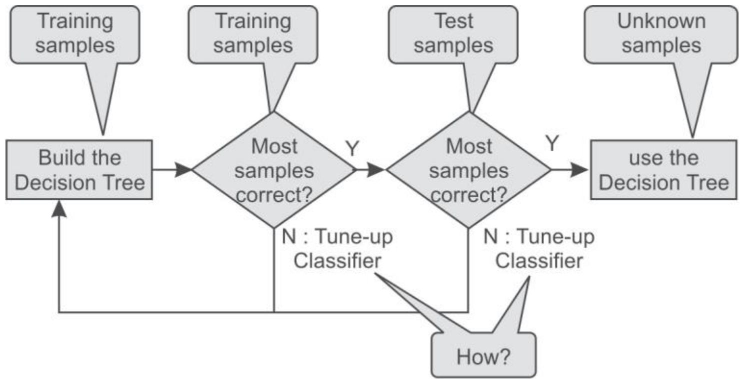


Figure 5.2 Training and testing of the classifier

For example, by analyzing the data of previous loan applications as shown in Figure 5.3, the classification rules obtained can be used to approve or reject the new or future loan applicants as shown in Figure 5.4.

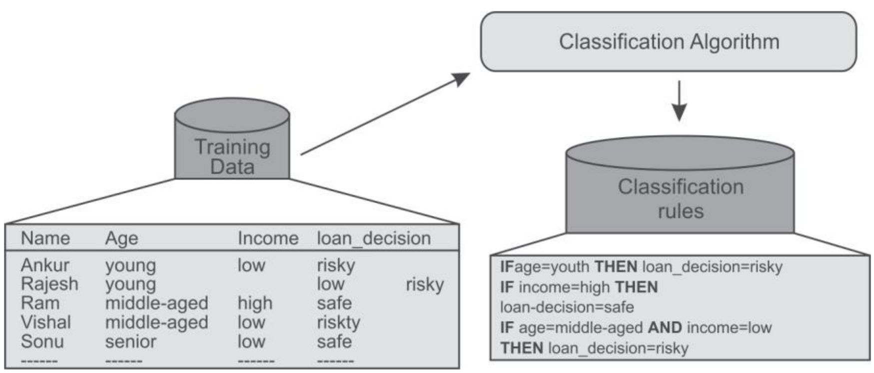


Figure 5.3 Building a classifier to approve or reject loan applications

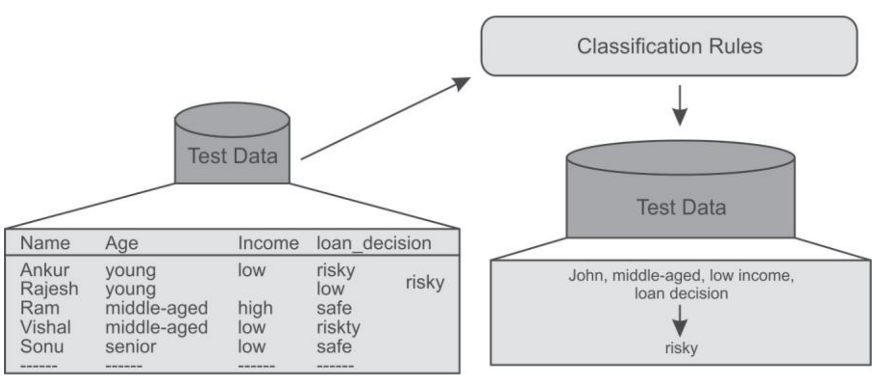


Figure 5.4 Predicting the type of customer based on trained classifier

The same process of training and testing the classifier has also been illustrated in Figure 5.5.

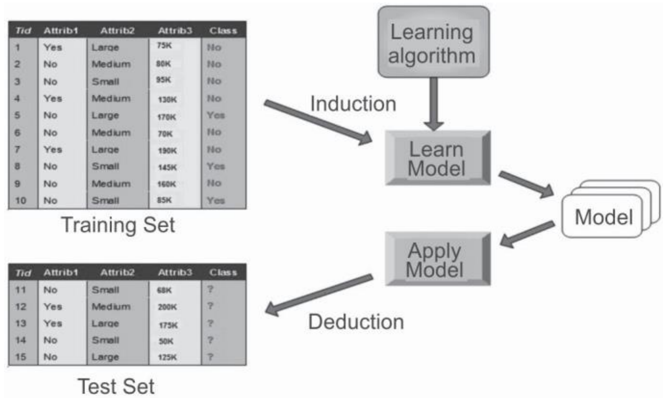


Figure 5.5 Training and testing of the classifier

5.4 Guidelines for Size and Quality of the Training Dataset

There should be a balance between the number of training samples and independent attributes. It has been observed that generally, the number of training samples required is likely to be relatively small if the number of independent or input attributes is small and similarly, number of training samples required is likely to be relatively large if the number of independent or input attributes is large. The quality of the classifier depends upon the quality of the training data. If there are two or more than two classes, then sufficient training data should be available belonging to each of these classes.

Researchers have developed a number of classifiers that include: Decision Tree, Naïve Bayes, Support Vector Machine and Neural Networks. In this chapter, two important classification methods, namely, Decision Tree and Naive Bayes are discussed in detail, as these are widely used by scientists and organizations for classification of data.

5.5 Introduction to the Decision Tree Classifier

In the decision tree classifier, predictions are made by using multiple 'if…then...' conditions which are similar to the control statements in different programming languages, that you might have learnt. The decision tree structure consists of a root node, branches and leaf nodes. Each internal node represents a condition on some input attribute, each branch specifies the outcome of the condition and each leaf node holds a class label. The root node is the topmost node in the tree.

The decision tree shown in Figure 5.6 represents a classifier tasked for predicting whether a customer will buy a laptop or not. Here, each internal node denotes a condition on the input attributes and each leaf node denotes the predicted outcome (class). By traversing the decision tree, one can analyze that if a customer is middle aged then he will probably buy a laptop, if a customer is young and a student then he will probably not buy a laptop. If a customer is a senior citizen and has an excellent credit rating then he can probably buy a laptop. The system makes these predictions with a certain level of probability.

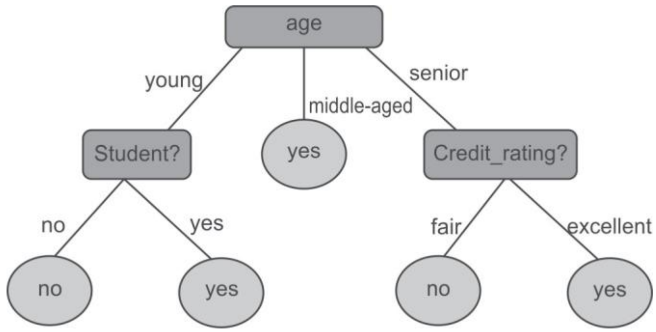


Figure 5.6 Decision tree to predict whether a customer will buy a laptop or not

Decision trees can easily be converted to classification rules in the form of if-then statements. Decision tree based classification is very similar to a '20 questions game'. In this game, one player writes something on a page and other player has to find what was written by asking at most 20 questions; the answers to which can only be yes or no. Here, each node of the decision tree denotes a choice between numbers of alternatives and the choices are binary. Each leaf node specifies a decision or prediction. The training process that produces this tree is known as induction.

5.5.1 Building decision tree

J. Ross Quinlan, a researcher in machine learning, developed a decision tree algorithm known as **ID3 (Iterative Dichotomiser)** during the late 1970s and early 1980s. Quinlan later proposed C4.5 (a successor of ID3), which became a benchmark to which newer supervised learning algorithms are often compared. Decision tree is a common machine learning technique which has been implemented in many machine learning tools like Weka, R, Matlab as well as some programming languages such as Python, Java, *etc*.

These algorithms are based on the concept of Information Gain and Gini Index. So, let us first understand the role of information gain in building the decision tree.

5.5.2 Concept of information theory

Decision tree algorithm works on the basis of information theory. It has been observed that information is directly related with uncertainty. If there is uncertainty then there is information and if there is no uncertainty then there is no information. For example, if a coin is biased having a head on both sides, then the result of tossing it does not give any information but if a coin is unbiased having a head and a tail then the result of the toss provides some information.

Usually the newspaper carries the news that provides maximum information. For example, consider the case of an India-UAE world cup cricket match. It appears certain that India will beat UAE, so this news will not appear on front page as main headlines, but if UAE beats India in a world cup cricket match then this news being very unexpected (uncertain) will appear on the first page as headlines.

Let us consider another example, if in your university or college, there is holiday on Sunday then a notice regarding the same will not carry any information (because it is certain) but if some particular Sunday becomes a working day then it will be information and henceforth becomes a news.

From these examples we can observe that information is related to the probability of occurence of an event. Another important question to consider is, whether the probability of occurrence of an event is more. Then, the information gain will be more frequent or less frequent?

It is certain from above examples that 'more certain' events such as India defeating UAE in cricket or Sunday being a holiday carry very little information. But if UAE beats India or Sunday is working, then even though the probability of these events is lesser than the previous event, it will carry more information. Hence, less probability means more information.

5.5.3 Defining information in terms of probability

Information theory was developed by Claude Shannon. Information theory defines entropy which is average amount of information given by a source of data. Entropy is measured as follows.

$$\text{entropy}(p_1, p_2, \ldots, p_n) = -p_1 \log(p_1) - p_2\log(p_2) - \ldots - p_n\log(p_n)$$

Therefore, the total information for an event is calculated by the following equation:

$$I = \sum_i \left(-p_i \log p_i\right)$$

In this, information is defined as $-p_i \log p_i$ where p_i is the probability of some event. Since, probability p_i is always less than 1, $\log p_i$ is always negative; thus negating *log pi* we get the overall information gain (*–pi log pi*) as positive.

It is important to remember that the logarithm of any number greater than 1 is always positive and the logarithm of any number smaller than 1 is always negative. Logarithm of 1 is always zero, no matter what the base of logarithm is. In case of log with base 2, following are some examples.

$$\log_2(2) = 1$$

$$\log_2(2^n) = n$$

$$\log_2(1/2) = -1$$

$$\log_2(1/2^n) = -n$$

Let us calculate the information for the event of throwing a coin. It has two possible values, i.e., head (p_1) or tail (p_2). In case of unbiased coin, the probability of head and tail is 0.5 respectively. Thus, the information is

$$I = -0.5 \log(0.5) - 0.5 \log(0.5)$$

$$= -(0.5) * (-1) - (0.5) * (-1) \qquad [\text{As, } \log_2(0.5) = -1]$$

$$= 0.5 + 0.5 = 1$$

The result is 1.0 (using log base 2) and it is the maximum information that we can have for an event with two possible outcomes. This is also known as entropy.

But if the coin is biased and has heads on both the sides, then probability for head is 1 while the probability of tails will be 0. Thus, total information in tossing this coin will be as follows.

$$I = -1 \log(1) - 0 \log(0) = 0 \qquad [\text{As, } \log_2(1) = 0]$$

You can clearly observe that tossing of biased coin carries no information while tossing of unbiased coin carries information of 1.

Suppose, an unbiased dice is thrown which has six possible outcomes with equal probability, then the information is given by:

$$I = 6(-1/6)\log(1/6) = 2.585$$

[the probability of each possible outcome is 1/6 and there are in total six possible outcomes from 1 to 6]

But, if dice is biased such that there is a 50% chance of getting a 6, then the information content of rolling the die would be lower as given below.

$$I = 5(-0.1)\log(0.1) - 0.5\log(0.5) = 2.16$$

[One event has a probability of 0.5 while 5 other events has probability of 0.5, which makes 0.5/5 = 0.1 as the probability of each of remaining 5 events.]

And if the dice is further biased such that there is a 75% chance of getting a 6, then the information content of rolling the die would be further low as given below.

$$I = 5(-0.05)\log(0.05) - 0.75\log(0.75) = 1.39$$

[One event has a probability of 0.75 while 5 other events has probability of 0.25, which makes 0.25/5 = 0.05 as probability of each of remaining 5 events.]

We can observe that as the certainty of an event goes up, the total information goes down.

Information plays a key role in selecting the root node or attribute for building a decision tree. In other words, selection of a *split attribute* plays an important role. Split attribute is an attribute that reduces the uncertainty by largest amount, and is always accredited as a root node. So, the attribute must distribute the objects such that each attribute value results in objects that have as little uncertainty as possible. Ideally, each attribute value should provide us with objects that belong to only one class and therefore have zero information.

5.5.4 Information gain

Information gain specifies the amount of information that is gained by knowing the value of the attribute. It measures the 'goodness' of an input attribute for predicting the target attribute. The attribute with the highest information gain is selected as the next split attribute.

Mathematically, it is defined as the entropy of the distribution before the split minus the entropy of the distribution after split.

Information gain = (Entropy of distribution before the split)
– (Entropy of distribution after the split)

The largest information gain is equivalent to the smallest entropy or minimum information. It means that if the result of an event is certain, i.e., the probability of an event is 1 then information provided by it is zero while the information gain will be the largest, thus it should be selected as a split attribute.

Assume that there are two classes, P and N, and let the set of training data S (with a total number of records s) contain p records of class P and n records of class N. The amount of information is defined as

$$I = -(p/s)\log(p/s) - (n/s)\log(n/s)$$

Obviously if $p = n$, i.e., the probability is equally distributed then I is equal to 1 and if $p = s$ *or* $n = 0$, i.e., training data S contains all the elements of a single class only, then $I = 0$. Therefore if there was an attribute for which all the records had the same value (for example, consider the attribute gender, when all people are male), using this attribute would lead to no information gain that is, no reduction in uncertainty. On the other hand, if an attribute divides the training sample such that all female records belong to Class *A*, and male records belong to Class *B*, then uncertainty has been reduced to zero and we have a large information gain.

Thus after computing the information gain for every attribute, the attribute with the highest information gain is selected as split attribute.

5.5.5 Building a decision tree for the example dataset

Let us build decision tree for the dataset given in Figure 5.7.

Instance Number	*X*	*Y*	*Z*	*Class*
1	1	1	1	A
2	1	1	0	A
3	0	0	1	B
4	1	0	0	B

X	*Y*	*Z*	*Class*
1 = 3	1 = 2	1 = 2	A = 2
0 = 1	0 = 2	0 = 2	B = 2

Figure 5.7 Dataset for class C prediction based on given attribute condition

The given dataset has three input attributes X, Y, Z and one output attribute Class. The instance number has been given to show that the dataset contains four records (basically for convenience while making references). The output attribute or class can be either A or B.

There are two instances for each class so the frequencies of these two classes are given as follows:

A = 2 (Instances 1, 2)

B = 2(Instances 3, 4)

The amount of information contained in the whole dataset is calculated as follows:

I = - probability for Class A * log (probability for class A)

- probability for class B * log (probability for class N)

Here, probability for class A = (Number of instances for class A/Total number of instances) = 2/4
And probability for class B = (Number of instances for class B/Total number of instances) = 2/4
Therefore, I = (-2/4) log (2/4) – (2/4) log (2/4) = 1

Let us consider each attribute one by one as a split attribute and calculate the information for each attribute.

Attribute 'X'

As given in the dataset, there are two possible values of X, i.e., 1 or 0. Let us analyze each case one by one.

For X= 1, there are 3 instances namely instance 1, 2 and 4. The first two instances are labeled as class A and the third instance, i.e, record 4 is labeled as class B.

For X = 0, there is only 1 instance, i.e, instance number 3 which is labeled as class B.

Given the above values, let us compute the information given by this attribute. We divide the dataset into two subsets according to X either being 1 or 0. Computing information for each case,

$$I\ (\text{for } X = 1) = I(X1) = -\ (2/3)\log(2/3) - (1/3)\log(1/3) = 0.92333$$

$$I\ (\text{for } X = 0) = I(X0) = -\ (0/1)\log(0/1) - (1/1)\log(1/1) = 0$$

Total information for above two sub-trees = probability for X having value 1 * I(X1) + probability for X having value 0 * I(X0)

Here, probability for X having value 1 = (Number of instances for X having value 1/Total number of instances) = 3/4

And probability for X having value 0 = (Number of instances for X having value 0/Total number of instances) = 1/4

Therefore, total information for the two sub-trees = (3/4) I(X1) + (1/4) I(X0)
= 0.6925 + 0
= 0.6925

Attribute 'Y'

There are two possible values of Y attribute, i.e., 1 or 0. Let us analyze each case one by one.

There are 2 instances where Y has value 1. In both cases when Y=1 the record belongs to class A and, in the 2 instances when Y = 0 both records belong to class B.

Given the above values, let us compute the information provided by Y attribute. We divide the dataset into two subsets according to Y either being 1 and 0. Computing information for each case,

$$I\ (\text{For } Y = 1) = I\ (Y1) = -\ (2/2)\log(2/2) - (0/2)\log(0/2) = 0$$

$$I\ (\text{For } Y = 0) = I\ (Y0) = -\ (0/2)\log(0/2) - (2/2)\log(2/2) = 0$$

Total information for the two sub-trees = probability for Y having value 1* I(Y1) + probability for Y having value 0 * I(Y0)

Here, probability for Y in 1 = (Number of instances for Y having 1/Total number of instances) = 2/4

And probability for Y in 0 = (Number of instances for Y having 0/Total number of instances) = 2/4

Therefore, the total information for the two sub-trees = (2/4) I(Y1) + (2/4) I(Y0)
= 0 + 0
= 0

Attribute 'Z'

There are two possible values of Z attribute, i.e., 1 or 0. Let us analyze each case one by one.

There are 2 instances where Z has value 1 and 2 instances where Z has value 0. In both cases, there exists a record belonging to class A and class B with Z is either 0 or 1.

Given the above values, let us compute the information provided by the Z attribute. We divide the dataset into two subsets according to Z either being 1 or 0. Computing information for each case,

$$I\ (\text{For } Z = 1) = I\ (Z1) = -\ (1/2)\log(1/2) - (1/2)\log(1/2) = 1.0$$

$$I\ (\text{For } Z = 0) = I\ (Z0) = -\ (1/2)\log(1/2) - (1/2)\log(1/2) = 1.0$$

Total information for the two sub-trees = probability for Z having value 1 * I(Z1) + probability for Z having value 0 * I(Z0)

Here, probability for Z having value 1 = (Number of instances for Z having value 1/Total number of instances) = 2/4

And probability for Z having value 0 = (Number of instances for Z having value 0/Total number of instances) = 2/4

Therefore, total information for two sub-trees = (2/4) I(Z1) + (2/4) I(Z0)

= 0.5 + 0.5

= 1.0

The Information gain can now be computed:

Potential Split attribute	*Information before split*	*Information after split*	*Information gain*
X	1.0	0.6925	0.3075
Y	**1.0**	**0**	**1.0**
Z	1.0	1.0	0

Hence, the largest information gain is provided by the attribute 'Y' thus it is used for the split as depicted in Figure 5.8.

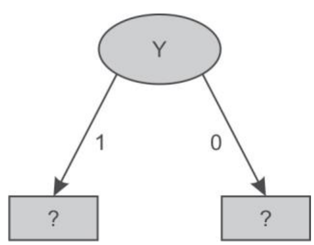


Figure 5.8 Data splitting based on Y attribute

For Y, as there are two possible values, i.e., 1 and 0, therefore the dataset will be split into two subsets based on distinct values of the Y attribute as shown in Figure 5.8.

Dataset for Y = '1'

Instance Number	*X*	*Z*	*Class*
1	1	1	A
2	1	0	A

There are 2 samples and the frequency of each class is as follows.

A = 2 (Instances 1, 2)
B = 0 Instances

Information of the whole dataset on the basis of class is given by

$$I = (-2/2) \log (2/2) - (0/2) \log(0/2) = 0$$

As it represents the same class 'A' for all recorded combinations of X and Z, therefore, it represents class 'A' as shown in Figure 5.9.

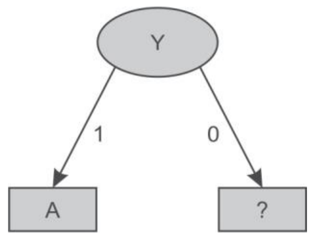


Figure 5.9 Decision tree after splitting of attribute Y having value '1'

Dataset for Y = '0'

Instance Number	X	Z	Class
3	0	1	B
4	1	0	B

For Y having value 0, it represents the same class 'B' for all the records. Thus, the decision tree will look like as shown in Figure 5.10 after analysis of Y dataset.

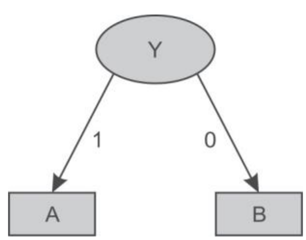


Figure 5.10 Decision tree after splitting of attribute Y value '0'

Let us consider another example and build a decision tree for the dataset given in Figure 5.11. It has 4 input attributes outlook, temperature, humidity and windy. As before we have added instance number for explanation purposes. Here, 'play' is the output attribute and these 14 records contain the information about weather conditions based on which it was decided if a play took place or not.

Instance Number	*Outlook*	*Temperature*	*Humidity*	*Windy*	*Play*
1	sunny	hot	high	false	No
2	sunny	hot	high	true	No
3	overcast	hot	high	false	Yes
4	rainy	mild	high	false	Yes
5	rainy	cool	normal	false	Yes
6	rainy	cool	normal	true	No
7	overcast	cool	normal	true	Yes
8	sunny	mild	high	false	No
9	sunny	cool	normal	false	Yes
10	rainy	mild	normal	false	Yes
11	sunny	mild	normal	true	Yes
12	overcast	mild	high	true	Yes
13	overcast	hot	normal	false	Yes
14	rainy	mild	high	true	No

Attribute values and counts

Outlook	Temp.	Humidity	Windy	Play
sunny = 5	hot = 4	high = 7	true = 6	yes = 9
overcast = 4	mild = 6	normal = 7	false = 8	no = 5
rainy = 5	cool = 4			

Figure 5.11 Dataset for play prediction based on given day weather conditions

In the dataset, there are 14 samples and two classes for target attribute 'Play', i.e., Yes or No. The frequencies of these two classes are given as follows:

Yes = 9 (Instance number 3,4,5,7,9,10,11,12,13 and 14)

No = 5 (Instance number 1,2,6,8 and 15)

Information of the whole dataset on the basis of whether play is held or not is given by

I = - probability for play being held * log (probability for play being held) - probability for play not being held * log (probability for play not being held)

Here, probability for play having value Yes = (Number of instances for Play is Yes/Total number of instances) = 9/14

And probability for play having value No = (Number of instances for Play is No/Total number of instances) = 5/14

Therefore, I = - (9/14) log (9/14) – (5/14) log (5/14) = 0.9435142

Let us consider each attribute one by one as split attributes and calculate the information for each attribute.

Attribute 'Outlook'

As given in dataset, there are three possible values of outlook, i.e., sunny, overcast and rainy. Let us analyze each case one by one.

There are 5 instances where outlook is sunny. Out of these 5 instances, in 2 instances (9 and 11) the play is held and in remaining 3 instances (1, 2 and 8) the play is not held.

There are 4 instances where outlook is overcast and in all these instances the play always takes place.

There are 5 instances where outlook is rainy. Out of these 5 instances, in 3 instances (4, 5 and 10) the play is held whereas in remaining 2 instances (6 and 14) the play is not held.

Given the above values, let us compute the information provided by the outlook attribute. We divide the dataset into three subsets according to outlook conditions being sunny, overcast or rainy. Computing information for each case,

$$\text{I (Sunny)} = \text{I (S)} = -\,(2/5)\log(2/5) - (3/5)\log(3/5) = 0.97428$$

$$\text{I (Overcast)} = \text{I (O)} = -\,(4/4)\log(4/4) - (0/4)\log(0/4) = 0$$

$$\text{I (Rainy)} = \text{I (R)} = -\,(3/5)\log(3/5) - (2/5)\log(2/5) = 0.97428$$

Total information for these three sub-trees = probability for outlook Sunny * I(S) + probability for outlook Overcast * I(O) + probability for outlook Rainy * I(R)

Here, probability for outlook Sunny = (Number of instances for outlook Sunny/Total number of instances) = 5/14

And probability for outlook Overcast = (Number of instances for outlook Overcast/Total number of instances) = 4/14

And probability for outlook Rainy = (Number of instances for outlook Rainy/Total number of instances) = 5/14

Therefore, total information for three sub-trees = (5/14) I(S) + (4/14) I(O) + (5/14) I(R)

$$= 0.3479586 + 0.3479586$$
$$= 0.695917$$

Attribute 'Temperature'

There are three possible values of the Temperature attribute, i.e., Hot, Mild and Cool. Let us analyze each case one by one.

There are 4 instances for Temperature hot. Play is held in case of 2 of these instances (3 and 13) and is not held in case of the other 2 instances (1 and 2).

There are 6 instances for Temperature mild. Play is held in case of 4 instances (4, 10, 11 and 12) and is not held in case of 2 instances (8 and 14).

There are 4 instances for Temperature cool. Play is held in case of 3 instances (5, 7 and 9) and is not held in case of 1 instance (6).

Given the above values, let us compute the information provided by Temperature attribute. We divide the dataset into three subsets according to temperature conditions being Hot, Mild or Cool. Computing information for each case,

$$\text{I (Hot)} = \text{I (H)} = -\,(2/4)\log(2/4) - (2/4)\log(2/4) = 1.003433$$

$$\text{I (Mild)} = \text{I (M)} = -\,(4/6)\log(4/6) - (2/6)\log(2/6) = 0.9214486$$

$$I\ (\text{Cool}) = I\ (C) = -\ (3/4)\ \log(3/4) - (1/4)\ \log(1/4) = 0.814063501$$

Total information for the three sub-trees = probability for temperature hot * I(H) + probability for temperature mild * I(M) + probability for temperature cool * I(C)

Here, probability for temperature hot = (Number of instances for temperature hot/Total number of instances) = 4/14

And probability for temperature mild = (Number of instances for temperature mild/Total number of instances) = 6/14

And probability for temperature cool = (Number of instances for temperature cool/Total number of instances) = 4/14

Therefore, total information for these three sub-trees

$$\begin{aligned} &= (4/14)\ I(H) + (6/14)\ I(M) + (4/14)\ I(C) \\ &= 0.2866951429 + 0.3949065429 + 0.23258957 \\ &= 0.9141912558 \end{aligned}$$

Attribute 'Humidity'

There are two possible values of the Humidity attribute, i.e., High and Normal. Let us analyze each case one by one.

There are 7 instances where humidity is high. Play is held in case of 3 instances (3, 4 and 12) and is not held in case of 4 instances (1, 2, 8 and 14).

There are 7 instances where humidity is normal. Play is held in case of 6 instances (5, 7, 9, 10, 11 and 13) and is not held in case of 1 instance (6).

Given the above values, let us compute the information provided by the humidity attribute. We divide the dataset into two subsets according to humidity conditions being high or normal. Computing information for each case,

$$I\ (\text{High}) = I\ (H) = -\ (3/7)\ \log(3/7) - (4/7)\ \log(4/7) = 0.98861$$

$$I\ (\text{Normal}) = I\ (N) = -\ (6/7)\ \log(6/7) - (1/7)\ \log(1/7) = 0.593704$$

Total information for the two sub-trees = probability for humidity high * I(H) + probability for humidity normal * I(N)

Here, probability for humidity high = (Number of instances for humidity high/Total number of instances) = 7/14

And probability for humidity normal = (Number of instances for humidity normal/Total number of instances) = 7/14

Therefore, Total information for the two sub-trees = (7/14) I(H) + (7/14) I(N)

$$\begin{aligned} &= 0.494305 + 0.29685 \\ &= 0.791157 \end{aligned}$$

Attribute 'Windy'

There are two possible values for this attribute, i.e., true and false. Let us analyze each case one by one.

There are 6 instances when it is windy. On windy days, play is held in case of 3 instances (7, 11 and 12) and is not held in case of remaining 3 instances (2, 6 and 14).

For non-windy days, there are 8 instances. On non-windy days, the play is held in case of 6 instances (3, 4, 5, 9, 10 and 13) and is not held in case of 2 instances (1 and 8).

Given the above values, let us compute the information provided by the windy attribute. We divide the dataset into two subsets according to windy being true or false. Computing information for each case,

$$I\ (True) = I\ (T) = -\ (3/6)\log(3/6) - (3/6)\log(3/6) = 1.003433$$

$$I\ (False) = I\ (F) = -\ (6/8)\log(6/8) - (2/8)\log(2/8) = 0.81406$$

Total information for the two sub-trees = probability for windy true * I(T) + probability for windy true * I(F)

Here, The probability for windy being True = (Number of instances for windy true/Total number of instances) = 6/14

And probability for windy being False = (Number of instances for windy false/Total number of instances) = 8/14

Therefore, Total information for the two sub-trees = (6/14) I(T) + (8/14) I(F)

= 0.4300427 + 0.465179

= 0.89522

The information gain can now be computed:

Potential Split attribute	*Information before split*	*Information after split*	*Information gain*
Outlook	0.9435	0.6959	0.2476
Temperature	0.9435	0.9142	0.0293
Humidity	0.9435	0.7912	0.15234
Windy	0.9435	0.8952	0.0483

From the above table, it is clear that the largest information gain is provided by the attribute 'Outlook' so it is used for the split as depicted in Figure 5.12.

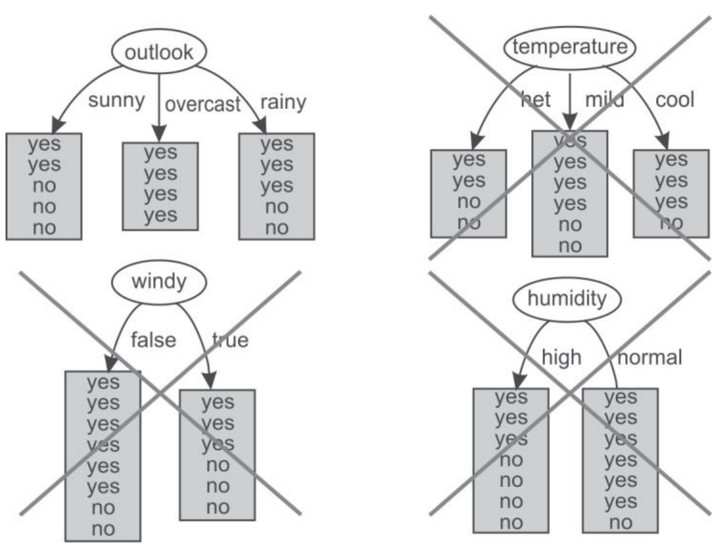


Figure 5.12 Selection of Outlook as root attribute

For Outlook, as there are three possible values, i.e., Sunny, Overcast and Rain, the dataset will be split into three subsets based on distinct values of the Outlook attribute as shown in Figure 5.13.

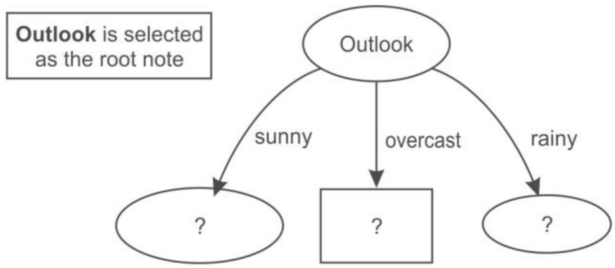


Figure 5.13 Data splitting based on the Outlook attribute

Dataset for Outlook 'Sunny'

Instance Number	*Temperature*	*Humidity*	*Windy*	*Play*
1	Hot	high	false	No
2	Hot	high	true	No
3	Mild	high	false	No
4	Cool	normal	false	Yes
5	Mild	normal	true	Yes

Again, in this dataset, a new column instance number is added to the dataset for making explanation easier. In this case, we have three input attributes Temperature, Humidity and Windy. This dataset consists of 5 samples and two classes, i.e., Yes and No for the Play attribute. The frequencies of classes are as follows:

Yes = 2 (Instances 4, 5)

No = 3 (Instances 1, 2, 3)

Information of the whole dataset on the basis of whether play is held or not is given by

I = - probability for Play Yes * log (probability for Play Yes)
- probability for Play No * log (probability for Play No)

Here, probability for play being Yes = (Number of instances for Play Yes/Total number of instances) = 2/5

And probability for Play being No = (Number of instances for Play No/Total number of instances) = 3/5

Therefore, I = (-2/5) log (2/5) – (3/5) log(3/5) = 0.97

Let us consider each attribute one by one as a split attribute and calculate the information for each attribute.

Attribute 'Temperature'

There are three possible values of Temperature attribute, i.e., hot, mild and cool. Let us analyze each case one by one.

There are 2 instances for temperature hot. Play is never held when the temperature is hot as shown in 2 instances (1 and 2).

There are 2 instances when temperature is mild. Play is held in case of 1 instance (5) and is not held in case of 1 instance (3).

There is 1 instance when temperature is cool. Play is held in this case as shown in instance 4.

Given the above values, let us compute the information provided by this attribute. We divide the dataset into three subsets according to temperature conditions being hot, mild or cool. Computing information for each case,

$$I\ (\text{Hot}) = I\ (H) = -\ (0/2)\log(0/2) - (2/2)\log(2/2) = 0$$

$$I\ (\text{Mild}) = I\ (M) = -\ (1/2)\log(1/2) - (1/2)\log(1/2) = 1.003433$$

$$I\ (\text{Cool}) = I\ (C) = -\ (1/1)\log(1/1) - (0/1)\log(1/1) = 0$$

Total information for these three sub-trees = probability for temperature hot * I(H) + probability for temperature mild * I(M) + probability for temperature cool * I(C)

Here, probability for temperature hot = (No of instances for temperature hot/Total no of instances) = 2/5

And probability for temperature mild = (Number of instances for temperature mild/Total number of instances) = 2/5

And probability for temperature cool = (Number of instances for temperature cool/Total number of instances) = 1/5

Therefore, total information for three subtrees = (2/5) I(H) + (2/5) I(M) + (1/5) I(C)
= 0 + (0.4) * (1.003433) + 0
= 0.40137

Attribute 'Humidity'

There are two possible values of Humidity attribute, i.e., High and Normal. Let us analyze each case one by one.

There are 3 instances when humidity is high. Play is never held when humidity is high as shown in case of 3 instances (1, 2 and 3).

There are 2 instances when humidity is normal. Play is always held as shown in case of 2 instances (4 and 5).

Given the above values, let us compute the information provided by this attribute. We divide the dataset into two subsets according to humidity conditions being high or normal. Computing information for each case,

$$I\ (\text{High}) = I\ (H) = -(0/3)\log(0/3) - (3/3)\log(3/3) = 0$$

$$I\ (\text{Normal}) = I\ (N) = -(2/2)\log(2/2) - (0/2)\log(0/2) = 0$$

Total information for these two sub-trees = probability for humidity high * I(H) + probability for humidity normal * I(N)

Here, probability for humidity high = (Number of instances for humidity high/Total number of instances) = 3/5

And probability for humidity normal = (Number of instances for humidity normal/Total number of instances) = 2/5

Therefore, total information for these two sub-trees = (3/5) I(H) + (2/5) I(N) = 0

Attribute 'Windy'

There are two possible values for this attribute, i.e., true and false. Let us analyze each case one by one.

There are 2 instances when windy is true. On windy days play is held in case of 1 instance (5) and it is not held in case of another 1 instance (2).

There are 3 instances when windy is false. The play is held in case of 1 instance (4) and is not held in case of 2 instances (1 and 3).

Given the above values, let us compute the information by using this attribute. We divide the dataset into two subsets according to windy being true or false. Computing information for each case,

$$I\ (\text{True}) = I\ (T) = -\ (1/2)\ \log(1/2) - (1/2)\ \log(1/2) = 1.003433$$

$$I\ (\text{False}) = I\ (F) = -\ (1/3)\ \log(1/3) - (2/3)\ \log(2/3) = 0.9214486$$

Total information for these two sub-trees = probability for windy true * I(T) + probability for windy true * I(F)

Here, the probability for windy true = (Number of instances for windy true/Total number of instances) = 2/5

And probability for windy false = (Number of instances for windy false/Total number of instances) = 3/5

Therefore, total information for two sub-trees = (2/5) I(T) + (3/5) I(F)
= 0.40137 + 0.55286818
= 0.954239

The Information gain can now be computed:

Potential Split attribute	*Information before split*	*Information after split*	*Information gain*
Temperature	0.97	0.40137	0.56863
Humidity	0.97	0	0.97
Windy	0.97	0.954239	0.015761

Thus, the largest information gain is provided by the attribute 'Humidity' and it is used for the split. This algorithm can be tuned by stopping when we get the 0 value of information as in this case to reduce the number of calculations for big datasets. Now, the Humidity attribute is selected as split attribute as depicted in Figure 5.14.

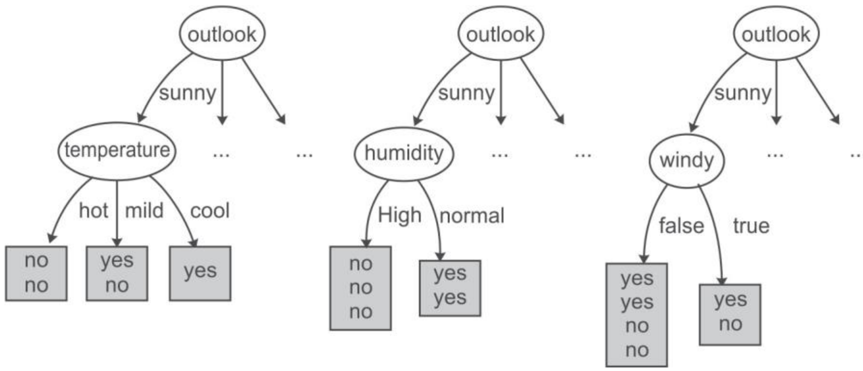


Figure 5.14 Humidity attribute is selected from dataset of Sunny instances

There are two possible values of humidity so data is split into two parts, i.e. humidity 'high' and humidity 'low' as shown below.

Dataset for Humidity 'High'

Instance Number	*Temperature*	*Windy*	*Play*
1	hot	false	No
2	hot	true	No
3	mild	false	No

Again, in this dataset a new column instance number has been introduced for explanation purposes. For this dataset, we have two input attributes Temperature and Windy. As the dataset represents the same class 'No' for all the records, therefore for Humidity value 'High' the output class is always 'No'.

Dataset for Humidity 'Normal'

Instance No	*Temperature*	*Windy*	*Play*
1	cool	false	Yes
2	mild	true	Yes

When humidity's value is 'Normal' the output class is always 'Yes'. Thus, decision tree will look like as shown in Figure 5.15 after analysis of the Humidity dataset.

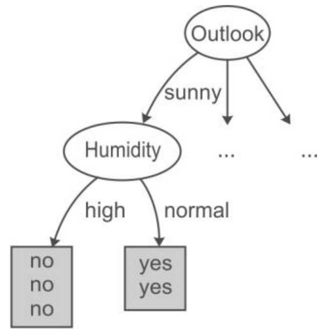


Figure 5.15 Decision tree after spitting of data on Humidity attribute

Now, the analysis of Sunny dataset is over. From the decision tree shown in Figure 5.15, it has been analyzed that if the outlook is 'Sunny' and humidity is 'normal' then play is always held while on the other hand if the humidity is 'high' then play is not held.

Let us take next subset which has Outlook as 'Overcast' for further analysis.

Dataset for Outlook 'Overcast'

Instance Number	*Temperature*	*Humidity*	*Windy*	*Play*
1	hot	high	false	Yes
2	cool	normal	true	Yes
3	mild	high	true	Yes

For outlook Overcast, the output class is always 'Yes'. Thus, decision tree will look like Figure 5.16 after analysis of the overcast dataset.

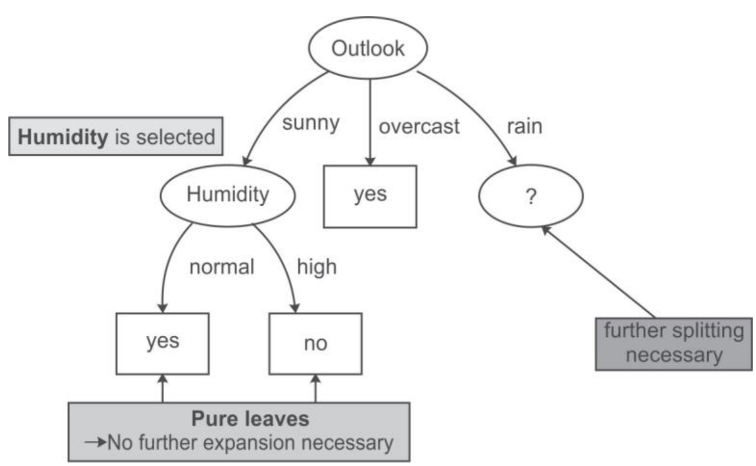


Figure 5.16 Decision tree after analysis of Sunny and Overcast dataset

Therefore, it has been concluded that if the outlook is 'Overcast' then play is always held. Now we have to select another split attribute for the outlook rainy and subset of the dataset for this is given below.

Dataset for Outlook 'Rainy'

Instance Number	*Temperature*	*Humidity*	*Windy*	*Play*
1	Mild	High	False	Yes
2	Cool	Normal	False	Yes
3	Cool	Normal	True	No
4	Mild	Normal	False	Yes
5	Mild	High	True	No

In the above dataset, a new column for instance numbers has again been added for ease of explanation. This dataset consists of 5 samples having input attributes Temperature, Humidity and Windy; and the single output attribute Play has two classes, i.e., Yes and No. The frequencies of these classes are as follows:

Yes = 3 (Instances 1, 2, 4)

No = 2 (Instances 3, 5)

Information of the whole dataset on the basis of whether play is held or not is given by

I = - probability for Play Yes * log (probability for Play Yes)

- probability for Play No * log (probability for Play No)

Here, probability for Play Yes = (Number of instances for Play Yes/Total number of instances) = 3/5

And probability for Play No = (Number of instances for Play No/Total number of instances) = 2/5

Therefore, I = (-3/5) log (3/5) – (2/5) log (2/5) = 0.97428

Let us consider each attribute one by one as split attributes and calculate the information provided for each attribute.

Attribute 'Temperature'

There are two possible values of Temperature attribute, i.e., mild and cool. Let us analyze each case one by one.

There are 3 instances with temperature value mild. Play is held in case of 2 instances (1 and 4) and is not held in case of 1 instance (5).

There are 2 instances with temperature value cool. Play is held in case of 1 instance (2) and is not held in case of other instance (3).

Given the above values, let us compute the information provided by this attribute. We divide the dataset into two subsets according to temperature being mild and cool. Computing information for each case, I (Mild) = I(M) = - (2/3) log(2/3) - (1/3) log(1/3) = 0.921545

I (Cool) = I(C) = - (1/2) log(1/2) - (1/2) log(1/2) = 1.003433

Total information for the two sub-trees = probability for temperature mild * I(M) + probability for temperature cool * I(C)

And probability for temperature mild = (Number of instances for temperature mild/Total number of instances) = 3/5

And probability for temperature cool = (Number of instances for temperature cool/Total number of instances) = 2/5

Therefore, total information for three subtrees = (3/5) I(M) + (2/5) I(C)
= 0.552927 + 0.40137
= 0.954297

Attribute 'Humidity'

There are two possible values of Humidity attribute, i.e., High and Normal. Let us analyze each case one by one.

There are 2 instances with high humidity. Play is held in case of 1 instance (1) and is not held in case of another instance (5).

There are 3 instances with normal humidity. The play is held in case of 2 instances (2 and 4) and is not held in case of single instance (3).

Given the above values, let us compute the information provided by this attribute. We divide the dataset into two subsets according to humidity being high or normal. Computing information for each case,

I (High) = I (H) = - (1/2) log(1/2) - (1/2) log(1/2)= 1.0

I (Normal) = I (N) = -(2/3) log(2/3)-(1/3) log(1/3)= 0.9187

Total information for the two sub-trees = probability for humidity high * I(H) + probability for humidity normal * I(N)

Here, probability for humidity high = (Number of instances for humidity high/Total number of instances) = 2/5

And probability for humidity normal = (Number of instances for humidity normal/Total number of instances) = 3/5

Therefore, total information for the two sub-trees = (2/5) I(H) + (3/5) I(N)
= 0.4 + 0.5512 = 0.9512

Attribute 'Windy'

There are two possible values for this attribute, i.e., true and false. Let us analyze each case one by one.

There are 2 instances where windy is true. Play is not held in case of both of the 2 instances (3 and 5).

For non-windy days, there are 3 instances. Play is held in all of the 3 instances (1, 2 and 4).

Given the above values, let us compute the information provided by this attribute. We divide the dataset into two subsets according to windy being true or false. Computing information for each case,

$$I\ (True) = I\ (T) = -\ (0/2)\ \log(0/2) - (2/2)\ \log(2/2) = 0$$

$$I\ (False) = I\ (F) = -\ (3/3)\ \log(3/3) - (0/3)\ \log(0/3) = 0$$

Total information for the two sub-trees = probability for windy true * I(T) + probability for windy false * I(F)

Here, probability for windy true = (Number of instances for windy true/Total number of instances) = 2/5

And, probability for windy false = (Number of instances for windy false/Total number of instances) = 3/5

Therefore, total information for the two sub-trees = (2/5) I(T) + (3/5) I(F) = 0

The Information gain can now be computed:

Potential Split attribute	*Information before split*	*Information after split*	*Information gain*
Temperature	0.97428	0.954297	0.019987
Humidity	0.97428	0.9512	0.02308
Windy	0.97428	0	0.97428

Hence, the largest information gain is provided by the attribute 'Windy' and this attribute is used for the split.

Dataset for Windy 'TRUE'

Instance Number	*Temperature*	*Humidity*	*Play*
1	cool	normal	No
2	mild	high	No

From above table it is clear that for Windy value 'TRUE', the output class is always 'No'.

Dataset for Windy 'FALSE'

Instance Number	*Temperature*	*Humidity*	*Play*
1	Mild	High	Yes
2	Cool	Normal	Yes
3	Mild	Normal	Yes

Also for Windy value 'FALSE', the output class is always 'Yes'. Thus, the decision tree will look like Figure 5.17 after analysis of the rainy attribute.

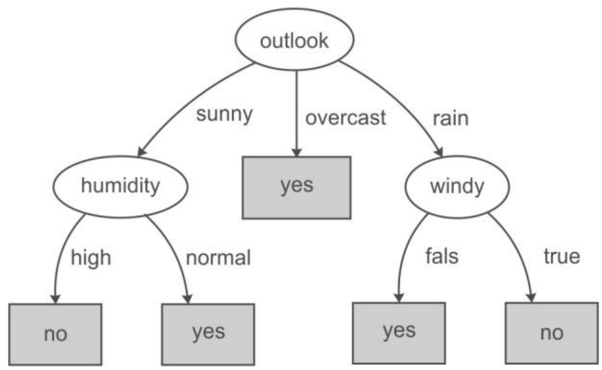


Figure 5.17 Decision tree after analysis of Sunny, Overcast and Rainy dataset

We have concluded that if the outlook is 'Rainy' and value of windy is 'False' then play is held and on the other hand, if value of windy is 'True' then it means that play is not held in that case.

Figure 5.18 represents the final tree view of all the 14 records of the dataset given in Figure 5.11 when it is generated using Weka for the prediction of play on the basis of given weather conditions. The numbers shown along with the classes in the tree represent the number of instances that are classified under that node. For example, for outlook 'overcast', play is always held and there are total 4 instances in the dataset which agree with this rule. Similarly, there are 2 instances in the dataset for which play is not held if outlook is rainy and windy is true.

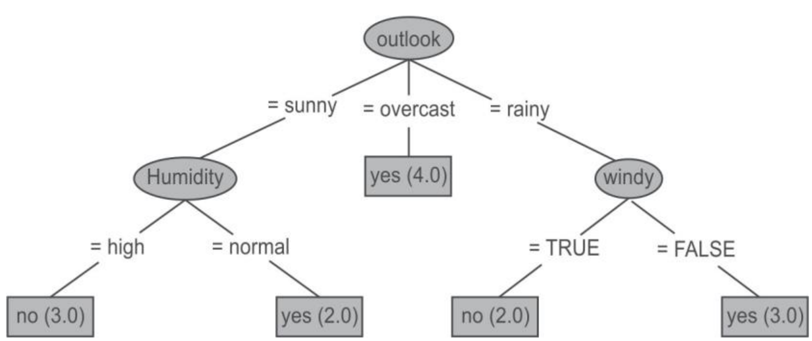


Figure 5.18 Final decision tree after analysis of Sunny, Overcast and Rainy dataset

Suppose, we have to predict whether the play will be held or not for an unknown instance having Temperature 'mild', Outlook 'sunny', Humidity 'normal' and windy 'false', it can be easily predicted on the basis of decision tree shown in Figure 5.18. For the given unknown instance, the play will be held based on conditional checks as shown in Figure 5.19.

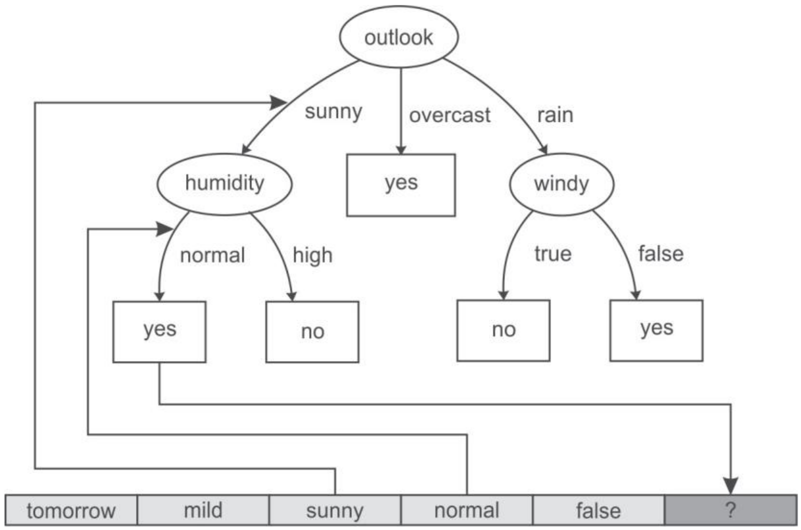


Figure 5.19 Prediction of Play for an unknown instance

5.5.6 Drawbacks of information gain theory

Though information gain is a good measure for determining the significance of an attribute, it has its own limitations. When it is applied to attributes with a large number of distinct values, a noticeable problem occurs. For example, while building a decision tree for a dataset consisting of the records of customers. In this case, information gain is used to determine the most significant attributes so that they can be tested for the root of the tree. Suppose, customers' credit card number is one of the input attributes in the customer dataset. As this input attribute uniquely identifies each customer, it has high mutual information. However, it cannot be included in the decision tree for making decisions because not all customers have credit cards and there is also a problem of how to treat them on the basis of their credit card number. This method will not work for all the customers.

5.5.7 Split algorithm based on Gini Index

The Gini Index is used to represent level of equality or inequality among objects. It can also be used to make decision trees. It was developed by Italian scientist Corrado Gini in 1912 and was used to analyze equality distribution of income among people. We will consider the example of wealth distribution in society in order to understand the concept of the Gini Index. The Gini Index always ranges between 0 and 1. It was designed to define the gap between the rich and the poor, with 0 signifying perfect equality where all people have the same income while 1 demonstrating perfect inequality where only one person gets everything in terms of income and rest of the others get nothing.

From this, it is evident that if Gini Index is very high, there will be huge inequality in income distribution. Therefore, we will be interested in knowing person's income. But in a society where everyone has same income then no one will be interested in knowing each other's income because they know that everyone is at the same level. Thus, the attribute of interest can be decided on the basis that if attribute has a high value Gini Index then it carriers more information and if it has a low value Gini Index then its information content is low.

To define the index, a graph is plotted by considering the percentage of income of the population as the Y-axis and the percentage of the population as the X-axis as shown in Figure 5.20.

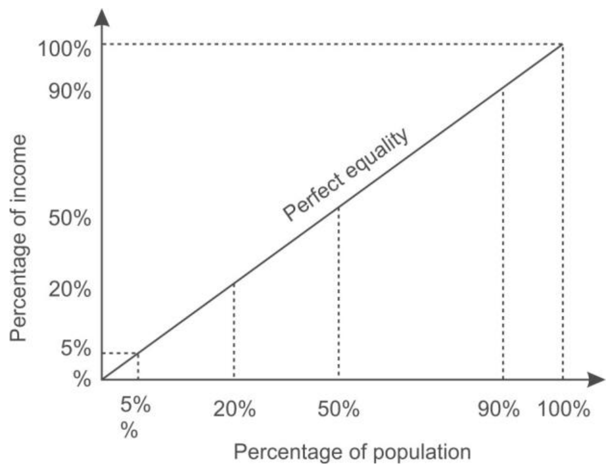


Figure 5.20 Gini Index representing perfect equality

In case of total equality in society, 5% of the people own 5% of the wealth, 20% of the people own 20% of the wealth similarly 90% of people own 90% of wealth. The line at 45 degrees thus represents perfect equal distribution of income in society.

The Gini Index is the ratio of the area between the Lorenz curve and the 45 degree line to the area under the 45 degree line given as follows.

$$\text{Gini Index (G)} = \frac{\text{area between the Lorenz curve and the 45} - \text{degree line}}{\text{area under the 45} - \text{degree line}}$$

As shown in Figure 5.21, the area that lies between the line of equality and the Lorenz curve is marked with *A* and the total area under the line of equality is represented by *(A + B)* in the figure. Therefore,

$$G = \frac{A}{(A + B)}$$

Smaller the ratio, lesser is the area between the two curves and more evenly distributed is the wealth in the society.

The most equal society will be the one in which every person has the same income, making $A = 0$, thus making $G = 0$.

However, the most unequal society will be the one in which a single person receives 100% of the total wealth thus making $B = 0$ and $G = A/A = 1$. From this, we can observe that G always lies between 0 and 1.

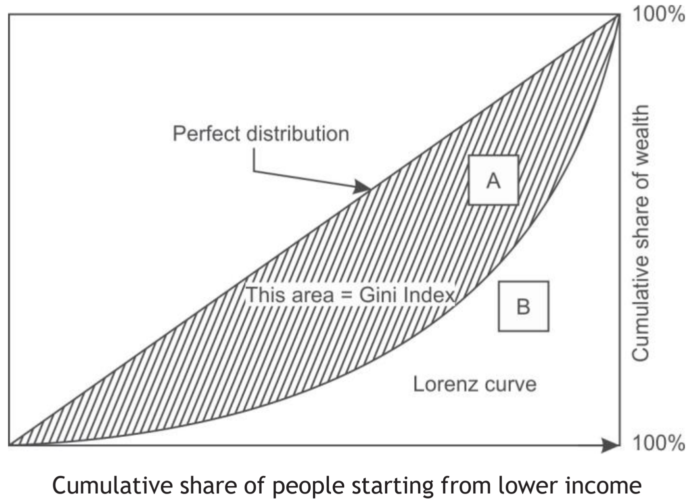


Figure 5.21 Lorenz curve

The more nearly equal a country's income distribution is, the closer is its Lorenz curve to the 45 degree line and the lower is its Gini Index. The more unequal a country's income distribution is, the farther is its Lorenz curve from the 45 degree line and the higher is its Gini Index. For example, as shown in Figure 5.22 (a), the bottom 10% of the people have 90% of total income, therefore Gini Index will be larger as the area between the equality line and the Lorenz curve is larger because there is large inequality in income distribution. Similarly, as shown in Figure 5.22 (b), the bottom 20% of the people have 80% of the total income and the bottom 30% of the people have 70% of the total income is shown in Figure 5.22 (c).

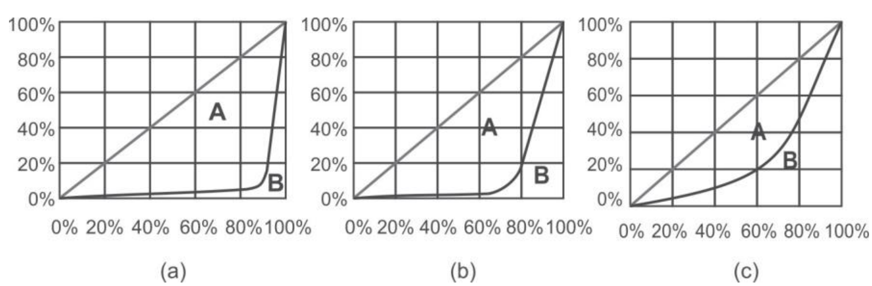


Figure 5.22 Lorenz curves with varying income distributions

From this, it can be concluded that, if income were distributed with perfect equality, the Lorenz curve would coincide with the 45 degree line and the Gini Index would be zero. However, if income were distributed with perfect inequality, the Lorenz curve would coincide with the horizontal axis and the right vertical axis and the index would be 1.

Gini Index can also be calculated in terms of probability and if a dataset *D* contains instances from *n* classes, the Gini Index, G(D), is defined as

$$G(D) = 1 - \sum (p_i)^{\text{No. of classes}}$$

Here, p_i is the relative frequency or probability of class i in D. *Let us calculate the Gini Index for tossing an unbiased coin.*

$$G = 1 - (\text{Probability of getting head})^{\text{No of classes}} - (\text{Probability of getting tail})^{\text{No of classes}}$$

$$G = 1 - (0.5)^2 - (0.5)^2 = 0.5$$

But if coin is biased having head on both sides, then there is 100% chance of a head and 0% chance of tail then the Gini Index is:

$$G = 1 - (1)^2 = 0$$

As there is no uncertainty about the coin in this case so the index is 0. It is maximum at a value of 0.5 in case of an unbiased coin. If the coin is biased, such that head occurs 60% of the times. then the Gini Index is reduced to 0.48.

Similarly we can calculate the Gini Index for a dice with six possible outcomes with equal probability as

$$G = 1 - 6(1/6)^2 = 5/6 = 0.833$$

If the dice is biased, let us say there is 50% chance of getting 6 and remaining 50% is being shared by other 5 numbers leaving only a 10% chance of getting each number other than 6 then the Gini Index is

$$G = 1 - 5(0.1)^2 - (0.5)^2 = 0.70$$

$$G = 1 - 5(0.05)^2 - (0.75)^2 = 0.425$$

Here, Gini Index has been reduced from 0.833 to 0.70. Clearly, the high value of index means high uncertainty.

It is important to observe that the Gini Index behaves in the same manner as the information gain discussed in Section 5.6.4. Table 5.1 clearly shows that same trend in both the cases.

Table 5.1 Information and Gini Index for a number of events

Event	*Information*	*Gini Index*
Toss of a unbiased coin	1.0	0.5
Toss of a biased coin (60% heads)	0.881	0.42
Throw of a unbiased dice	2.585	0.83
Throw of a biased die (50% chance of a 6)	2.16	0.7

5.5.8 Building a decision tree with Gini Index

Let us build the decision tree for the dataset given in Figure 5.23.

It has 3 input attributes X, Y, Z and one output attribute 'Class'. We have added a new column for instance numbers for easy explanation. The dataset contains four records and the output attribute or class can be either A or B.

Instance Number	*X*	*Y*	*Z*	*Class*
1	1	1	1	A
2	1	1	0	A
3	0	0	1	B
4	1	0	0	B

X	*Y*	*Z*	*Class*
1 = 3	1 = 2	1 = 2	A = 2
0 = 1	0 = 2	0 = 2	B = 2

Figure 5.23 Dataset for class C prediction based on given attribute condition

The frequencies of the two output classes are given as follows:
A = 2 (Instances 1,2)
B = 2 (Instances 3,4)
The Gini Index of the whole dataset is calculated as follows:

$$G = 1 - (\text{probability for Class A})^{\text{No of classes}} - (\text{probability for Class B})^{\text{No of classes}}$$

Here, probability for class A = (No of instances for class I/Total no of instances) = 2/4
And probability for class B = (No of instances for class II/Total no of instances) = 2/4
Therefore, $G = 1 - (2/4)^2 - (2/4)^2 = 0.5$
Let us consider each attribute one by one as split attribute and calculate Gini Index for each attribute.

Attribute 'X'

As given in dataset, there are two possible values of X, i.e., 1, 0. Let us analyze each case one by one.

For X = 1, there are 3 instances namely instance 1, 2 and 4. The first two instances are labeled as class A and the third instance, i.e, record number 4 is labeled as class B.

For X = 0, there is only 1 instance, i.e, instance number 3 which is labeled as class B.

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into two subsets according to X being 1 or 0.

Computing the Gini Index for each case,

$$G(X1) = 1 - (2/3)^2 - (1/3)^2 = 0.44444$$

$$G(X0) = 1 - (0/1)^2 - (1/1)^2 = 0$$

Total Gini Index for the two sub-trees = probability for X having value 1 * G(X1) + probability for X having value 0 * G(X0)

Here, probability for X having value 1 = (Number of instances for X having value 1/Total number of instances) = 3/4

And probability for X having value 0 = (Number of instances for X having value 0/Total number of instances) = 1/4

Therefore, total Gini Index for two subtrees = (3/4) G(X1) + (1/4) G(X0)

= 0.333333+0

= 0.333333

Attribute 'Y'

There are two possible values of the Y attribute, i.e., 1 or 0. Let us analyze each case one by one.

There are 2 instances where Y has value 1. In both cases when Y = 1, the record belongs to class A and in 2 instances when Y = 0 both records belong to class B.

Given the above values, let us compute Gini Index by using this attribute. We divide the dataset into two subsets according to Y being 1 or 0.

Computing the Gini Index for each case,

$$G(Y1) = 1 - (2/2)^2 - (0/2)^2 = 0$$

$$G(Y0) = 1 - (0/2)^2 - (2/2)^2 = 0$$

Total Gini Index for two sub-trees = probability for Y having value 1* G(Y1) + probability for Y having value 0 * G(Y0)

Here, probability for Y in 1 = (Number of instances for Y having 1/Total number of instances) = 2/4

And probability for Y in 0 = (Number of instances for Y having 0/Total number of instances) = 2/4

Therefore, Total Gini Index for the two sub-trees = (2/4) G(Y1) + (2/4) G(Y0)

= 0 + 0 = 0

Attribute 'Z'

There are two possible values of the Z attribute, i.e., 1 or 0. Let us analyze each case one by one.

There are 2 instances where Z has value 1 and 2 instances where Z has value 0. In both cases, there exists a record belonging to class A and class B with Z either being 0 or 1.

Given the above values, let us compute the information by using this attribute. We divide the dataset into two subsets according to Z conditions, i.e., 1 and 0.

Computing the Gini Index for each case,

$$G\ (Z1) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$G(Z0) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

Total Gini Index for the two sub-trees = probability for Z having value 1 * G(Z1) + probability for Z having value 0 * G(Z0)

Here, probability for Z having value 1 = (Number of instances for Z having value 1/Total number of instances) = 2/4

And probability for Z having value 0 = (Number of instances for Z having value 0/Total number of instances) = 2/4

Therefore, total information for the two subtrees = (2/4) G(Z1) + (2/4) G(Z0)

= 0.25 + 0.25 = 0.50

The Gain can now be computed:

Potential Split attribute	*Gini Index before split*	*Gini Index after split*	*Gain*
X	0.5	0.333333	0.166667
Y	0.5	0	0.5
Z	0.5	0.50	0

Since the largest Gain is provided by the attribute 'Y' it is used for the split as depicted in Figure 5.24.

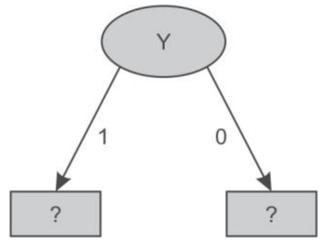


Figure 5.24 Data splitting based on Y attribute

There are two possible values for attribute Y, i.e., 1 or 0, and so the dataset will be split into two subsets based on distinct values of Y attribute as shown in Figure 5.24.

Dataset for Y '1'

Instance Number	*X*	*Z*	*Class*
1	1	1	A
2	1	0	A

There are 2 samples records both belonging to class A. Thus, frequencies of classes are as follows:

A = 2 (Instances 1, 2)

B = 0

Information provided by the whole dataset on the basis of class is given by

$$I = - (2/2) \log (2/2) – (0/2) \log(0/2) = 0$$

As irrespective of value of X and Z both the records belong to class 'A' hence for Y = 1 the tree will classify as class 'A' as shown in Figure 5.25.

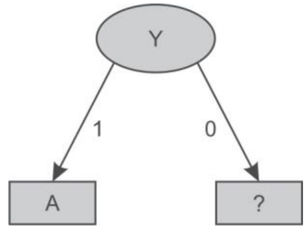


Figure 5.25 Decision tree after splitting of attribute Y having value '1'

Dataset for Y '0'

Instance Number	*X*	*Z*	*Class*
3	0	1	B
4	1	0	B

Here, for Y having value 0, both records are classified as class 'B' irrespective of value of X and Z. Thus, the decision tree will look like as shown in Figure 5.26 after analysis of the Y = 0 subset.

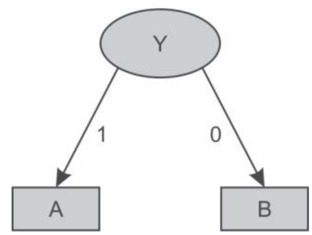


Figure 5.26 Decision tree after splitting of attribute Y value '0'

Let us consider another example to build a decision tree for the dataset given in Figure 5.27. It has four input attributes outlook, temperature, humidity and windy. Instance numbers have been added for easier explanation. Here, 'Play' is the class or output attributes and there are 14 records containing the information if play was held or not, based on weather conditions.

Instance Number	*Outlook*	*Temperature*	*Humidity*	*Windy*	*Play*
1	sunny	hot	high	false	No
2	sunny	hot	high	true	No
3	overcast	hot	high	false	Yes
4	rainy	mild	high	false	Yes
5	rainy	cool	normal	false	Yes
6	rainy	cool	normal	true	No
7	overcast	cool	normal	true	Yes
8	sunny	mild	high	false	No
9	sunny	cool	normal	false	Yes
10	rainy	mild	normal	false	Yes
11	sunny	mild	normal	true	Yes
12	overcast	mild	high	true	Yes
13	overcast	hot	normal	false	Yes
14	rainy	mild	high	true	No

Attribute, Values and Counts

Outlook	*Temperature*	*Humidity*	*Windy*	*Play*
sunny = 5	hot = 4	high = 7	true = 6	yes = 9
overcast = 4	mild = 6	normal = 7	false = 8	no = 5
rainy = 5	cool = 4			

Figure 5.27 Dataset for play prediction based on given day weather conditions

In this dataset, there are 14 samples and two classes for target attribute Play, i.e., Yes and No. The frequencies of these two classes are given as follows:

Yes = 9 (Instances 3,4,5,7,9,10,11,12,13 and 14)

No = 5(Instances 1,2,6,8 and 15)

The Gini Index of the whole dataset is given by

$$G = 1 - (\text{probability for Play Yes})^{\text{No of classes}} - (\text{probability for Play No})^{\text{No of classes}}$$

Here, probability for Play Yes = 9/14

And, probability for Play No = 5/14

Therefore, $G = 1\text{-}(9/14)^2 - (5/14)^2 = 0.45918$

Let us consider each attribute one by one as split attributes and calculate the Gini Index for each attribute.

Attribute 'Outlook'

As given in the dataset, there are three possible values of outlook, i.e., Sunny, Overcast and Rainy. Let us analyze each case one by one.

There are 5 instances for outlook having value sunny. Play is held in case of 2 instances (9 and 11) and is not held in case of 3 instances (1, 2 and 8).

There are 4 instances for outlook having value overcast. Play is held in case of all the 4 instances (3, 7, 12 and 13).

There are 5 instances for outlook having value rainy. Play is held in case of 3 instances (4, 5 and 10) and is not held in case of 2 instances (6 and 14).

Given the above values, let us compute the Gini Index by using this 'Outlook' attribute. We divide the dataset into three subsets according to outlook being sunny, overcast or rainy. Computing Gini Index for each case,

$$G(\text{Sunny}) = G(S) = 1 - (2/5)^2 - (3/5)^2 = 0.48$$

$$G(\text{Overcast}) = G(O) = 1 - (4/4)^2 - (0/4)^2 = 0$$

$$G(\text{Rainy}) = G(R) = 1 - (3/5)^2 - (2/5)^2 = 0.48$$

Total Gini Index for three sub-trees = probability for outlook Sunny * G(S) + probability for outlook Overcast * G(O) + probability for outlook Rainy * G (R)

The probability for outlook Sunny = (Number of instances for outlook Sunny/Total number of instances) = 5/14

The probability for outlook Overcast = (Number of instances for outlook Overcast/Total number of instances) = 4/14

The probability for outlook Rainy = (Number of instances for outlook Rainy/Total number of instances) = 5/14

Therefore, the total Gini Index for the three sub-trees = (5/14) G(S) + (4/14) G(O) + (5/14) G(R)
= 0.171428+ 0 + 0.171428
= 0.342857

Attribute 'Temperature'

There are three possible values of the 'Temperature' attribute, i.e., Hot, Mild or Cool. Let us analyze each case one by one.

There are 4 instances for temperature having value hot. Play is held in case of 2 instances (3 and 13) and is not held in case of other 2 instances (1 and 2).

There are 6 instances for temperature having value mild. Play is held in case of 4 instances (4, 10, 11 and 12) and is not held in case of 2 instances (8 and 14).

There are 4 instances for temperature having value cool. Play is held in case of 3 instances (5, 7 and 9) and is not held in case of 1 instance (6).

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into three subsets according to temperature conditions, i.e., hot, mild and cool. Computing the Gini Index for each case,

$$G\ (Hot) = G\ (H) = 1 - (2/4)^2 - (2/4)^2 = 0.50$$

$$G\ (Mild) = G\ (M) = 1 - (4/6)^2 - (2/6)^2 = 0.444$$

$$G\ (Cool) = G(C) = 1 - (3/4)^2 - (1/4)^2 = 0.375$$

Total Gini Index of 3 sub-trees = probability for temperature hot * G(H) + probability for temperature mild * G(M) + probability for temperature cool * G(C)

Here, probability for temperature hot = (Number of instances for temperature hot/Total number of instances) = 4/14

And probability for temperature mild = (Number of instances for temperature mild/Total number of instances) = 6/14

And probability for temperature cool = (Number of instances for temperature cool/Total number of instances) = 4/14

Therefore, the total Gini Index of the three sub-trees = (4/14) G(H) + (6/14) G(M) + (4/14) G(C)
= 0.44028

Attribute 'Humidity'

There are two possible values of the 'Humidity' attribute, i.e., High and Normal. Let us analyze each case one by one.

There are 7 instances for humidity having high value. Play is held in case of 3 instances (3, 4 and 12) and is not held in case of 4 instances (1, 2, 8 and 14).

There are 7 instances for humidity having normal value. Play is held in case of 6 instances (5, 7, 9, 10, 11 and 13) and is not held in case of 1 instance (6).

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into two subsets according to humidity conditions, i.e., high and normal. Computing Gini Index for each case,

$$G\ (\text{High}) = G(H) = 1 - (3/7)^2 - (4/7)^2 = 0.48979$$

$$G\ (\text{Normal}) = G(N) = 1 - (6/7)^2 - (1/7)^2 = 0.24489$$

Total Gini Index for the two sub-trees = probability for humidity high* G(H) + probability for humidity normal * G(N)

Here, probability for humidity high = (Number of instances for humidity high/Total number of instances) = 7/14

The probability for humidity normal = (Number of instances for humidity normal/Total number of instances) = 7/14

Therefore, the total Gini Index for two sub-trees = (7/14) G(H) + (7/14) G(N)

= 0.3673439

Attribute 'Windy'

There are two possible values for this attribute, i.e., true and false. Let us analyze each case one by one.

There are 6 instances for windy having value true. Play is held in case of 3 instances (7, 11 and 12) and is not held in case of another 3 instances (2, 6 and 14).

There are 8 instances for windy having value false. Play is held in case of 6 instances (3, 4, 5, 9, 10 and 13) and is not held in case of 2 instances (1 and 8).

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into two subsets according to windy being true or false. Computing Gini Index for each case,

$$G\ (\text{True}) = G(T) = 1 - (3/6)^2 - (3/6)^2 = 0.5$$

$$G\ (\text{False}) = G(F) = 1 - (6/8)^2 - (2/8)^2 = 0.375$$

Total Gini Index for the two sub-trees = probability for windy true * G(T) + probability for windy false * G(F)

Here, The probability for windy true = (Number of instances for windy true/Total number of instances) = 6/14

And the probability for windy false = (Number of instances for windy false/Total number of instances) = 8/14

Therefore, total Gini Index for the two sub-trees = (6/14) G(T) + (8/14) G(F)

= 0.42857

The gain can now be computed as follows:

Potential Split attribute	*Gini Index before split*	*Gini Index after split*	*Gain*
Outlook	*0.45918*	*0.342857*	*0.116323*
Temperature	0.45918	0.44028	0.0189
Humidity	0.45918	0.3673439	0.0918361
Windy	0.45918	0.42857	0.03061

Hence, the largest gain is provided by the attribute 'Outlook' and it is used for the split as depicted in Figure 5.28.

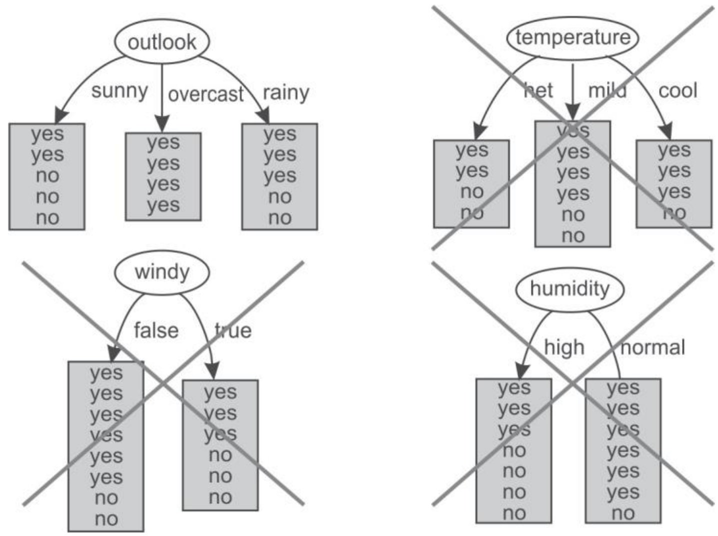


Figure 5.28 Selection of Outlook as root attribute

For Outlook, as there are three possible values, i.e., sunny, overcast and rainy, the dataset will be split into three subsets based on distinct values of the Outlook attribute as shown in Figure 5.29.

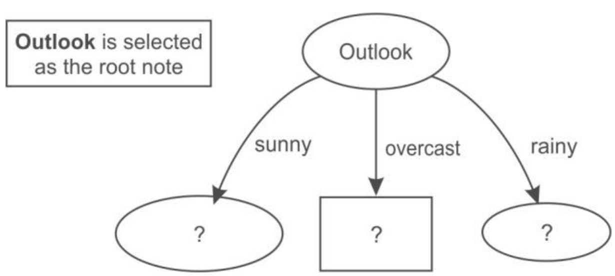


Figure 5.29 Data splitting based on Outlook attribute

Dataset for Outlook 'Sunny'

Instance Number	*Temperature*	*Humidity*	*Windy*	*Play*
1	hot	high	false	No
2	hot	high	true	No
3	mild	high	false	No
4	cool	normal	false	Yes
5	mild	normal	true	Yes

In this case, we have three input attributes Temperature, Humidity and Windy. This dataset consists of 5 samples. The frequencies of the classes are as follows:

Yes = 2 (Instances 4, 5)

No = 3 (Instances 1,2,3)

The Gini Index of the whole dataset is given by

$$G = 1 - (\text{probability for Play Yes})^{\text{Number of classes}} - (\text{probability for Play No})^{\text{Number of classes}}$$

Here, probability for Play Yes = 2/5

And probability for Play No = 3/5

Therefore, $G = 1 - (2/5)^2 - (3/5)^2 = 0.48$

Let us consider each attribute one by one as split attributes and calculate the Gini Index for each attribute.

Attribute 'Temperature'

There are three possible values of Temperature attribute, i.e., hot, mild and cool. Let us analyze each case one by one.

There are 2 instances for temperature having value hot. Play is not held in both of 2 instances (1 and 2).

There are 2 instances for temperature having value mild. Play is held in case of 1 instance (5) and is not held in case of another instance (3).

There is only 1 instance for temperature having value cool. Play is held in this single instance (4).

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into three subsets according to temperature being hot, mild or cool. Computing Gini Index for each case,

$$G(\text{Hot}) = G(H) = 1 - (0/2)^2 - (2/2)^2 = 0$$

$$G(\text{Mild}) = G(M) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$G(\text{Cool}) = G(C) = 1 - (1/1)^2 - (0/1)^2 = 0$$

Total Gini Index for the three sub-trees = probability for temperature hot * G(H) + probability for temperature mild * G(M) + probability for temperature cool *G(C)

Here, probability for temperature hot = (Number of instances for temperature hot/Total number of instances) = 2/5

And probability for temperature mild = (Number of instances for temperature mild/Total number of instances) = 2/5

And probability for temperature cool = (Number of instances for temperature cool/Total number of instances) = 1/5

Therefore, total Gini Index for these three sub-trees = (2/5)G(H) + (1/5) G(M) + (2/5) G(C)

= 0 + 0.1 + 0 = 0.1

Attribute 'Humidity'

There are two possible values of Humidity attribute, i.e., High and Normal. Let us analyze each case one by one.

There are 3 instances when humidity is high. Play is not held in any of these 3 instances (1, 2 and 3).

There are 2 instances when humidity is normal. Play is held in both of 2 instances (4 and 5).

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into two subsets according to humidity being high or normal. Computing Gini Index for each case,

$$G\ (High) = G(H) = 1 - (0/3)^2 - (3/3)^2 = 0$$

$$G\ (Normal) = G(N) = 1 - (2/2)^2 - (0/2)^2 = 0$$

Total Gini Index for the two sub-trees = probability for humidity high * G(H) + probability for humidity normal * G(N)

Here, probability for humidity high = (Number of instances for humidity high/Total number of instances) = 3/5

And probability for humidity normal = (Number of instances for humidity normal/Total number of instances) = 2/5

Therefore, total Gini Index for the two sub-trees = (3/5) G(H) + (2/5) G(N) = 0

Attribute 'Windy'

There are two possible values for this attribute, i.e., true and false. Let us analyze each case one by one.

There are 2 instances when it is windy. Play is held in case of 1 instance (5) and is not held in case of another 1 instance (2).

There are 3 instances when it is not windy. Play is held in case of 1 instance (4) and is not held in case of 2 instances (1 and 3).

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into two subsets according to windy being true or false. Computing Gini Index for each case,

$$G\ (True) = G\ (T) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$G\ (False) = G\ (F) = 1 - (1/3)^2 - (2/3)^2 = 0.4444$$

Total Gini Index for the two sub-trees = probability for windy true * G(T) + probability for windy false * G(F)

Here, probability for windy true = (Number of instances for windy true/Total number of instances) = 2/5

And probability for windy false = (Number of instances for windy false/Total number of instances) = 3/5

Therefore, total Gini Index for the two sub-trees = (2/5) G(T) + (3/5) G(F) = 0.2 + 0.2666 = 0.4666

The Gain can now be computed:

Potential Split attribute	*Gini Index before split*	*Gini Index after split*	*Gain*
Temperature	0.48	0.1	0.38
Humidity	0.48	0	0.48
Windy	0.48	0.4666	0.014

The largest gain is provided by the attribute 'Humidity' and so, it is used for the split. Here, the algorithm is to be tuned in such a way that it should stop when we get the 0 value of the Gini Index to reduce the calculations for larger datasets. Now, the Humidity attribute is selected as the split attribute as depicted in Figure 5.30.

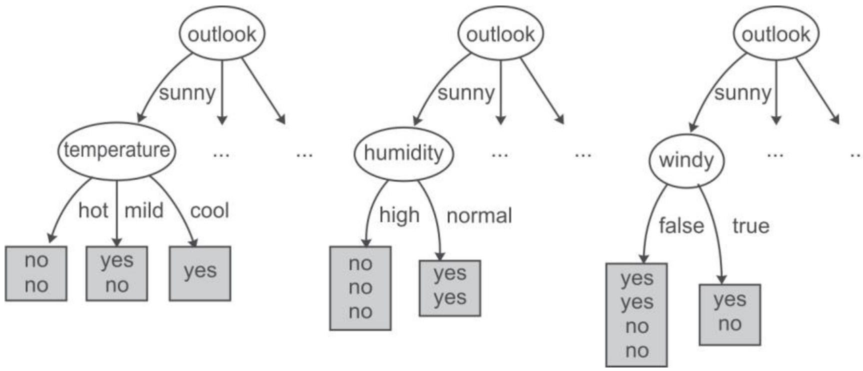


Figure 5.30 Humidity attribute is selected from dataset of Sunny instances

As the dataset consists of two possible values of humidity so data is split in two parts, i.e. humidity 'high' and humidity 'low' as shown below.

Dataset for Humidity 'High'

Instance Number	*Temperature*	*Windy*	*Play*
1	Hot	false	No
2	Hot	true	No
3	Mild	false	No

We can clearly see that all records with high humidity are classified as play having 'No' value. On the other hand, all records with normal humidity value are classified as play having 'Yes' value.

Dataset for Humidity 'Normal'

Instance Number	*Temperature*	*Windy*	*Play*
1	cool	false	Yes
2	mild	true	Yes

Thus, the decision tree will look like as shown in Figure 5.31 after analysis of the Humidity attribute.

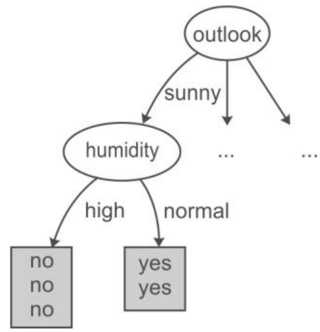


Figure 5.31 Decision tree after spitting data on the Humidity attribute

The analysis of the 'Sunny' dataset is now over and has allowed the the generation of the decision tree shown in Figure 5.31. It has been analyzed that if the outlook is 'Sunny' and humidity is 'Normal' then play will be held while on the other hand if the humidity is 'high' then play will not be held.

Now, let us take next dataset of where Outlook has value overcast for further analysis.

Dataset for Outlook 'Overcast'

Instance Number	*Temperature*	*Humidity*	*Windy*	*Play*
1	hot	high	false	Yes
2	cool	normal	true	Yes
3	mild	high	true	Yes

This dataset consists of 3 records. For outlook overcast all records belong to the 'Yes' class only. Thus, the decision tree will look like Figure 5.32 after analysis of the overcast dataset.

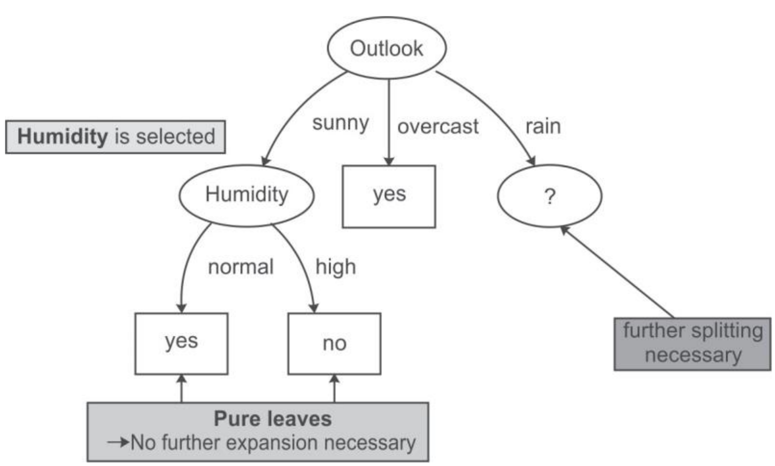


Figure 5.32 Decision tree after analysis of Sunny and Overcast datasets

Therefore, it may be concluded that if the outlook is 'Overcast' then play is held. Now we have to select another split attribute for the outlook rainy and dataset for this is given as follows.

Dataset for Outlook 'Rainy'

Instance Number	*Temperature*	*Humidity*	*Windy*	*Play*
1	Mild	High	False	Yes
2	Cool	Normal	False	Yes
3	Cool	Normal	Ttrue	No
4	Mild	Normal	False	Yes
5	Mild	High	True	No

This dataset consists of 5 records. The frequencies of the classes are as follows:
Yes= 3 (Instances 1,2 and 4)
No= 2 (Instances 3 and 5)
Gini Index of the whole dataset on the basis of whether play held or not is given by

$$G = 1 - (\text{probability for Play Yes})^{\text{Number of classes}} - (\text{probability for Play No})^{\text{Number of classes}}$$

Here, probability for Play Yes = 3/5
And probability for Play No = 2/5
Therefore, $G = 1 - (3/5)^2 - (2/5)^2 = 0.48$
Let us consider each attribute one by one as split attributes and calculate the Gini Index for each attribute.

Attribute 'Temperature'

There are two possible values of Temperature attribute, i.e., mild and cool. Let us analyze each case one by one.

There are 3 instances for temperature having value mild. Play is held in case of 2 instances (1 and 4) and is not held in case of 1 instance (5).

There are 2 instances for temperature having value cool. Play is held in case of 1 instance (2) and is not held in case of another 1 instance (3).

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into two subsets according to temperature being mild or cool. Computing Gini Index for each case,

$$G\text{ (Mild)} = G(M) = 1 - (2/3)^2 - (1/3)^2 = 0.444$$

$$G\text{ (Cool)} = G(C) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

Total Gini Index for the two sub-trees = probability for temperature mild * G(M) + probability for temperature cool * G (C)

Here, probability for temperature mild = (Number of instances for temperature mild/Total number of instances) = 3/5

And probability for temperature cool = (Number of instances for temperature cool/Total number of instances) = 2/5

Therefore, Total Gini Index for the two sub-trees = (3/5) G(M) + (2/5) G(C) = 0.2666 + 0.2 = 0.4666

Attribute 'Humidity'

There are two possible values of Humidity attribute, i.e., High and Normal. Let us analyze each case one by one.

There are 2 instances for humidity having high value. Play is held in case of 1 instance (1) and is not held in case of 1 instance (5).

There are 3 instances for humidity having normal value. Play is held in case of 2 instances (2 and 4) and is not held in case of 1 instance (3).

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into two subsets according to humidity being high or normal. Computing the Gini Index for each case,

$$G\ (\text{High}) = G(H) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$G\ (\text{Normal}) = G(N) = 1 - (2/3)^2 - (1/3)^2 = 0.4444$$

Total Gini Index for the two sub-trees = probability for humidity high* G(H) + probability for humidity normal * G(N)

Here, probability for humidity high = (Number of instances for humidity high/Total number of instances) = 2/5

And probability for humidity normal = (Number of instances for humidity normal/Total number of instances) = 3/5

Therefore, Total Gini Index for the two sub-trees = (2/5) G(H) + (3/5) G(N) = 0.2 + 0.2666 =0.4666

Attribute 'Windy'

There are two possible values for this attribute, i.e., true and false. Let us analyze each case one by one.

There are 2 instances where windy is true. Play is not held in case of both of 2 instances (3 and 5).

There are 3 instances where windy is false. Play is held in case of all 3 instances (1, 2 and 4).

Given the above values, let us compute the Gini Index by using this attribute. We divide the dataset into two subsets according to windy being true or false. Computing the Gini Index for each case,

$$G\ (\text{True}) = G(T) = 1 - (0/2)^2 - (2/2)^2 = 0$$

$$G\ (\text{False}) = G(F) = 1 - (3/3)^2 - (0/3)^2 = 0$$

Total Gini Index for the two sub-trees = probability for windy true * G(T) + probability for windy false * G(F)

Here, probability for windy true = (Number of instances for windy true/Total number of instances) = 2/5

And probability for windy false = (Number of instances for windy false/Total number of instances) = 3/5

Therefore, total Gini Index for these two subtrees = (2/5) G(T) + (3/5) G(F) = 0
The Gain can now be computed:

Potential Split attribute	Gini Index before split	Gini Index after split	Gain
Temperature	0.48	0.4666	0.0134
Humidity	0.48	0.4666	0.0134
Windy	0.48	0	0.48

Hence, the largest gain is provided by the attribute 'Windy' and it is used for the split. Now, the Windy attribute is selected as split attribute.

Dataset for Windy 'TRUE'

Instance Number	*Temperature*	*Humidity*	*Play*
1	cool	normal	No
2	mild	high	No

It is clear from the data that whenever it is windy the play is not held. Play is held whenever conditions are otherwise.

Dataset for Windy 'FALSE'

Instance Number	*Temperature*	*Humidity*	*Play*
1	mild	high	Yes
2	cool	normal	Yes
3	mild	normal	Yes

Thus, the decision tree will look like Figure 5.33 after analysis of the 'Rainy' attribute.

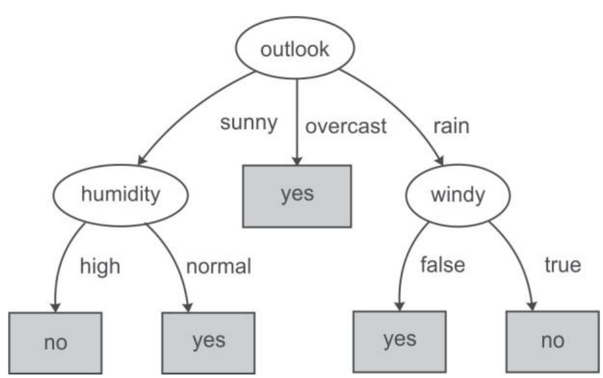


Figure 5.33 Decision tree after analysis of Sunny, Overcast and Rainy datasets

It has been concluded that if the outlook is 'Rainy' and value of windy is 'False' then play will be held and on the other hand, if value of windy is 'True' then the play will not be held.

Figure 5.34 represents the final tree view of all the 14 records of the dataset given in Figure 5.27 when it is generated using Weka for the prediction of play on the basis of given weather conditions. The numbers shown along with the classes in the tree represent the number of instances that are classified under that node. For example, for outlook overcast, play is always held and there are total 4 instances in the dataset which agree with this rule. Similarly, there are 2 instances in the dataset for which play is not held if outlook is rainy and windy is true.

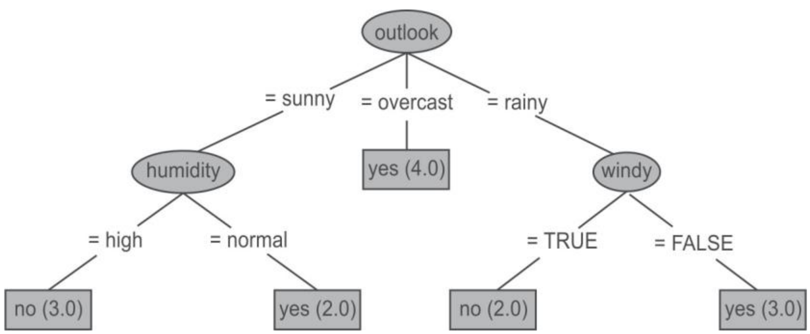


Figure 5.34 Final decision tree after analysis of Sunny, Overcast and Rainy datasets

If we have to predict whether play will be held or not for an unknown instance having Temperature 'mild', Outlook 'sunny', Humidity 'normal' and Windy 'false', it can be easily predicted on the basis of decision tree shown in Figure 5.34. For the given unknown instance, play will be held as shown in Figure 5.35.

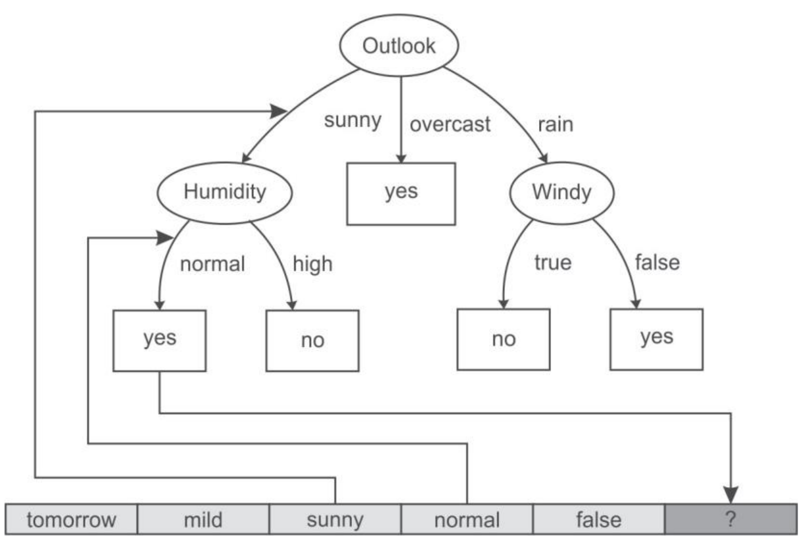


Figure 5.35 Prediction of play for unknown instance

5.5.9 Advantages of the decision tree method

The main advantages of a decision tree classifier are as follows:.

- The rules generated by a decision tree classifier are easy to understand and use.
- Domain knowledge is not required by the decision tree classifier.
- Learning and classification steps of the decision tree are simple and quick.

5.5.10 Disadvantages of the decision tree

The disadvantages of a decision tree classifier can be:

- Decision trees are easy to use compared to other decision-making models, but preparing decision trees, especially large ones with many branches, are complex and time-consuming affairs.
- They are unstable, meaning that a small change in the data can lead to a large change in the structure of the optimal decision tree.
- They are often relatively inaccurate. Many other predictors perform better with similar data.
- Decision trees, while providing easy to view illustrations, can also be unmanageable. Even data that is perfectly divided into classes and uses only simple threshold tests may require a large decision tree. Large trees are not intelligible, and pose presentation difficulties.

The other important classification technique is the Naïve Bayes Method which is based on Bayes theorem. The details of Naïve Bayes method have been discussed in next section.

5.7 Naïve Bayes Method

It is based on Bayes theorem given by Thomas Bayes in middle of the eighteenth century. It is amazing that despite being such an old theorem it has found its way into many modern fields such as AI and machine learning. Classification using Bayes theorem is different from the decision tree approach. It is based on a hypothesis that the given data belongs to a particular class. In this theorem probability is calculated for the hypothesis to be true.

Our understanding of this theorem begins with the fact that the Bayes Theorem in based on probabilities and so it is important to define some notations in the beginning.

$P(A)$ refers to the probability of occurrence of event A, while $P(A|B)$ refers to the conditional probability of event A given that event B has already occurred.

Bayes theorem is defined as follows …

$$P(A|B) = P(B|A)\, P(A)/P(B) \qquad \ldots(1)$$

Let us first prove this theorem.

We already know that

$$P(A|B) = P(A \,\&\, B)/P(B) \qquad \ldots(2)$$

[It is the probability that next event will be A when B has already happened]

Similarly, $$P(B|A) = P\,(B \,\&\, A)/P(A) \qquad \ldots(3)$$

From equation (3):

Thus, $P(B \& A) = P(B|A) * P(A)$

By putting this value of $P(B \& A)$ in equation (2), we get ….

$$P(A|B) = P(B|A)\, P(A)/P(B)$$

This proves the Bayes theorem.

Now let us consider that we have to predict the class of X out of three given classes C_1, C_2 and C_3. Here, the hypothesis is that event X has already occurred. Thus, we have to calculate $P(C_i|X)$, conditional probability of class being C_i when X has already occurred.

According to Bayes theorem:

$$P(C_i|X) = P(X|C_i)\, P(C_i)/P(X)$$

In this equation:

$P(C_i|X)$ is the conditional probability of class being C_i when X has already occurred or it is probability that X belongs to class C_i.

$P(X|C_i)$ is the conditional probability of record being X when it is known that output class is C_i

$P(C_i)$ is probability that object belongs to class C_i.

$P(X)$ is the probability of occurrence of record X.

Here, we have already made the hypothesis that X has already occurred so $P(X)$ is 1 so we have to calculate $P(X|C_i)$ and $P(C_i)$ in order to find required value.

To further understand this concept let us consider the following database, where the class of customer is defined based on his/her marital status, gender, employment status and credit rating as shown below.

Owns Home	*Married*	*Gender*	*Employed*	*Credit rating*	*Class*
Yes	Yes	Male	Yes	A	II
No	No	Female	Yes	A	I
Yes	Yes	Female	Yes	B	III
Yes	No	Male	No	B	II
No	Yes	Female	Yes	B	III
No	No	Female	Yes	B	I
No	No	Male	No	B	II
Yes	No	Female	Yes	A	I
No	Yes	Female	Yes	A	III
Yes	Yes	Female	Yes	A	III

Here, we have to predict the class of occurrence of X and let us suppose X is as shown below.

Owns Home	*Married*	*Gender*	*Employed*	*Credit rating*	*Class*
Yes	No	Male	Yes	A	?

Then we have to calculate the probability for class C_i, when X has already occurred. Thus it is

$$P(C_i \mid Yes, No, Male, Yes, A) = P(Yes, No, Male, Yes, A \mid C_i) * P(C_i)$$

Let us first calculate probability of each class, i.e., $P(C_i)$. Here, we have three classes, i.e., *I*, *II* and *III*. There are total 10 instances in the given dataset and there are three instances for class *I*, three for class *II* and four for class *III*. Thus,

Probability of class *I*, i.e., $P(I) = 3/10 = 0.3$

Probability of class *II*, i.e., $P(II) = 3/10 = 0.3$

Probability of class *III*, i.e., $P(III) = 4/10 = 0.4$

Now, let us calculate $P(X|C_i)$, i.e., $P(Yes, No, Male, Yes, A \mid C_i)$

$P(Yes, No, Male, Yes, A \mid C_i) = P(Owns\ Home = Yes| C_i) * P(Married = No| C_i) * P(Gender = Male| C_i) * P(Employed = Yes| C_i) * P(Credit\ rating = A| C_i)$

Thus, we need to calculate

$P(Owns\ Home = Yes|C_i)$,

$P(Married = No| C_i)$,

$P(Gender = Male|C_i)$,

$P(Employed = Yes|C_i)$,

$P(Credit\ rating = A|C_i)$.

Owns Home	*Married*	*Gender*	*Employed*	*Credit rating*	*Class*
No	No	Female	Yes	A	I
No	No	Female	Yes	B	I
Yes	No	Female	Yes	A	I
Yes	**No**	**Male**	**Yes**	A	**? [To be found]**
1/3	1	0	1	2/3	Probability of having{Yes, No, Male, Yes, A} Attribute value given the risk Class I
Yes	Yes	Male	Yes	A	II
Yes	No	Male	No	B	II
No	No	Male	No	B	II
Yes	**No**	**Male**	**Yes**	A	**? [To be found]**

Contd.

Owns Home	*Married*	*Gender*	*Employed*	*Credit rating*	*Class*
2/3	2/3	1	1/3	1/3	Probability of having{Yes, No, Male, Yes, A} Attribute value given the risk Class II
Yes	Yes	Female	Yes	B	III
No	Yes	Female	Yes	B	III
No	Yes	Female	Yes	A	III
Yes	Yes	Female	Yes	A	III
Yes	**No**	**Male**	**Yes**	**A**	**? [To be found]**
1/2	0	0	1	1/2	Probability of having{Yes, No, Male, Yes, A} Attribute value given the risk Class III

Thus, $P(X|I) = 1/3 * 1 * 0 * 1 * 2/3 = 0$

$P(X|II) = 2/3 * 2/3 * 1 * 1/3 * 1/3 = 4/81$

$P(X|III) = 1/2 * 0 * 0 * 1 * 1/2 = 0$

$P(I \mid Yes, No, Male, Yes, A) = P(Yes, No, Male, Yes, A \mid I) * P(I) = 0*0.3 = 0$

$P(II \mid Yes, No, Male, Yes, A) = P(Yes, No, Male, Yes, A \mid II) * P(II) = 4/81*0.3 = 0.0148$

$P(III \mid Yes, No, Male, Yes, A) = P(Yes, No, Male, Yes, A \mid III) * P(III) = 0*0.4 = 0$

Therefore, *X* is assigned to Class *II*. It is very unlikely in real life datasets that the probability of class comes out to be 0 as in this example.

5.7.1 Applying Naïve Bayes classifier to the 'Whether Play' dataset

Let us consider another example of Naïve Bayes classifier for the dataset shown in Figure 5.36. It has 4 input attributes outlook, temperature, humidity and windy. Play is the class or output attribute. These 14 records contain the information if play has been held or not on the basis of any given day's weather conditions.

Instance Number	*Outlook*	*Temperature*	*Humidity*	*Windy*	*Play*
1	sunny	hot	high	false	No
2	sunny	hot	high	true	No
3	overcast	hot	high	false	Yes
4	rainy	mild	high	false	Yes
5	rainy	cool	normal	false	Yes

Contd.

Instance Number	*Outlook*	*Temperature*	*Humidity*	*Windy*	*Play*
6	rainy	cool	normal	true	No
7	overcast	cool	normal	true	Yes
8	sunny	mild	high	false	No
9	sunny	cool	normal	false	Yes
10	rainy	mild	normal	false	Yes
11	sunny	mild	normal	true	Yes
12	overcast	mild	high	true	Yes
13	overcast	hot	normal	false	Yes
14	rainy	mild	high	true	No

Attribute values and counts

Outlook	Temperature	Humidity	Windy	Play
sunny = 5	hot = 4	high = 7	true = 6	yes = 9
overcast = 4	mild = 6	normal = 7	false = 8	no = 5
rainy = 5	cool = 4			

Figure 5.36 Dataset for play prediction based on a given day's weather conditions

From this dataset, we can observe that when the outlook is sunny, play is not held on 3 out of the 5 days as shown in Figure 5.37.

Instance Number	Outlook	Temperature	Humidity	Windy	Play
1	Sunny	Hot	High	False	No
2	Sunny	Hot	High	True	No
8	Sunny	Mild	High	False	No
9	Sunny	Cool	Normal	False	Yes
11	Sunny	Mild	Normal	True	Yes

Figure 5.37 Probability of whether play will be held or not on a Sunny day

Therefore, it can be concluded that there are 60% chances that Play will not be held when it is a sunny day.

The Naïve Bayes theorem is based on probabilities, so we need to calculate probabilities for each occurrence in the instances. So, let us calculate the count for the each occurrence as shown in Figure 5.38.

	Play	
Outlook	yes	no
sunny	2	3
overcast	4	0
rainy	3	2
TOTAL	9	5

Instance Number	Outlook	Temperature	Humidity	Windy	Play
1	sunny	hot	high	false	No
2	sunny	hot	high	true	No
3	overcast	hot	high	false	Yes
4	rainy	mild	high	false	Yes
5	rainy	cool	normal	false	Yes
6	rainy	cool	normal	true	No
7	overcast	cool	normal	true	Yes
8	sunny	mild	high	false	No
9	sunny	cool	normal	false	Yes
10	rainy	mild	normal	false	Yes
11	sunny	mild	normal	true	Yes
12	overcast	mild	high	true	Yes
13	overcast	hot	normal	false	Yes
14	rainy	mild	high	true	No

	Play			Play			Play			Play			Play
Outlook	yes	no	Temp.	yes	no	Humid.	yes	no	Windy	yes	no		
sunny	2	3	hot	2	2	high	3	4	false	6	2	yes	9
overcast	4	0	mild	4	2	normal	6	1	true	3	3	no	5
rainy	3	2	cool	3	1								
TOTAL	9	5	TOTAL	9	5	TOTAL	9	5	TOTAL	9	5	TOTAL	14

Figure 5.38 Summarization of count calculations of all input attributes

Now, we can calculate the probability of play being held or not for each value of input attribute as shown in Figure 5.39. For example, we have 2 instances for play not being held when outlook is rainy and there are in total 5 instances for the outlook attribute where play is not held.

Probability for play no given outlook rainy

$$= \frac{\text{occurrences of play no. given outlook = rainy}}{\text{total no. of occurrences for play no. for attribute outlook}} = 2/5 = 0.40$$

	Play			Play			Play			Play			Play
Outlook	yes	no	Temp.	yes	no	Humid.	yes	no	Windy	yes	no		
sunny	0.22	0.60	hot	0.22	0.40	high	0.33	0.80	false	0.67	0.40	yes	0.64
overcast	0.44	0.00	mild	0.44	0.40	normal	0.67	0.20	true	0.33	0.60	no	0.36
rainy	0.33	0.40	cool	0.33	0.20								

Figure 5.39 Probability of play held or not for each value of attribute

Here, we have to predict the output class for some *X*. Let us suppose *X* is as given below.

Outlook	*Temperature*	*Humidity*	*Windy*	*Play*
sunny	cool	high	true	?

Then, we have to calculate probability of class C_i, when *X* has already occurred. Thus, it is

$$P(C_i \mid sunny, cool, high, true) = P(sunny, cool, high, true \mid C_i) * P(C_i)$$

Let us first calculate probability of each class, i.e., $P(C_i)$. Here, we have two classes, i.e., Yes and No. There are total 14 instances in the given dataset from which 9 instances are for class Yes and 5 for class No. Thus,

Probability of class Yes, i.e., P(Yes) = 9/14=0.64

Probability of class No, i.e., P(No) = 5/14=0.36

Now, let us calculate $P(X|C_i)$, i.e., $P(sunny, cool, high, true\ |C_i)$

$$P(sunny, cool, high, true|C_i) = P(outlook = sunny|C_i)*P(Temperature$$
$$= cool|C_i)* P(humidity$$
$$= high|C_i) * P(windy = true|C_i)$$

Therefore, probability of play to be held under these weather conditions is as shown in Figure 5.40.

$P(outlook = sunny|Yes) = 0.22$

$P(Temperature = cool|Yes) = 0.33$

$P(humidity = high|Yes) = 0.33$

$P(windy = true|Yes) = 0.33$

$P(Yes) = 0.64$

	Play			Play			Play			Play			Play
Outlook	**yes**	**no**	**Temp.**	**yes**	**no**	**Humid.**	**yes**	**no**	**Windy**	**yes**	**no**		
sunny	0.22	0.60	hot	0.22	0.40	high	0.33	0.80	false	0.67	0.40	yes	0.64
overcast	0.44	0.00	mild	0.44	0.40	normal	0.67	0.20	true	0.33	0.60	no	0.36
rainy	0.33	0.40	cool	0.33	0.20								

Figure 5.40 Probability for play 'Yes' for an unknown instance

Likelihood of play being held = $P(X|Yes) * P(Yes)$ = 0.22 * 0.33 * 0.33 * 0.33 * 0.64 = 0.0053

Similarly, probability of play not being held under these weather conditions is as shown in Figure 5.41.

$P(outlook = sunny|No) = 0.60$

$P(Temperature = cool|No) = 0.20$

$P(humidity = high|No) = 0.80$

$P(windy = true|No) = 0.60$

$P(No) = 0.36$

	Play			Play			Play			Play			Play
Outlook	**yes**	**no**	**Temp.**	**yes**	**no**	**Humid.**	**yes**	**no**	**Windy**	**yes**	**no**		
sunny	0.22	0.60	hot	0.22	0.40	high	0.33	0.80	false	0.67	0.40	yes	0.64
overcast	0.44	0.00	mild	0.44	0.40	normal	0.67	0.20	true	0.33	0.60	no	0.36
rainy	0.33	0.40	cool	0.33	0.20								

Figure 5.41 Probability for play 'No' for an unknown instance

Likelihood of play being held = *P(X|No)* * *P(No)* = 0.60 * 0.20 * 0.80 * 0.60 * 0.36 = 0.0206

Now, above calculated likelihoods are converted to probabilities to decide whether the play is held or not under these weather conditions.

Probability of play Yes = Likelihood for play Yes/(Likelihood for play Yes + Likelihood for play No) = 0.0053/(0.0053 + 0.0206) = 20.5%

Probability of play No = Likelihood for play No/(Likelihood for play Yes + Likelihood for play No) = 0.0206/(0.0053 + 0.0206) = 79.5%

From this, we can conclude that there are approximately 80% chances that play will not be held and 20% chances that play will be held. This makes a good prediction and Naïve Bayes has performed very well in this case.

But Naïve Bayes has a drawback to understand which, let us consider another example from this dataset under same weather conditions except that now the outlook is overcast instead of rainy. The problem here is that there is no instance in the dataset when outlook is overcast and play is No which indicates that play is always held on overcast day as shown in Figure 5.38. This result about likelihood of play not being held on overcast days is 0 which causes all the other attributes to be rejected. Simply put, we can say that play is always being held when outlook is overcast, regardless of what the other attributes are. The calculations for this are shown in Figure 5.42.

Therefore, likelihood of play not being held under these weather conditions = 0.00 * 0.20 * 0.80 * 0.60 * 0.36 = 0

	Play			Play			Play			Play			Play
Outlook	**yes**	**no**	**Temp.**	**yes**	**no**	**Humid.**	**yes**	**no**	**Windy**	**yes**	**no**		
sunny	0.22	0.60	hot	0.22	0.40	high	0.33	0.80	false	0.67	0.40	yes	0.64
overcast	0.44	0.00	mild	0.44	0.40	normal	0.67	0.20	true	0.33	0.60	no	0.36
rainy	0.33	0.40	cool	0.33	0.20								

Figure 5.42 Probability of play not being held when outlook is overcast

However, we may have reached this result because we may not have enough instances in the dataset especially of cases where play was not held when the outlook was overcast. To solve this issue, a Laplace estimator is used which simply adds a value of 1 to each count of attribute values.

5.7.2 Working of Naïve Bayes classifier using the Laplace Estimator

After adding 1 to each count of attribute values, the modified values are shown in Figure 5.43.

	Play			Play			Play			Play			Play
Outlook	yes	no	Temp.	yes	no	Humid.	yes	no	Windy	yes	no		
sunny	2	3	hot	2	2	high	3	4	false	6	2	yes	9
overcast	4	0	mild	4	2	normal	6	1	true	3	3	no	5
rainy	3	2	cool	3	1								
TOTAL	9	5	TOTAL	9	5	TOTAL	9	5	TOTAL	9	5	TOTAL	14

Laplace estimator:
Add 1 to each count

Add 1 to count (Laplace estimator)

	Play			Play			Play			Play			Play
Outlook	yes	[illegible]	Temp.	yes	no	Humid.	yes	no	Windy	yes	no		
sunny	3	[illegible]	hot	3	3	high	4	5	false	7	3	yes	9
overcast	5	1	mild	5	3	normal	7	2	true	4	4	no	5
rainy	4	3	cool	4	2								
TOTAL	12	8	TOTAL	12	8	TOTAL	11	7	TOTAL	11	7	TOTAL	14

Figure 5.43 Values of attributes after adding Laplace estimator

Now, probability of play being held or not for each modified value of input attribute is recomputed as shown in Figure 5.44.

	Play			Play			Play			Play			Play
Outlook	yes	no	Temp.	yes	no	Humid.	yes	no	Windy	yes	no		
sunny	0.25	0.50	hot	0.25	0.38	high	0.36	0.71	false	0.64	0.43	yes	0.64
overcast	0.42	0.13	mild	0.42	0.38	normal	0.64	0.29	true	0.36	0.57	no	0.36
rainy	0.33	0.38	cool	0.33	0.25								

Figure 5.44 Probability of play held or not for each modified value of attribute

Now, probability of play whether held or not for the given weather conditions such as Outlook = overcast, Temperature = cool, Humidity = high, Windy = true can be calculated from the Figure 5.45.

	Play			Play			Play			Play			Play
Outlook	yes	no	Temp.	yes	no	Humid.	yes	no	Windy	yes	no		
sunny	0.25	0.50	hot	0.25	0.38	high	0.36	0.71	false	0.64	0.43	yes	0.64
overcast	0.42	0.13	mild	0.42	0.38	normal	0.64	0.29	true	0.36	0.57	no	0.36
rainy	0.33	0.38	cool	0.33	0.25								

Figure 5.45 Attribute values for given example instance

Likelihood for play Yes = 0.42 * 0.33 * 0.36 * 0.36* 0.64 = 0.0118

Likelihood for play No = 0.13 * 0.25 * 0.71 * 0.57 * 0.36 = 0.0046

Probability of play Yes = 0.0118/(0.0118+0.0046) = 72%

Probability of play No = 0.0046/(0.0118+0.0046) =28%

From this, it can be concluded that instead of 100% chance of play being held whenever outlook is overcast, we now calculate the probability of 72% for this combination of weather conditions, which is more realistic.

Thus, the Laplace estimator has been used to handle the case of zero probabilities which commonly occurs due to insufficient training data.

5.8 Understanding Metrics to Assess the Quality of Classifiers

The quality of any classifier is measured in terms of True Positive, False Positive, True Negative and False Negative, Precision, Recall and F-Measure.

To understand this concept, let us first consider the story of boy who shouted 'Wolf' to fool the villagers. The story goes as follows.

5.8.1 The boy who cried wolf

Once a boy was getting bored and thought of making fools out of fellow villagers. To have some fun, he shouted out, 'Wolf!' even though no wolf was in sight. The villagers ran to rescue him, but then got angry when they realized that the boy was playing a joke on them. The boy repeated the same prank a number of times and each time the villagers rushed out. They got angrier when they found he was joking.

One night, the boy saw a real wolf approaching and shouted 'Wolf!'. This time villagers stayed in their houses and the hungry wolf turned the flock into lamb chops.

Let's make the following definitions: Here, 'Wolf' is a **positive class** and 'No wolf' is a **negative class**. We can summarize our 'wolf-prediction' model using a 2x2 confusion matrix that depicts all four possible outcomes as shown below.

True Positive (TP): Reality: A wolf threatened. Boy said: 'Wolf.' Outcome: Boy is a hero.	**False Positive (FP):** Reality: No wolf threatened. Boy said: 'Wolf.' Outcome: Villagers are angry at Boy for waking them up.
False Negative (FN): Reality: A wolf threatened. Boy said: 'No wolf.' Outcome: The wolf ate all the flock.	**True Negative (TN):** Reality: No wolf threatened. Boy said: 'No wolf.' Outcome: Everyone is fine.

Now consider a classifier, whose task is to predict whether the image is of a bird or not. In this case, let us assume, 'Bird' is a positive class and 'Not a Bird' is a negative class.

Let us suppose we have a dataset of 15,000 images having 6000 images of birds and 9000 images of anything that is not a bird. The matrix illustrating actual vs. predicted results also known as confusion matrix is given in Figure 5.46 below.

Results for 15,000 Validation Images

(6000 images are birds, 9000 images are not birds)

	Predicted 'bird'	Predicted 'not a bird'
Bird	5,450 ***True Positives***	550 ***False Negatives***
Not a Bird	162 ***False Positives***	8,838 ***True Negatives***

Figure 5.46 Confusion matrix for bird classifier

5.8.2 True positive

Those instances where predicted class is equal to the actual class are called as true positive or a true positive is an outcome where the model correctly predicts the positive class.

For example in the case of our bird classifier, the birds that are correctly identified as birds are called true positive.

5.8.3 True negative

Those instances where predicted class and actual class are both negative are called as true negative or a true negative is an outcome where the model correctly predicts the negative class.

For example, in the case of our bird classifier there are images that are not of birds which our classifier correctly identified as 'not a bird' are called true negatives.

5.8.4 False positive

Those instances where predicted class/answer is positive, but actually the instances are negative or a false positive is an outcome where the model incorrectly predicts the positive class.

For example, in case of our bird classifier there are some images that the classifier predicted as birds but they were something else. These are our false positives.

5.8.5 False negative

Those instances where predicted class is negative, but actually the instances are positive or a false negative is an outcome where the model incorrectly predicts the negative class.

For example, in case of our bird classifier there are some images of birds that the classifier did not correctly recognize as birds. These are our false negatives.

In simple words, predicted 'bird' column is considered as Positive and if the prediction is correct then cell is labeled as true positive, otherwise it is false positive. The column where prediction is 'not a bird' is considered as negative and if prediction is correct, the cell is labeled as true negative otherwise it is false negative as shown in Figure 5.46.

5.8.6 Confusion matrix

Confusion matrix is an $N \times N$ table that summarizes the accuracy of a classification model's predictions. Here, N represents the number of classes. In a binary classification problem, $N = 2$. In simple words, it is a correlation between the actual labels and the model's predicted labels. One axis of a confusion matrix is the label that the model predicted, and the other axis is the actual label.

For example, consider a sample confusion matrix for a binary classification problem to predict if a patient has a tumor or not.

	Tumor (predicted)	*No-Tumor (predicted)*
Tumor (actual)	18 (TP)	1 (FN)
No-Tumor (actual)	*6 (FP)*	*452 (TN)*

Figure 5.47 Confusion matrix for tumor prediction

The confusion matrix shown in Figure 5.47 has 19 samples that actually had tumor, the model correctly classified 18 as having tumor (18 true positives), and incorrectly classified 1 as not having a tumor (1 false negative). Similarly, of 458 samples that actually did not have tumor, 452 were correctly classified (452 true negatives) and 6 were incorrectly classified (6 false positives).

Confusion matrices contain sufficient information to calculate a variety of performance metrics, including precision, recall and F-Measure. Instead of just looking at overall accuracy, we usually calculate precision, recall and F-Measure. These metrics give us a clear picture of how well the classifier performed.

5.8.7 Precision

Precision identifies the frequency with which a model was correct when predicting the positive class. It is defined as:

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive + False Positive}}$$

Or we can say that Precision = True Positives/All Positive predicted. In other words, if we predict positive then how often was it really a positive instance.

For example, in case of our bird classifier, it refers to the situation where a bird was predicted, but how often was it really a bird.

For classifier shown in Figure 5.46, the Precision = 5450/(5450 + 162) =5450/5612 =0.9711. In terms of percentage it is 97.11%.

5.8.8 Recall

Recall identifies out of all the possible positive labels, how many did the model correctly identify? In simple word, it refers to what percentage of actual positive instances we are able to find.

For example, In case of our bird classifier, it refers what percentage of the actual birds did we find?

So,

$$\text{Recall} = \frac{\text{True Positive}}{\text{All actual positive instances}} = \frac{\text{True Positive}}{\text{True Positive + False Negatives}}$$

For classifier shown in Figure 5.46, the Recall = 5450/(5450 + 550) = 5450/6000 = 0.9083 and in terms of percentage it is 90.83%.

The table below shows both precision and recall for the bird classifier discussed above.

Precision *If we predicted 'bird', how often was it really a bird?*	**97.11%** *(True positives ÷ All Positive Guesses)*
Recall *What percentage of the actual birds did we find?*	**90.83%** *(True positives ÷ Total Bird in Dataset)*

In our example of bird classifier, this tells us that 97.11% of the time the prediction was right! But it also tells us that we only found 90.83% of the actual birds in the dataset. In other words, we might not find every bird but we are pretty sure about it when we do find one!

Now, let us consider a multi class classifier, where a classifier has to predict one class out of three classes. Here, we have three classes A, B and C.

		Predicted Class		
		A	*B*	*C*
Actual Class	A	25	3	2
	B	2	34	5
	C	2	3	20

Let's calculate Precision and Recall for each predicted class.

For Class A

Precision = True Positives/All Positive Predicted

Precision = 25/29

Recall = True Positive/All actual positive instances = 25/30

For Class B

Precision = True Positives/All Positive Predicted

Precision = 34/40

Recall = True Positive/All actual positive instances =34/41

For Class C

Precision = True Positives/All Positive Predicted

Precision = 20/27

Recall = True Positive/All actual positive instances =20/25

5.8.9 F-Measure

In statistical analysis of binary classification, the F-Measure (also known as F-score or F-1 score) is a measure of a test's accuracy. It considers both the precision *p* and the recall *r* of the test to compute the score. Here, *p* is the number of correct positive results divided by the number of all positive results returned by the classifier, and *r* is the number of correct positive results divided by the number of all relevant samples (all samples that should have been identified as positive). The F1 score is the harmonic average of the precision and the recall, where, an F1 score reaches its best value at 1 (perfect precision and recall), and its worst at 0.

$$F = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Cluster Analysis

Chapter Objectives

- ✓ To comprehend the concept of clustering, its applications, and features.
- ✓ To understand various distance metrics for clustering of data.
- ✓ To comprehend the process of K-means clustering.
- ✓ To comprehend the process of hierarchical clustering algorithms.
- ✓ To comprehend the process of DBSCAN algorithms.

7.1 Introduction to Cluster Analysis

Generally, in the case of large datasets, data is not labeled because labeling a large number of records requires a great deal of human effort. The unlabeled data can be analyzed with the help of clustering techniques. Clustering is an unsupervised learning technique which does not require a labeled dataset.

Clustering is defined as grouping a set of similar objects into classes or clusters. In other words, during cluster analysis, the data is grouped into classes or clusters, so that records within a cluster (intra-cluster) have high similarity with one another but have high dissimilarities in comparison to objects in other clusters (inter-cluster), as shown in Figure 7.1.

The similarity of records is identified on the basis of values of attributes describing the objects. Cluster analysis is an important human activity. The first human beings Adam and Eve actually learned through the process of clustering. They did not know the name of any object, they simply observed each and every object. Based on the similarity of their properties, they identified these objects in groups or clusters. For example, one group or cluster was named as trees, another as fruits and so on. They further classified the fruits on the basis of their properties like size, colour, shape, taste, and others. After that, people assigned labels or names to these objects calling them mango, banana, orange, and so on. And finally, all objects were labeled. Thus, we can say that the first human beings used clustering for their learning and they made clusters or groups of physical objects based on the similarity of their attributes.

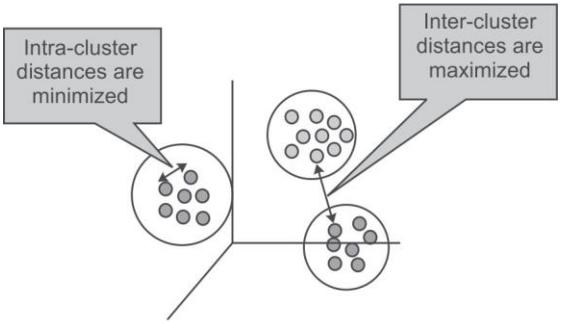


Figure 7.1 Characteristics of clusters

7.2 Applications of Cluster Analysis

Cluster analysis has been widely used in various important applications such as:

- Marketing: It helps marketers find out distinctive groups among their customer bases, and this knowledge helps them improve their targeted marketing programs.
- Land use: Clustering is used for identifying areas of similar land use from the databases of earth observations.
- Insurance: Clustering is helpful for recognizing clusters of insurance policyholders with a high regular claim cost.
- City-planning: It also helps in identifying clusters of houses based on house type, geographical location, and value.
- Earthquake studies: Clustering is helpful for analysis of earthquakes as it has been noticed that earthquake epicenters are clustered along continent faults.
- Biology studies: Clustering helps in defining plant and animal classifications, identifying genes with similar functionalities, and in gaining insights into structures inherent to populations.
- Web discovery: Clustering is helpful in categorizing documents on the web for information discovery.
- Fraud detection: Clustering is also helpful in outlier detection applications such as credit card fraud detection.

7.3 Desired Features of Clustering

The desired feature of an ideal clustering technique is that intra-cluster distances should be minimized and inter-cluster distances should be maximized. Following are the other important features that an ideal cluster analysis method should have:

- ***Scalability:*** Clustering algorithms should be capable of handling small as well as large datasets smoothly.

- ***Ability to handle different types of attributes:*** Clustering algorithms should be able to handle different kinds of data such as binary, categorical and interval-based (numerical) data.
- ***Independent of data input order:*** The clustering results should not be dependent on the ordering of input data.
- ***Identification of clusters with different shapes:*** The clustering algorithm should be capable of identifying clusters of any shape.
- ***Ability to handle noisy data:*** Usually, databases consist of noisy, erroneous or missing data, and algorithm must be able to handle these.
- ***High performance:*** To have a high performance algorithm, it is desirable that the algorithm should need to perform only one scan of the dataset. This capability would reduce the cost of input-output operations.
- ***Interpretability:*** The results of clustering algorithms should be interpretable, logical and usable.
- ***Ability to stop and resume:*** For a large dataset, it is desirable to stop and resume the task as it can take a huge amount of time to accomplish the full task and breaks may be necessary.
- ***Minimal user guidance:*** The clustering algorithm should not expect too much supervision from the analyst, because commonly the analyst has a limited knowledge of the dataset.

In clustering, distance metrics play a vital role in comprehending the similarity between the objects. In the next section, we will discuss different distance metrics that play an important role in the process of clustering of objects.

7.4 Distance Metrics

A distance metric is a function ***d(x, y)*** that specifies the distance between elements of a set as a non-negative real number. Two elements are equal under a particular metric if the distance between them is zero. Distance functions present a method to measure the closeness of two elements. Here, elements can be matrices, vectors or arbitrary objects and do not necessarily need to be numbered.

In the following subsections, important distance metrics used in the measuring similarity among objects have been illustrated.

7.4.1 Euclidean distance

Euclidean distance is mainly used to calculate distances. The distance between two points in the plane with coordinates (x, y) and (a, b) according to the Euclidean distance formula is given by:

$$\boxed{\text{Euclidean dist}((x, y), (a, b)) = \sqrt{(x - a)^2 + (y - b)^2}}$$

For example, the (Euclidean) distance between points (-2, 2) and (2, -1) is calculated as

$$\begin{aligned}\text{Euclidean dist}((-2, 2), (2, -1)) &= \sqrt{(-2 - (2))^2 + (2 - (-1))^2}\\ &= \sqrt{(-4)^2 + (3)^2}\\ &= \sqrt{16 + 9}\\ &= \sqrt{25}\\ &= 5\end{aligned}$$

Let's find the Euclidean distance among three persons to find the similarities and dissimilarities among them, on the basis of two variables. The data is given in Table 7.1.

Table 7.1 Data to calculate Euclidean distances among three persons

	Variable 1	*Variable 2*
Person 1	30	70
Person 2	40	54
Person 3	80	50

Using the formula of Euclidean distance, we can calculate the similarity distance among persons.

The calculation for the distance between person 1 and 2 is:

$$\text{Euclidean dist}((30, 70), (40, 54)) = \sqrt{(30 - 40)^2 + (70 - 54)^2}$$
$$= \sqrt{(-10)^2 + (16)^2}$$
$$= \sqrt{100+256}$$
$$= \sqrt{356}$$
$$= 18.86$$

The calculation for the distance between person 1 and 3 is:

$$\text{Euclidean dist}((30, 70), (80, 50)) = \sqrt{(30 - 80)^2 + (70 - 50)^2}$$
$$= \sqrt{(-50)^2 + (20)^2}$$
$$= \sqrt{2500 + 400}$$
$$= \sqrt{2900}$$
$$= 53.85$$

The calculation for the distance between person 2 and 3 is:

$$\text{Euclidean dist}((40, 54), (80, 50)) = \sqrt{(40 - 80)^2 + (54 - 50)^2}$$
$$= \sqrt{(-40)^2 + (4)^2}$$
$$= \sqrt{1600+16}$$
$$= \sqrt{1616}$$
$$= 40.19$$

This indicates that the persons 1 and 2 are most similar while person 1 and person 3 are most dissimilar. Thus, Euclidean distance is used to determine the dissimilarity between two objects by comparing them across a range of variables. These two objects might be profiles of two persons, e.g., a person and a target profile, or in fact, any two vectors taken across equivalent variables.

In the Euclidean distance, the attribute with the largest value may dominate the distance. Thus, it's important that attributes should be properly scaled before the application of the formula. It is also possible to use the Euclidean distance formula without the square root if one wishes to place greater weightage on large differences. Euclidean distance can also be represented as shown in Figure 7.2.

Figure 7.2 Representation of Euclidean distance [see colour plate]

[*Credits*: https://numerics.mathdotnet.com/Distance.html]

7.4.2 Manhattan distance

Manhattan distance is also called L1-distance. It is defined as the sum of the lengths of the projections of the line segment between the two points on the coordinate axes.

For example, the distance between two points in the plane with coordinates (x, y) and (a, b) according to the Manhattan distance formula, is given by:

$$\boxed{\text{Manhattan dist}((x, y), (a, b)) = |x - a| + |y - b|}$$

Let's do the calculations for finding the Manhattan distance among the same three persons, on the basis of their scores on two variables as shown in Table 7.1.

Using the formula of Manhattan distance, we can calculate the similarity distance among persons.

The calculation for the distance between person 1 and 2 is:

$$\begin{aligned} \text{Manhattan dist}((30, 70), (40, 54)) &= |30 - 40| + |70 - 54| \\ &= |-10| + |16| \\ &= 10+16 \\ &= 26 \end{aligned}$$

The calculation for the distance between person 1 and 3 is:

$$\begin{aligned} \text{Manhattan dist}((30, 70), (80, 50)) &= |30 - 80| + |70 - 50| \\ &= 50+20 \\ &= 70 \end{aligned}$$

The calculation for the distance between person 2 and 3 is:

$$\begin{aligned} \text{Manhattan dist}((40, 54), (80, 50)) &= |40 - 80| + |54 - 50| \\ &= 40+4 = 44 \end{aligned}$$

This indicates that the persons 1 and 2 are most similar while person 1 and person 3 are most dissimilar and it produces the same conclusion as Euclidean distance.

Manhattan distance is also called city block distance because like Manhattan, it is the distance a car would drive in a city laid out in square blocks. In Manhattan city, one-way, oblique streets and real streets only exist at the edges of blocks. The Manhattan distance can be represented as shown in Figure 7.3.

Figure 7.3 Representation of Manhattan distance [see colour plate]
[*Credits*: https://numerics.mathdotnet.com/Distance.html]

In Manhattan distance, the largest valued attribute can again dominate the distance, although not as much as in the Euclidean distance.

7.4.3 Chebyshev distance

It is also called as chessboard distance because, in a game of chess, the minimum number of moves required by a king to go from one square to another on a chessboard equals Chebyshev distance between the centers of the squares. Chebyshev distance is defined on a vector space, where the distance between two vectors is the maximum value of their differences along any coordinate dimension.

Formula of Chebyshev distance is given by:

$$\text{Chebyshev dist}((r1, f1), (r2, f2)) = \max(|r2-r1|, |f2-f1|)$$

Using the formula of Chebyshev distance, let us find the distance between object A and object B.

Features	*Coord1*	*Coord2*	*Coord3*	*Coord4*
Object A	0	1	2	3
Object B	6	5	4	-2

Object A coordinate = {0,1,2,3}
Object B coordinate = {6,5,4,-2}
According to Chebyshev distance formula

$$\begin{aligned} D &= \max(|r2-r1|, |f2-f1|) \\ &= \max(|6-0|, |5-1|, |4-2|, |-2-3)|) \\ &= \max(6,4,2,5) = 6 \end{aligned}$$

Let's compute the calculations for finding the Chebyshev distance among three persons, on the basis of their scores on two variables as shown in Table 7.1.

The calculation for the distance between person 1 and 2 is:

$$\begin{aligned} \text{Chebyshev dist}((30, 70), (40, 54)) &= \max(|30 - 40|, |70 - 54|) \\ &= \max(|-10|, |16|) = 16 \end{aligned}$$

The calculation for the distance between person 1 and 3 is:

$$\begin{aligned} \text{Chebyshev dist}((30, 70), (80, 50)) &= \max(|30 - 80|, |70 - 50|) \\ &= \max(50,20) = 50 \end{aligned}$$

The calculation for the distance between person 2 and 3 is:

$$= \max(|40-80|, |54-50|)$$

$$\text{Chebyshev dist}((40, 54), (80, 50)) = \max(40,4) = 40$$

This indicates that the persons 1 and 2 are most similar while person 1 and person 3 are most dissimilar and it produces the same conclusion as Euclidean and Manhattan distance.

The Chebyshev distance can be represented as shown in Figure 7.4.

Figure 7.4 Representation of Chebyshev distance [see colour plate]

[*Credits*: https://numerics.mathdotnet.com/Distance.html]

7.5 Major Clustering Methods/Algorithms

Clustering methods/algorithms can be categorized into five categories which are given as follows:

- ***Partitioning method:*** It constructs random partitions and then iteratively refines them by some criterion.
- ***Hierarchical method:*** It creates a hierarchical decomposition of the set of data (or objects) using some criterion.
- ***Density-based method:*** It is based on connectivity and density functions.
- ***Grid based method:*** It is based on a multiple-level granularity structure.
- ***Model based method:*** A model is considered for each of the clusters and the idea is to identify the best fit for that model.

The categorization of clustering algorithms is shown in Figure 7.5.

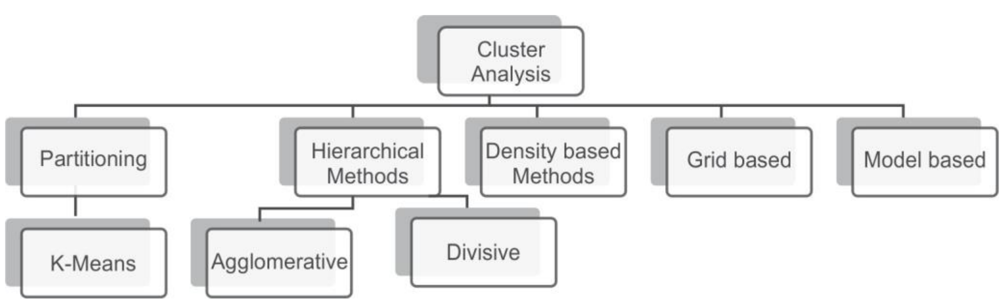


Figure 7.5 Major clustering methods/algorithms

Now, let us understand each method one by one.

7.6 Partitioning Clustering

Clustering is the task of splitting a group of data or dataset into a small number of clusters. For example, the items in a grocery store are grouped into different categories (butter, milk, and cheese are clustered in dairy products). This is a qualitative kind of partitioning. A quantitative approach on the other hand, measures certain features of the products such as the percentage of milk and suchlike, i.e., products having a high percentage of milk would be clustered together.

In the partitioning method, we cluster objects based on attributes into a number of partitions. The k-means clustering is an important technique which falls under partitioning clustering.

7.6.1. k-means clustering

In the k-means clustering algorithm, n objects are clustered into k clusters or partitions on the basis of attributes, where $k < n$ and k is a positive integer number. In simple words, in k-means clustering algorithm, the objects are grouped into 'k' number of clusters on the basis of attributes or features. The grouping of objects is done by minimizing the sum of squares of distances, i.e., a Euclidean distance between data and the corresponding cluster centroid.

Working of the k-means algorithm

The working of k-means clustering algorithm can be illustrated in five simple steps, as given below.

Step 1: Start with a selection of the value of k where k = number of clusters

In this step, the k centroids (or clusters) are initiated and then the first $\boldsymbol{k}$ training samples out of $\boldsymbol{n}$ samples of data are taken as single-element clusters. Each of the remaining (n-k) training samples are assigned to the cluster with the nearest centroid and the centroid of the gaining cluster is recomputed after each assignment.

Step 2: Creation of distance matrix between the centroids and each pattern

In this step, distance is computed from each sample of data to the centroid of each of the clusters. The heavy calculation involved is the major drawback of this step since there are k centroids and n samples, the algorithm will have to compute $n*k$ distances.

Step 3: Assign each sample in the cluster with the closest centroid (minimal distance)

Now, the samples are grouped on the basis of their distance from the centroid of each of the clusters. If currently, the sample is not in the cluster then switch it to the cluster with the closest centroid. When there is no movement of samples to another cluster anymore, the algorithm will end.

Step 4: Update the new centroids for each cluster

In this step, the locations of centroids are updated. Update the location for each centroid of the cluster that has gained or lost a sample by computing the mean of each attribute of all samples belonging to respective clusters.

Step 5: Repeat until no further change occurs

Return to Step 2 of the algorithm and repeat the updating process of each centroid location until a convergence condition is satisfied, which is until a pass through the training sample causes no new changes.

The flow chart of k-means algorithm is illustrated in Figure 7.6.

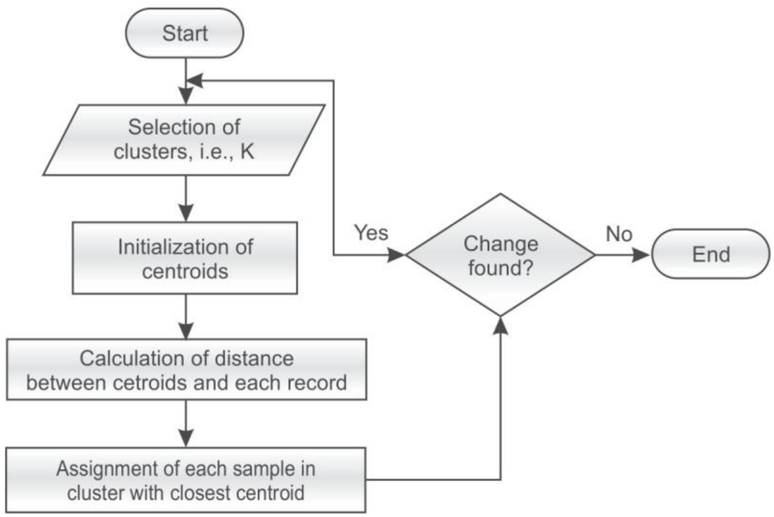


Figure 7.6 Flowchart for k-means algorithm

Example 7.1: Let's understand this procedure through an example; the database for which is given in Table 7.2. Here k=2 (there are just going to be two clusters).

Table 7.2 Database for the k-means algorithm example

Individual	*Variable 1*	*Variable 2*
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Step 1:

The first step will be to choose the value of *k*. In this problem, let's suppose, *k*=2, i.e., we are going to create two clusters C1 and C2 from this database. Let's suppose, record number 1^{st} and 4^{th} are randomly chosen as candidates for clusters (By just browsing the database given in Table 7.2, we can conclude that 1st and 4th are the most dissimilar records or farthest from each other. Otherwise, we can choose any two records as the starting point for initialization of this algorithm).

In this case, the two centroids are: centroid 1 (first record) = (1.0,1.0) and centroid 2 (fourth record) = (5.0,7.0) as shown in Figure 7.7.

Individual	*Variable 1*	*Variable 2*
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

	Individual	*Mean Vector*
Group 1	1	(1.0, 1.0)
Group 2	4	(5.0,7.0)

Figure 7.7 Database after initialization

Step 2:

Now, the distance of each object is calculated from the other objects in this database on the basis of Euclidean distance as shown in Table 7.3.

The Euclidean distance of record 2 (1.5,2.0) from centroid 1 and centroid 2 has been given below.

Table 7.3 Database after first iteration

Individual	*Centroid$_1$ (1,1) for C1*	*Centroid$_2$ (5,7) for C2*	*Assigned Cluster*
1 (1.0,1.0)	0	$\sqrt{\lvert 1-5\rvert^2+\lvert 1-7\rvert^2}$ $=\sqrt{16+36}$ $=\sqrt{52}$ = 7.21	C1
2 (1.5,2.0)	$\sqrt{\lvert 1.5-1\rvert^2+\lvert 2.0-1\rvert^2}$ $=\sqrt{0.25+1}$ $=\sqrt{1.25}$ = 1.12	$\sqrt{\lvert 1.5-5\rvert^2+\lvert 2.0-7\rvert^2}$ $=\sqrt{12.25+25}$ $=\sqrt{37.25}$ = 6.10	C1

Contd.

Individual	*Centroid$_1$ (1,1) for C1*	*Centroid$_2$ (5,7) for C2*	*Assigned Cluster*
3 (3.0,4.0)	$\sqrt{\lvert 3-1\rvert^2+\lvert 4-1\rvert^2}$ $= \sqrt{4+9}$ $= \sqrt{13}$ $= 3.61$	$\sqrt{\lvert 3-5\rvert^2+\lvert 4.0-7\rvert^2}$ $= \sqrt{4+9}$ $= \sqrt{13}$ $= 3.61$	C1
4 (5.0, 7.0)	$\sqrt{\lvert 5-1\rvert^2+\lvert 7-1\rvert^2}$ $= \sqrt{16+36}$ $= \sqrt{52}$ $= 7.21$	$\sqrt{\lvert 5-5\rvert^2+\lvert 7.0-7\rvert^2}$ $= \sqrt{0}$ $= 0$	C2
5 (3.5, 5.0)	$\sqrt{\lvert 3.5-1\rvert^2+\lvert 5-1\rvert^2}$ $= \sqrt{6.25+16}$ $= \sqrt{22.25}$ $= 4.72$	$\sqrt{\lvert 3.5-5\rvert^2+\lvert 5.0-7\rvert^2}$ $= \sqrt{2.25+4}$ $= 2.5$	C2
6 (4.5, 5.0)	$\sqrt{\lvert 4.5-1\rvert^2+\lvert 5.0-1\rvert^2}$ $= \sqrt{12.25+16}$ $= \sqrt{28.25}$ $= 5.31$	$\sqrt{\lvert 4.5-5\rvert^2+\lvert 5.0-7\rvert^2}$ $= \sqrt{0.25+4}$ $= 2.06$	C2
7 (3.5, 4.5)	$\sqrt{\lvert 3.5-1\rvert^2+\lvert 4.5-1\rvert^2}$ $= \sqrt{6.25+12.25}$ $= \sqrt{18.5}$ $= 4.30$	$\sqrt{\lvert 3.5-5\rvert^2+\lvert 4.5-7\rvert^2}$ $= \sqrt{2.25+6.25}$ $= \sqrt{8.5}$ $= 2.92$	C2

Thus, we obtain two clusters containing: {1,2,3} and {4,5,6,7}.

The updated centroid for cluster 1 will be average of instances 1, 2 and 3 and for cluster 2 it will be average of instances 4, 5, 6 and 7 as shown below.

$$\text{centroid}_1 = \left(\frac{1}{3}(1.0 + 1.5 + 3.0),\ \frac{1}{3}(1.0 + 2.0 + 4.0)\right) = (1.83, 2.33)$$

$$\text{centroid}_2 = \left(\frac{1}{4}(5.0 + 3.5 + 4.5 + 3.5),\ \frac{1}{4}(7.0 + 5.0 + 5.0 + 4.5)\right) = (4.12, 5.38)$$

Now, the distance of each object to the modified centroid is calculated as shown in Table 7.4.

Table 7.4 Database after the second iteration

Individual	*Centroid$_1$ (1.83,2.33) for C1*	*Centroid$_2$ (4.12,5.38) for C2*	*Assigned Cluster*
1 (1.0,1.0)	$\sqrt{\lvert 1.0-1.83\rvert^2+\lvert 1.0-2.33\rvert^2} = 1.56$	$\sqrt{\lvert 1.0-4.12\rvert^2+\lvert 1.0-5.83\rvert^2} = 4.83$	C1
2 (1.5,2.0)	$\sqrt{\lvert 1.5-1.83\rvert^2+\lvert 2.0-2.33\rvert^2} = 0.46$	$\sqrt{\lvert 1.5-4.12\rvert^2+\lvert 2.0-5.83\rvert^2} = 4.64$	C1
3 (3.0,4.0)	$\sqrt{\lvert 3.0-1.83\rvert^2+\lvert 4.0-2.33\rvert^2} = 2.03$	$\sqrt{\lvert 3.0-4.12\rvert^2+\lvert 4.0-5.83\rvert^2} = 2.14$	C2
4 (5.0,7.0)	$\sqrt{\lvert 5.0-1.83\rvert^2+\lvert 7.0-2.33\rvert^2} = 5.64$	$\sqrt{\lvert 5.0-4.12\rvert^2+\lvert 7.0-5.83\rvert^2} = 1.84$	C2
5 (3.5,5.0)	$\sqrt{\lvert 3.5-1.83\rvert^2+\lvert 5.0-2.33\rvert^2} = 3.14$	$\sqrt{\lvert 3.5-4.12\rvert^2+\lvert 5.0-5.83\rvert^2} = 1.03$	C2
6 (4.5,5.0)	$\sqrt{\lvert 4.5-1.83\rvert^2+\lvert 5.0-2.33\rvert^2} = 3.77$	$\sqrt{\lvert 4.5-4.12\rvert^2+\lvert 5.0-5.83\rvert^2} = 0.91$	C2

The clusters obtained are: {1,2} and {3,4,5,6,7}

The updated centroid for cluster 1 will be average of instances 1 and 2 and for cluster 2 it will be average of instances 3, 4, 5, 6 and 7 as shown below.

centroid$_1$ = (1/2 (1.0+1.5), 1/2 (1.0+2.0)) = (1.25,1.5)

centroid$_2$ = (1/5 (3.0+5.0+3.5+4.5+3.5), 1/5 (4.0+7.0+5.0+5.0+4.5)) = (3.9,5.1)

Now, the distance of each object to the modified centroid is calculated as shown in Table 7.5.

Table 7.5 Database after the second iteration

Individual	*Centroid$_1$ (1.25, 1.5) for C1*	*Centroid$_2$ (3.9, 5.1) for C2*	*Assigned Cluster*
1 (1.0,1.0)	$\sqrt{\lvert 1-1.25\rvert^2+\lvert 1.0-1.5\rvert^2} = 0.56$	$\sqrt{\lvert 1-3.9\rvert^2+\lvert 1-5.1\rvert^2} = 5.02$	C1
2 (1.5,2.0)	$\sqrt{\lvert 1.5-1.25\rvert^2+\lvert 2.0-1.5\rvert^2} = 0.56$	$\sqrt{\lvert 1.5-3.9\rvert^2+\lvert 2.0-5.1\rvert^2} = 3.92$	C1
3 (3.0,4.0)	$\sqrt{\lvert 3-1.25\rvert^2+\lvert 4-1.5\rvert^2} = 3.05$	$\sqrt{\lvert 3-3.9\rvert^2+\lvert 4.0-5.1\rvert^2} = 1.42$	C2
4 (5.0, 7.0)	$\sqrt{\lvert 5-1.25\rvert^2+\lvert 7-1.5\rvert^2} = 6.66$	$\sqrt{\lvert 5-3.9\rvert^2+\lvert 7.0-5.1\rvert^2} = 2.20$	C2
5 (3.5, 5.0)	$\sqrt{\lvert 3.5-1.25\rvert^2+\lvert 5-1.5\rvert^2} = 4.16$	$\sqrt{\lvert 3.5-3.9\rvert^2+\lvert 5.0-5.1\rvert^2} = 0.41$	C2
6 (4.5, 5.0)	$\sqrt{\lvert 4.5-1.25\rvert^2+\lvert 5.0-1.5\rvert^2} = 4.78$	$\sqrt{\lvert 4.5-3.9\rvert^2+\lvert 5.0-5.1\rvert^2} = 0.61$	C2
7 (3.5, 4.5)	$\sqrt{\lvert 3.5-1.25\rvert^2+\lvert 4.5-1.5\rvert^2} = 3.75$	$\sqrt{\lvert 3.5-3.9\rvert^2+\lvert 4.5-5.1\rvert^2} = 0.72$	C2

The clusters obtained are: {1,2} and {3,4,5,6,7}

At this iteration, there is no further change in the clusters. Thus, the algorithm comes to a halt here and final result consists of 2 clusters, {1,2} and {3,4,5,6,7} respectively. These instances can be plotted as two clusters as shown in Figure 7.8.

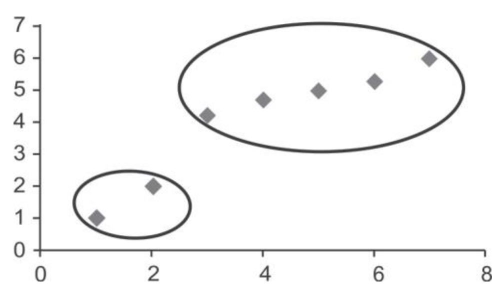


Figure 7.8 Plot of data for k=2 [see colour plate]

Step 3: Implementation of the k-means algorithm (using k=3)

For the same database, if k = 3; it means that there will be three clusters for this database. Let's randomly choose first three records as candidates for clusters. This time, readers are advised to perform the calculations and compare their results with final clusters data given in Table 7.6.

Table 7.6 Initial dataset for k = 3

Individual	*Centroid1 = 1*	*Centroid2 = 2*	*Centroid3 = 3*	*Assigned cluster*
1	0	1.11	3.61	C1
2	1.12	0	2.5	C2
3	3.61	2.5	0	C3
4	7.21	6.10	3.61	C3
5	4.72	3.61	1.12	C3
6	5.31	4.24	1.80	C3
7	4.30	3.20	0.71	C3

Clustering with initial centroids (1,2,3)

Now, if we calculate the centroid distances as discussed above, the algorithm will stop at the state of the database as shown in Table 7.7, by putting instance 1 in cluster 1, instance 2 in cluster 2 and instances 3,4,5,6 and 7 in cluster 3 as shown below.

Table 7.7 Final assigned cluster for k = 3

Individual	*Centroid1(0.56,0.55,3.05) for C1*	*Centroid2(4.21, 3.10, 0.61) for C2*	*Centroid3(6.26,5.17,0.56) for C3*	*Assigned cluster*
1	0	1.11	5.02	C1
2	1.12	0	3.92	C2
3	3.61	2.5	1.42	C3
4	7.21	6.10	2.20	C3
5	4.72	3.61	0.41	C3
6	5.31	4.24	0.61	C3
7	4.30	3.20	0.72	C3

The plot of this data for k = 3 is shown below in Figure 7.9.

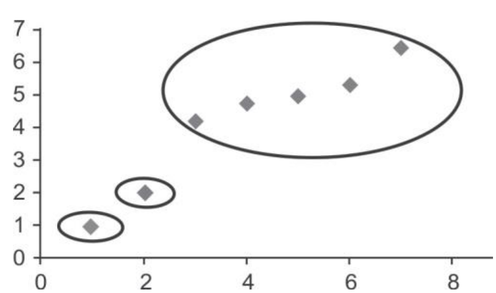


Figure 7.9 Plot of data for k=3 [see colour plate]

Example 7.2: Suppose the visitors visiting a website are to be grouped on the basis of their age given as follows:

15,15,16,19,19,20,20,21,22,28,35,40,41,42,43,44,60,61,65

To perform clustering for the given data, the first step will be to choose the value of k. In this case, let's suppose, k = 2, i.e., we are going to create just two clusters for this database and suppose we randomly choose the 3rd record i.e, 16 and the 9th , i.e., 22 as the initial centroids for the two clusters.

Thus: Centroid for C1 = 16 [16]

Centroid for C2 = 22 [22]

The distances of each record from centroid 1 and centroid 2 is given in Table 7.8. Here, the Manhattan distance has been used to calculate the distance metric which is the absolute distance between the points in this case. It has been used to make the calculations simple.

Table 7.8 Dataset after first iteration

Instances	*Centroid 16 for C1*	*Centroid 22 for C2*	*Assigned Cluster*
15	1	7	C1
15	1	7	C1
16	0	6	C1
19	3	3	C2
19	3	3	C2
20	4	2	C2
20	4	2	C2
21	5	1	C2
22	6	0	C2
28	12	6	C2
35	19	13	C2
40	24	18	C2
41	25	19	C2
42	26	20	C2
43	27	21	C2
44	28	22	C2
60	44	38	C2
61	45	39	C2
65	49	43	C2

Thus, we obtain two clusters containing:

{15, 15, 16} and {19,19,20,20,21,22,28,35,40,41,42,43,44,60,61,65}.

The updated centroid for clusters will be as follows:
Centroid1 = avg(15, 15, 16) = 15.33
Centroid2 = avg(19,19,20,20,21,22,28,35,40,41,42,43,44,60,61,65) = 36.25
Now, the distance of each object to modified centroid is calculated as shown in Table 7.9.
The least actual distances from the instance to the centroid have been highlighted.

Table 7.9 Dataset after second iteration

Instances	*Centroid 15.33 for C1*	*Centroid 36.25 for C2*	*Assigned Cluster*
15	0.33	21.25	C1
15	0.33	21.25	C1
16	0.67	20.25	C1
19	3.67	17.25	C1
19	3.67	17.25	C1
20	4.67	16.25	C1
20	4.67	16.25	C1
21	5.67	15.25	C1
22	6.67	14.25	C1
28	12.67	8.25	C2
35	19.67	1.25	C2
40	24.67	3.75	C2
41	25.67	4.75	C2
42	26.67	5.75	C2
43	27.67	6.75	C2
44	28.67	7.75	C2
60	44.67	23.75	C2
61	45.67	24.75	C2
65	49.67	28.75	C2

Now, we have obtain two fresh clusters containing:

{15,15,16,19,19,20,20,21,22} and {28,35,40,41,42,43,44,60,61,65}.

The updated centroids for these fresh clusters will be as follows:
Centroid 1 = avg(15,15,16,19,19,20,20,21,22) = 18.55
Centroid 2 = avg(28,35,40,41,42,43,44,60,61,65) = 45.9
Next, the distance of each object to the modified centroid is calculated as shown in Table 7.10

Table 7.10 Dataset after third iteration

Instances	*Centroid 18.55 for C1*	*Centroid 45.9 for C2*	*Assigned Cluster*
15	3.55	30.9	C1
15	3.55	30.9	C1
16	2.55	29.9	C1
19	0.45	26.9	C1
19	0.45	26.9	C1
20	1.45	25.9	C1
20	1.45	25.9	C1
21	2.45	24.9	C1
22	3.45	23.9	C1
28	9.45	17.9	C1
35	16.45	10.9	C2
40	21.45	5.9	C2
41	22.45	4.9	C2
42	23.45	3.9	C2
43	24.45	2.9	C2
44	25.45	1.9	C2
60	41.45	14.1	C2
61	42.45	15.1	C2
65	46.45	19.1	C2

Here, we obtain two clusters with still fresher changes containing:

{15,15,16,19,19,20,20,21,22,28} and {35,40,41,42,43,44,60,61,65}.

The updated centroid for clusters will be as follows:
Centroid 1 = avg(15,15,16,19,19,20,20,21,22,28) =19.5
Centroid 2 = avg(35,40,41,42,43,44,60,61,65) = 47.89
Now, the distance of each object to the most recently modified centroids is calculated as shown in Table 7.11.

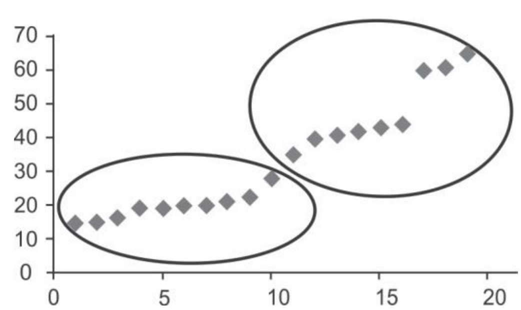
Table 7.11 Dataset after fourth iteration

Instances	*Centroid 19.50 for C1*	*Centroid 47.89 for C2*	*Assigned Cluster*
15	4.5	32.89	C1
15	4.5	32.89	C1
16	3.5	31.89	C1
19	0.5	28.89	C1
19	0.5	28.89	C1
20	0.5	27.89	C1
20	0.5	27.89	C1
21	1.5	26.89	C1
22	2.5	25.89	C1
28	8.5	19.89	C1
35	15.5	12.89	C2
40	20.5	7.89	C2
41	21.5	6.89	C2
42	22.5	5.89	C2
43	23.5	4.89	C2
44	24.5	3.89	C2
60	40.5	12.11	C2
61	41.5	13.11	C2
65	45.5	17.11	C2

At this iteration, no object has shifted from one cluster to another, i.e., no change has been found between iterations 3 and 4. Thus, the algorithm will stop at fourth iteration and the final clusters will be as follows.

Cluster 1 = [15,15,16,19,19,20,20,21,22,28]

Cluster 2 = [35,40,41,42,43,44,60,61,65]



[see colour plate]

Hence, two groups, i.e., 15-28 and 35-65 have been found by using clustering. The output clusters can be affected by the initial choice of centroids. The algorithm is run several times with different starting conditions in order to get a fair view of clusters.

Example 7.3: Clustering of Students record with the k-means algorithm

Let us consider another database containing the result of students' examination in a given course. Here, we wish to cluster the students according to their performance in the course. The students' marks (obtained out of 100) across Quiz1, MSE (Mid Semester Exam), Quiz2, and ESE (End Semester Exam) are given in Table 7.12.

Table 7.12 Record of students' performance

Sr. No	*Quiz1*	*MSE*	*Quiz2*	*ESE*
S1	8	20	6	45
S2	6	18	7	42
S3	5	15	6	35
S4	4	13	5	25
S5	9	21	8	48
S6	7	20	9	44
S7	9	17	8	49
S8	8	19	7	39
S9	3	14	4	22
S10	6	15	7	32

Steps 1 and 2:

Let's consider the first three students as the three seeds as shown in Table 7.13.

Table 7.13 Seed records

Sr. no	*Quiz1*	*MSE*	*Quiz2*	*ESE*
S1	8	20	6	45
S2	6	18	7	42
S3	5	15	6	35

Steps 3 and 4:

In this step, calculate the distance using the four attributes as well as the sum of absolute differences for simplicity, i.e., Manhattan distance. Table 7.14 shows the distance values for all the objects, wherein columns 7, 8 and 9 represent the three distances from the three seed records respectively.

On the basis of distances computed, each student is allocated to the nearest cluster, i.e., C1, C2 or C3. The result of the first iteration is given in Table 7.14.

Table 7.14 First iteration-allocation of each object to its nearest cluster

		Quiz1	*MSE*	*Quiz2*	*ESE*	*Distances from Cluster*			*Allocation to the nearest Cluster*
Seed Records	*C1*	*8*	*20*	*6*	*45*				
	C2	*6*	*18*	*7*	*42*				
	C3	*5*	*15*	*6*	*35*	*C1*	*C2*	*C3*	
S1	1	8	20	6	45	$(\lvert 8-8\rvert+\lvert 20-20\rvert+\lvert 6-6\rvert+\lvert 45-45\rvert) = 0$	$(\lvert 8-6\rvert+\lvert 20-18\rvert+\lvert 6-7\rvert+\lvert 45-42\rvert) = 8$	$(\lvert 8-5\rvert+\lvert 20-15\rvert+\lvert 6-6\rvert+\lvert 45-35\rvert) = 18$	C1
S2	2	6	18	7	42	$(\lvert 6-8\rvert+\lvert 18-20\rvert+\lvert 7-6\rvert+\lvert 42-45\rvert) = 8$	$(\lvert 6-6\rvert+\lvert 18-18\rvert+\lvert 7-7\rvert+\lvert 42-42\rvert) = 0$	$(\lvert 6-5\rvert+\lvert 18-15\rvert+\lvert 7-6\rvert+\lvert 42-35\rvert) = 12$	C2
S3	3	5	15	6	35	$(\lvert 5-8\rvert+\lvert 15-20\rvert+\lvert 6-6\rvert+\lvert 35-45\rvert) = 18$	$(\lvert 5-6\rvert+\lvert 15-18\rvert+\lvert 6-7\rvert+\lvert 35-42\rvert) = 12$	$(\lvert 5-5\rvert+\lvert 15-15\rvert+\lvert 6-6\rvert+\lvert 35-35\rvert) = 0$	C3
S4	4	4	13	5	25	32	26	14	C3
S5	5	9	21	8	48	7	13	25	C1
S6	6	7	20	9	44	5	7	19	C1
S7	7	9	17	8	49	10	12	22	C1
S8	8	8	19	7	39	8	6	12	C2
S9	9	3	14	4	22	36	30	18	C3
S10	10	6	15	7	32	21	13	5	C3

Instances in Cluster 1, i.e., C1 are 1, 5, 6 and 7

Instances in Cluster 2, i.e., C2 are 2 and 8

Instances in Cluster 3, i.e., C3 are 3, 4, 9 and 10

The first iteration results in four, two and four students in the first, second and third cluster, respectively.

Step 5:

New centroids for each attribute/column are updated after the first iteration as shown in Table 7.15.

Table 7.15 Updated centroids after first iteration

	Quiz1	*MSE*	*Quiz2*	*ESE*
C1 (record 1,5,6,7)	8.25 Avg(8,9,7,9)	19.5 Avg(20,21,20,17)	7.75 Avg(6,8,9,7)	46.5 Avg(45,48,44,39)
C2 (record 2,8)	7 Avg(6,8)	18.5 Avg(18,19)	7 Avg(7,7)	40.5 Avg(42,39)
C3 (record 3,4,9, 10)	4.5 Avg(5,4,3,6)	14.25 Avg(15,13,14,15)	5.5 Avg(6,5,4,7)	28.5 Avg(35,25,22,32)

It is interesting to note that the mean marks for cluster C3 are significantly lower than for clusters C1 and C2. So, the distances of each object to each of the means using new cluster are again re-computed and allocate each object to the nearest cluster. The results of the second iteration are shown in Table 7.16.

Table 7.16 Second iteration-allocation of each object to its nearest cluster

Seed Records		*Quiz1*	*MSE*	*Quiz2*	*ESE*	*Distances from Cluster*			*Allocation to the nearest Cluster*
	C1	*8.25*	*19.5*	*7.75*	*46.5*				
	C2	*7*	*18.5*	*7*	*40.5*				
	C3	*4.5*	*14.25*	*5.5*	*28.5*	*C1*	*C2*	*C3*	
Student 1	1	8	20	6	45	(|8-8.25|+|20-19.5|+|6-7.75|+|45-46.5|) = 4	(|8-7|+|20-18.5|+|6-7|+|45-40.5|) = 8	(|8-4.5|+|20-14.25|+|6-5.5|+|45-28.5|) = 26.25	C1
Student 2	2	6	18	7	42	(|6-8.25|+|18-19.5|+|7-7.75|+|42-46.5|) = 9	(|6-7|+|18-18.5|+|7-7|+|42-40.5|) = 3	(|6-4.5|+|18-14.25|+|7-5.5|+|42-28.5|) = 20.25	C2
Student 3	3	5	15	6	35	21	12	8.25	C3
Student 4	4	4	13	5	25	35	26	5.75	C3
Student 5	5	9	21	8	48	4	13	33.25	C1
Student 6	6	7	20	9	44	5.5	7	27.25	C1
Student 7	7	9	17	8	49	6	13	30.25	C1
Student 8	8	8	19	7	39	9	3	20.25	C2
Student 9	9	3	14	4	22	39	30	9.75	C3
Student 10	10	6	15	7	32	22	13	7.25	C3

Again, there are four, two and four students in clusters C1, C2 and C3, respectively. A more careful look shows that there is no change in clusters at all. Therefore, the method has converged quite quickly for this very simple dataset. For more clarity, the final assigned cluster has been reproduced in Table 7.17.

Table 7.17 Final allocation

Seed Records		*Quiz1*	*MSE*	*Quiz2*	*ESE*	*Allocation to Cluster*
	C1	*8.25*	*19.5*	*7.75*	*46.5*	
	C2	*7*	*18.5*	*7*	*40.5*	
	C3	*4.5*	*14.25*	*5.5*	*28.5*	
S1	1	8	20	6	45	C1
S2	2	6	18	7	42	C2
S3	3	5	15	6	35	C3
S4	4	4	13	5	25	C3
S5	5	9	21	8	48	C1
S6	6	7	20	9	44	C1
S7	7	9	17	8	49	C1
S8	8	8	19	7	39	C2
S9	9	3	14	4	22	C3
S10	10	6	15	7	32	C3

The membership of clusters is as follows:

Cluster 1, i.e., C1 - 1,5,6,7

Cluster 2, i.e., C2 - 2, 8

Cluster 3, i.e., C3 - 3, 4, 9, 10

It should be noted that the above method was k-median rather than k-means because we wanted to use a simple distance measure that a reader could check without using a computer or a calculator.

Interestingly, when k-means is used, the result is exactly the same and the method again converges rather quickly.

Another point worth noting is about the intra (within) cluster variance and inter (between) cluster variance. The average Euclidean distance of objects in each cluster to the cluster centroids is presented in Table 7.18. The average distance of objects within clusters C1, C2, and C3 from their centroids is 2.89, 1.87 and 5.51, respectively. Although these numbers do not indicate which result is better, they help however, in analyzing that the clustering method has performed well in maximizing inter (between) cluster variance and minimizing intra (within) cluster variance. Here, one can notice that the Euclidean distance between C1 and C2 is 6.89, while it is 44.89 in case of C1 and C3. The intra cluster distance in the case of C1, is the minimum i.e., 2.89 between the instances of C1. Similarly, the intra cluster distance is minimum in case of C2 and C3.

We can get different results with different seed records.

Table 7.18 Within (intra) cluster and between (inter) clusters distance

Cluster	C1	C2	C3
C1	2.89	6.89	19.46
C2	6.52	1.87	13.19
C3	20.02	14.17	5.51

The detail of calculations for the identification of within (intra) cluster and between (inter) cluster variance is shown in Table 7.19.

Table 7.19 Calculations for within-cluster and between-cluster variance using Euclidean distance

Calculation	*Quiz1*	*MSE*	*Quiz2*	*ESE*	*Description*
x_i-y_i for S1 of C1	-0.25 [8-8.25]	0.5 [20-19.5]	-1.75 [6-7.75]	-1.5 [45-46.5]	For Cluster C1
x_i-y_i for S5 of C1	0.75	1.5	0.25	1.5	
x_i-y_i for S6 of C1	-1.25	0.5	1.25	-2.5	
x_i-y_i for S7 of C1	0.75	-2.5	0.25	2.5	
sumsqr(x_i-y_i)	5.625 sumsqr(-0.25,0.5,-1.75,-1.5)	for S1			
sumsqr(x_i-y_i)	5.125 sumsqr(0.75,1.5,0.25, 1.5)	for S5			
sumsqr(x_i-y_i)	9.625	for S6			
sumsqr(x_i-y_i)	13.125	for S7			
avg(1,56,7)	8.375 avg (5.625, 5.125, 9.625, 13.125)	for 1,5,6,7 of c1 from Cluster C1			
sqrt (avg(1,56,7))	**2.893959**	for 1,5,6,7 of c1, Thus within-cluster variance of C1 is 2.89			
x_i-y_i for S1 of C1	1	1.5	-1	4.5	For Cluster C2
x_i-y_i for S5 of C1	2	2.5	1	7.5	
x_i-y_i for S6 of C1	0	1.5	2	3.5	
x_i-y_i for S7 of C1	2	-1.5	1	8.5	

Contd.

Calculation	*Quiz1*	*MSE*	*Quiz2*	*ESE*	*Description*
sqr(x_i-y_i)	24.5	for S1			
sqr(x_i-y_i)	67.5	for S5			
sqr(x_i-y_i)	18.5	for S6			
sqr(x_i-y_i)	79.5	for S7			
avg(1,56,7)	47.5	for 1,5,6,7 of C1 from C2			
sqrt (avg(1,56,7))	**6.892024**	for 1,5,6,7 of C1 from C2 Thus variance between C1 and C2 is 6.89			
x_i-y_i for S1 of C1	3.5	5.75	0.5	16.5	For Cluster C3
x_i-y_i for S5 of C1	4.5	6.75	2.5	19.5	
x_i-y_i for S6 of C1	2.5	5.75	3.5	15.5	
x_i-y_i for S7 of C1	4.5	2.75	2.5	20.5	
sqr(x_i-y_i)	317.8125	for S1 from C3			
sqr(x_i-y_i)	452.3125	for S5 from C3			
sqr(x_i-y_i)	291.8125	for S6 from C3			
sqr(x_i-y_i)	454.3125	for S7 from C3			
avg(1,56,7)	379.0625	for 1,5,6,7 of C1 from C3			
sqrt(avg(1,56,7))	**19.4695**	for 1,5,6,7 of C1 from C3 Thus variance between C1 and C3 is 19.4695			
x_i-y_i for S2 of C2	-2.25	-1.5	-0.75	-4.5	For Cluster C1
x_i-y_i for S8 of C2	-0.25	-0.5	-0.75	-7.5	
sqr(x_i-y_i)	28.125	for S2 of C2 from C1			
sqr(x_i-y_i)	57.125	for S8 of C2 from C1			
avg(2,8)	42.625	for 2,8 of C2 from C1			
sqrt(avg(2,8))	**6.528782**	for 2,8 of C2 from C1 Thus variance between C2 and C1 is 6.52			
x_i-y_i for S2 of C2	-1	-0.5	0	1.5	For Cluster C2
x_i-y_i for S8 of C2	1	0.5	0	-1.5	
sqr(x_i-y_i)	3.5	for S2 of C2 from C2			
sqr(x_i-y_i)	3.5	for S8 of C2 from C2			
avg(2,8)	3.5	for 2,8 of C2 from C2			

Contd.

Calculation	*Quiz1*	*MSE*	*Quiz2*	*ESE*	*Description*
sqrt(avg(2,8))	**1.870829**	for 2,8 of C2 from C2 Thus within cluster variance of C2, i.e. from C2 to C2 is 1.87			
x_i-y_i for S2 of C2	1.5	3.75	1.5	13.5	For Cluster C3
x_i-y_i for S8 of C2	3.5	4.75	1.5	10.5	
sqr(x_i-y_i)	200.8125	for S2 of C2 from C3			
sqr(x_i-y_i)	147.3125	for S8 of C2 from C3			
avg(2,8)	174.0625	for 2,8 of C2 from C3			
sqrt(avg(2,8))	**13.19327**	for 2,8 of C2 from C3 Thus variance between C2 and C3 is 13.19			
x_i-y_i for S3 of C3	-3.25	-4.5	-1.75	-11.5	For Cluster C1
x_i-y_i for S4 of C3	-4.25	-6.5	-2.75	-21.5	
x_i-y_i for S9 of C3	-5.25	-5.5	-3.75	-24.5	
x_i-y_i for S10 of C3	-2.25	-4.5	-0.75	-14.5	
sqr(x_i-y_i)	166.125	for S3 of C3 from C1			
sqr(x_i-y_i)	530.125	for S4 of C3 from C1			
sqr(x_i-y_i)	672.125	for S9 of C3 from C1			
sqr(x_i-y_i)	236.125	for S10 of C3 from C1			
avg(3,4,9,10)	401.125	for 3,4,9,10 of C3 from C1			
sqrt(avg(3,4,9,10))	**20.02811**	for 3,4,9,10 of C3 from C1 Thus variance between C3 and C1 is 20.02			
x_i-y_i for S3 of C3	-2	-3.5	-1	-5.5	For Cluster C2
x_i-y_i for S4 of C3	-3	-5.5	-2	-15.5	
x_i-y_i for S9 of C3	-4	-4.5	-3	-18.5	
x_i-y_i for S10 of C3	-1	-3.5	0	-8.5	
sqr(x_i-y_i)	47.5	for S3 of C3 from C2			
sqr(x_i-y_i)	283.5	for S4 of C3 from C2			
sqr(x_i-y_i)	387.5	for S9 of C3 from C2			
sqr(x_i-y_i)	85.5	for S10 of C3 from C2			
avg(3,4,9,10)	201	for 3,4,9,10 of C3 from C2			

Contd.

Calculation	*Quiz1*	*MSE*	*Quiz2*	*ESE*	*Description*
sqrt(avg(3,4,9,10))	**14.17745**	for 3,4,9,10 of C3 from C2 Thus variance between C3 and C2 is 14.17			
x_i-y_i for S3 of C3	0.5	0.75	0.5	6.5	For Cluster C3
x_i-y_i for S4 of C3	-0.5	-1.25	-0.5	-3.5	
x_i-y_i for S9 of C3	-1.5	-0.25	-1.5	-6.5	
x_i-y_i for S10 of C3	1.5	0.75	1.5	3.5	
sqr(x_i-y_i)	43.3125	for S3 of C3 from C3			
sqr(x_i-y_i)	14.3125	for S4 of C3 from C3			
sqr(x_i-y_i)	46.8125	for S9 of C3 from C3			
sqr(x_i-y_i)	17.3125	for S10 of C3 from C3			
avg(3,4,9,10)	30.4375	for 3,4,9,10 of C3 from C3			
sqrt(avg(3,4,9,10))	**5.517019**	for 3,4,9,10 of C3 from C3 Thus within cluster variance of C3, i.e. from C3 to C3 is 5.51			

7.6.2 Starting values for the k-means algorithm

Normally we have to specify the number of starting seeds and clusters at the start of the k-means algorithm. We can use an iterative approach to overcome this problem. For example, first select three starting seeds randomly and choose three clusters. Once the final clusters have been found, the process may be repeated several times with a different set of seeds. We should select the seed records that are at maximum distance from each other or are as far as possible. If two clusters are identified to be close together during the iterative process, it is appropriate to merge them. Also, a large cluster may be partitioned into two smaller clusters if the intra-cluster variance is above some threshold value.

7.6.3 Issues with the k-means algorithm

There are a number of issues with the k-Means algorithm that should be understood. Some of these are:

- The results of the k-means algorithm intensively depends on the initial guesses of the seed records.
- The k-means algorithm is sensitive to outliers. Thus, it may give poor results if an outlier is selected as a starting seed record.
- The k-means algorithm works on the basis of Euclidean distance and the mean(s) of the attribute values of intra-cluster objects. Thus, it is only suitable for continuous data as it is restricted to data for which there is the notion of a center (centroid).
- The k-means algorithm does not take into account the size of the clusters. Clusters may be either large or small.

- It does not deal with overlapping clusters.
- It does not work well with clusters of different sizes and density.

7.6.4 Scaling and weighting

Sometimes during clustering, the value of one attribute may be small as compared to other attributes. In such cases, that attribute will not have much influence on determining which cluster the object belongs to, as the values of this attribute are smaller than other attributes and hence, the variation within the attribute will also be small.

Consider the dataset given in Table 7.20 for the chemical composition of wine samples. Note that the values for different attributes cover a range of six orders of magnitude. It is also important to notice that for wine density, values vary by only 5% of the average value. It turns out that clustering algorithms struggle with numeric attributes that exhibit such ranges of values.

Table 7.20 Chemical composition of wine samples

fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	[illegible]	[illegible]	[illegible]	[illegible]
4.6	0.520	0.15	2.1	0.054	8	65	0.99340	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
4.7	0.600	0.17	2.3	0.058	17	106	0.99320	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
4.9	0.420	0.00	2.1	0.048	16	42	0.99154	3.71	0.74	14.0	R	7
5.0	0.380	0.01	1.6	0.048	26	60	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
5.0	0.400	0.50	4.3	0.046	29	80	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
5.0	0.420	0.24	2.0	0.060	19	50	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
5.0	0.740	0.00	1.2	0.041	16	46	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
5.0	1.020	0.04	1.4	0.045	41	85	0.99380	3.75	0.48	10.5	R	4
5.0	1.040	0.24	1.6	0.050	32	96	0.99340	3.74	0.62	11.5	R	5
5.1	0.420	0.00	1.8	0.044	18	88	0.99157	3.68	0.73	13.6	R	7
5.1	0.470	0.02	1.3	0.034	18	44	0.99210	3.90	0.62	12.8	R	6
5.1	0.510	0.18	2.1	0.042	16	101	0.99240	3.46	0.87	12.9	R	7

WineQuality.arff file available in WineQuality.zip at www.technologyforge.net/Datasets

10^6 range of magnitudes

Max – Min = 5% of average

[*Source*: Witten et. al.]

All attributes should be transformed to a similar scale for clustering to be effective unless we want to give more weight to some attributes that are comparatively large in scale. Commonly, we use two techniques to convert the attributes. These are Normalization and Standardization.

Normalization

In case of normalization, all the attributes are converted to a normalized score or to a range (0, 1). The problem of normalization is an outlier. If there is an outlier, it will tend to crunch all of the other values down toward the value of zero. In order to understand this case, let's suppose the range of students' marks is 35 to 45 out of 100. Then 35 will be considered as 0 and 45 as 1, and students will be distributed between 0 to 1 depending upon their marks. But if there is one student having marks 90, then it will act as an outlier and in this case, 35 will be considered as 0 and 90 as 1. Now, it will crunch most of the values down toward the value of zero. In this scenario, the solution is standardization.

Standardization

In case of standardization, the values are all spread out so that we have a standard deviation of 1.

Generally, there is no rule for when to use normalization versus standardization. However, if your data does have outliers, use standardization otherwise use normalization. Using standardization tends to make the remaining values for all of the other attributes fall into similar ranges since all attributes will have the same standard deviation of 1.

In next section, another important clustering technique, i.e., Hierarchical Clustering has been discussed.

7.7 Hierarchical Clustering Algorithms (HCA)

Hierarchical clustering is a type of cluster analysis which seeks to generate a hierarchy of clusters. It is also called Hierarchical Cluster Analysis (HCA). Hierarchical clustering methods build a nested series of clusters in comparison to partitioned methods that generate only a flat set of clusters.

There are two types of Hierarchical clustering: agglomerative and divisive.

Agglomerative

This is a 'bottom-up' approach. In this approach, each object is a cluster by itself at the start and its nearby clusters are repetitively combined resulting in larger and larger clusters until some stopping criterion is met. The stopping criterion may be the specified number of clusters or a stage at which all the objects are combined into a single large cluster that is the highest level of hierarchy as shown in Figure 7.10.

Divisive

This is a 'top-down' approach. In this approach, all objects start from one cluster, and partitions are performed repeatedly resulting in smaller and smaller clusters until some stopping criterion is met or each cluster consists of the only object in it as shown in Figure 7.10. Generally, the mergers and partitions are decided in a greedy manner.

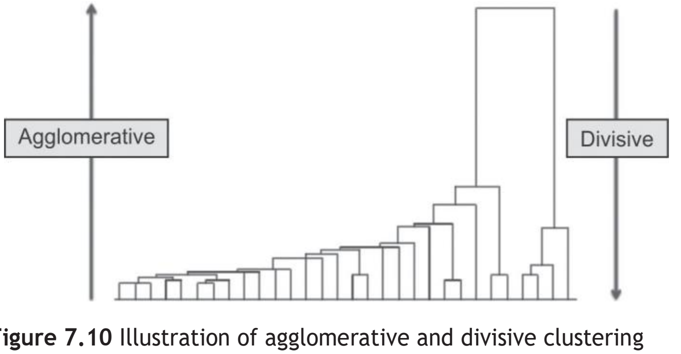


Figure 7.10 Illustration of agglomerative and divisive clustering

[*Credits*: http://chem-eng.utoronto.ca/~datamining/dmc/clustering_hierarchical.htm]

Let's discuss agglomerative and divisive clustering in detail.

7.7.1 Agglomerative clustering

In agglomerative clustering, suppose a set of *N* items are given to be clustered, the procedure of clustering is given as follows:

1. First of all, each item is allocated to a cluster. For example, if there are *N* items then, there will be *N* clusters, i.e., each cluster consisting of just one item. Assume that the distances (similarities) between the clusters are same as the distances (similarities) between the items they contain.
2. Identify the nearest (most similar) pair of clusters and merge that pair into a single cluster, so that now we will have one cluster less.
3. Now, the distances (similarities) between each of the old clusters and the new cluster are calculated.
4. Repeat steps 2 and 3 until all items are merged into a single cluster of size *N*.

As this type of hierarchical clustering merges clusters recursively, it is therefore known as agglomerative.

Once the complete hierarchical tree is obtained, then there is no point of grouping *N* items in a single cluster. You just have to cut the *k-1* longest links to get *k* clusters.

Let's understand the working of agglomerative clustering by considering the following raw data given in Figure 7.11.

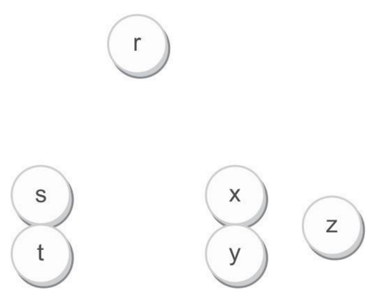


Figure 7.11 Raw data for agglomerative clustering

This method generates the hierarchy by recursively combining clusters from the individual items. For example, there are six items {r}, {s}, {t}, {x}, {y} and {z}. The first step is to identify the items to be combined into a cluster. Normally, the two closest elements are taken on the basis of the chosen distance.

When the tree is cut at a given height, it gives a partitioning clustering at a selected precision. In this case, cutting the tree after the second row produces clusters {r} {s t} {x y} and {z} as shown in Figure 7.12. Cutting the tree after the third row will produce clusters {r} {s t} and {x y z}, which is called as coarser clustering, with a smaller number of clusters, but which are larger in size. The complete hierarchical clustering tree diagram is as given in Figure 7.12.

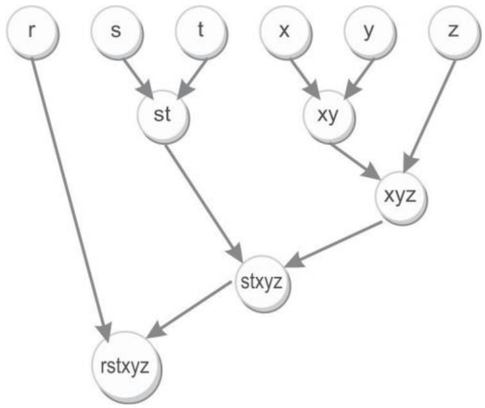


Figure 7.12 Complete hierarchical clustering tree diagram

7.7.1.1 *Role of linkage metrics*

It is required to compute the distance between clusters by using some metrics before any clustering is performed. These metrics are called linkage metrics.

It will be beneficial to identify the proximity matrix consisting of the distance between each point using a distance function.

The important linkage metrics to measure the distance between each cluster are given below:

Single linkage

In single linkage hierarchical clustering, the shortest distance between two points in each cluster is considered as the distance between two clusters. For example, the distance between two clusters 'r' and 's' to the left is equal to the length of the arrow between their two closest points as shown in Figure 7.13 and the formula of calculation of distance using single-linkage hierarchical clustering is given by:

$$\boxed{L(r,s) = \min(D(x_{ri}, x_{sj}))}$$

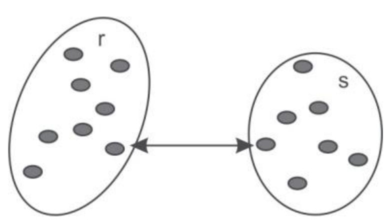


Figure 7.13 Single linkage

[*Credits*: http://chem-eng.utoronto.ca/~datamining/dmc/clustering_hierarchical.htm]

Complete linkage

In complete linkage hierarchical clustering, the longest distance between two points in each cluster is considered as the distance between two clusters. For example, the distance between clusters 'r' and 's' to the left is equal to the length of the arrow between their two farthest points as shown in Figure 7.14 and the formula of calculation of distance using complete linkage hierarchical clustering is given by:

$$L(r,s) = \max(D(x_{ri}, x_{sj}))$$

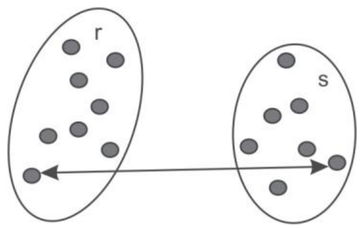


Figure 7.14 Complete linkage

[*Credits*: http://chem-eng.utoronto.ca/~datamining/dmc/clustering_hierarchical.htm]

Average linkage

In average linkage hierarchical clustering, the average distance between each point in one cluster to every point in the other cluster is considered as the distance between two clusters. For example, the distance between clusters 's' and 'r' to the left is equal to the average length of each arrow connecting the points of one cluster to the other as shown in Figure 7.15 and the formula of calculation of distance using average linkage hierarchical clustering is given by:

$$L(r,s) = \frac{1}{n_r n_s} \sum_{i=1}^{n_Y} \sum_{j=1}^{n_S} D\left(x_{ri}, x_{sj}\right)$$

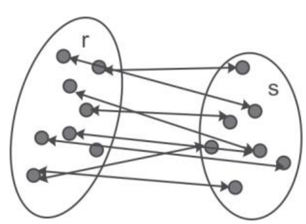


Figure 7.15 Average linkage

[*Credits*: http://chem-eng.utoronto.ca/~datamining/dmc/clustering_hierarchical.htm]

Example of Single-linkage hierarchical clustering

Let's perform a hierarchical clustering of some Indian cities on the basis of distance in kilometers between them. In this case, single-linkage clustering is used to calculate the distance between clusters.

The distance matrix (with a sequence number, m: 0) shown in Table 7.21 represents the distance in kilometers among some of the Indian cities. Suppose initially the level (L) is 0 for all the clusters.

Table 7.21 Input distance matrix (L = 0 for all the clusters)

	Delhi	*Jammu*	*Srinagar*	*Patiala*	*Amritsar*	*Pahalgam*
Delhi	0	575	853	267	452	842
Jammu	575	0	294	359	203	285
Srinagar	853	294	0	627	469	98
Patiala	267	359	627	0	235	610
Amritsar	452	203	469	235	0	456
Pahalgam	842	285	98	610	456	0

The nearest pair of cities is Srinagar and Pahalgam as the distance between them is 98. These cities are combined into a single cluster called 'Srinagar/Pahalgam'. Then, the distance is calculated from this new compound object to all other objects. In case of single link clustering, the shortest distance from any object of the cluster to the outside object is considered as the distance from the compound object to another object. Therefore, distance from 'Srinagar/Pahalgam' to Delhi is picked to be 842, i.e., min(853,842), which is the distance from Pahalgam to Delhi.

Now, the level of the new cluster is L(Srinagar/Pahalgam) = 98 and **m: 1 is the new sequence number**. By following the same method, new distance from the compound object is calculated as shown in Table 7.22 and the description of calculation is given as follows.

*min(853,842) **min(294,285) ***min(627,610)

Table 7.22 Input distance matrix, with m: 1

	Delhi	*Jammu*	*Srinagar/ Pahalgam*	*Patiala*	*Amritsar*
Delhi	0	575	842	267	452
Jammu	575	0	285	359	203
Srinagar/Pahalgam	842*	285**	0	610***	456
Patiala	267	359	610	0	235
Amritsar	452	203	456	235	0

At the level of m:1, the minimum distance (min(Jammu, Amritsar)) in the distance matrix is 203, i.e., the distance between Jammu and Amritsar so, Jammu and Amritsar will be merged into a new cluster called Jammu/Amritsar with new L(Jammu/Amritsar**)** = 203 and m = 2 as shown in

Table 7.23. The description of the calculation of the new distance from the compound object to other objects is given as follows.

*min(575,452) **min(285,456) ***min(359,235)

Table 7.23 Input distance matrix, with m: 2

	Delhi	*Jammu/Amritsar*	*Srinagar/ Pahalgam*	*Patiala*
Delhi	0	452	842	267
Jammu/Amritsar	452*	0	285**	235**
Srinagar/Pahalgam	842	285	0	610
Patiala	267	235	610	0

At the level of m:2, the minimum distance (min (Jammu/Amritsar, Patiala)) in the distance matrix is 235, i.e., distance between Jammu/Amritsar and Patiala so, Jammu/Amritsar will be merged with Patiala into a new cluster called Jammu/Amritsar/Patiala with L(Jammu/Amritsar/Patiala) = 235 and m = 3 as shown in Table 7.24. The description of the calculation of the new distance from the compound object to other objects is given as follows.

*min(452,267) **min(285,610)

Table 7.24 Input distance matrix, with m: 3

	Delhi	*Jammu/Amritsar/Patiala*	*Srinagar/Pahalgam*
Delhi	0	267	842
Jammu/Amritsar/Patiala	267*	0	285**
Srinagar/Pahalgam	842	285	0

At the level of m:3, the minimum distance (min(Jammu/Amritsar/Patiala, Delhi)) in the distance matrix is 267, i.e., the distance between Jammu/Amritsar/Patiala and Delhi so, Jammu/Amritsar/Patiala will be merged with Delhi into a new cluster called Delhi/Jammu/Amritsar/Patiala with L(Delhi/Jammu/Amritsar/Patiala) = 267 and m = 4 as shown in Table 7.25. The description of the calculation of the new distance from the compound object to other objects is given as follows.

*min(285,842)

Table 7.25 Input distance matrix, with m: 4

	Delhi/ Jammu/Amritsar/Patiala	*Srinagar/Pahalgam*
Delhi/Jammu/Amritsar/Patiala	0	285*
Srinagar/Pahalgam	285	0

Finally, the last two clusters are merged at level 285. The whole procedure is outlined by the hierarchical tree as shown in Figure 7.16.

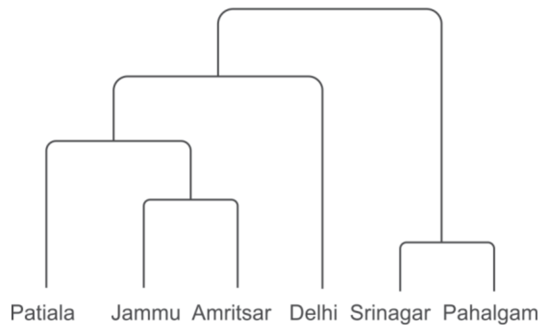


Figure 7.16 Hierarchical tree of clustering of Indian cities on the basis of distance

Example of clustering of students record with the agglomerative algorithm

Let us apply the Agglomerative algorithm on the database containing results of students' examination in a given course (earlier discussed for the k-mean algorithm). The students' performance on the basis of marks obtained out of 100 marks distributed across Quiz1, MSE, Quiz2, and ESE is given again in Table 7.26.

Table 7.26 Record of students' performance

Instances	*Quiz 1*	*MSE*	*Quiz2*	*ESE*
S1	8	20	6	45
S2	6	18	7	42
S3	5	15	6	35
S4	4	13	5	25
S5	9	21	8	48
S6	7	20	9	44
S7	9	17	8	49
S8	8	19	7	39
S9	3	14	4	22
S10	6	15	7	32

Step 1:

In this step, the distance matrix is computed from the above data using the centroid method between every pair of instances that are to be clustered. Due to the symmetric nature of the distance matrix (because the distance between p and q is same as the distance between q and p), only the lower triangular matrix is shown in Table 7.27 as the upper triangle can be filled in by reflection. The distance matrix consists of zeroes on the diagonal because every instance is at zero distance from itself.

The description of some of the calculations for cells of the second and third row of the distance matrix given in Table 7.27 drawn from Table 7.26 is given as follows.

*__Second row S2:__ [S2-S1] = [|6-8|+|18-20|+|7-6|+|42-45|] = 8
**__Third row S3:__ [S3-S1] = [|5-8|+|15-20|+|6-6|+|35-45|] = 18
***__Third row S3:__ [S3-S2] = [|5-6|+|15-18|+|6-7|+|35-42|] = 12

Table 7.27 Distance matrix at m: 0

Instances	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
S1	0									
S2	8*	0								
S3	18**	12***	0							
S4	32	26	14	0						
S5	7	13	25	39	0					
S6	5	7	19	33	8	0				
S7	10	12	22	36	5	11	0			
S8	8	6	12	26	13	9	14	0		
S9	36	30	18	6	43	37	40	30	0	
S10	21	13	5	13	26	20	23	13	17	0

Step 2:

Apply the agglomerative method on this distance matrix and calculate the minimum distance between clusters and combine them. The minimum distance in the distance matrix is 5. Let's randomly select the minimum distance, i.e., 5 between S7 and S5.

Thus, the instances S7 and S5 are merged into a cluster called C1 and the distance is calculated from this new compound object to all other objects by following the single link hierarchical clustering approach for calculation. To attain a new distance matrix, the instances S5 and S7 need to be removed and are replaced by cluster C1. Since single link hierarchical clustering is used, the distance between cluster C1 and every other instance is the shortest of the distance between an instance and S5; and an instance and S7. The cells involved in merging record number S5 and S7 for the creation of a new cluster are highlighted in Table 7.28.

Table 7.28 Cells involved in C1

Instances	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
S1	0									
S2	8*	0								
S3	18**	12***	0							
S4	32	26	14	0						
S5	7	13	25	39	0					
S6	5	7	19	33	8	0				
S7	10	12	22	36	5	11	0			
S8	8	6	12	26	13	~~9~~	14	0		
S9	36	30	18	6	43	~~37~~	40	30	0	
s10	21	13	5	13	26	~~20~~	23	13	17	0

The description of the some of the calculations to create a new distance matrix with m: 2 is given as follows.

For example, to create a new distance matrix after merging S5 and S7, calculations will be as:

d (S1, S5) = 7 and d (S1, S7) = 10, so d (S1, C1) = min (7, 10) = 7*

Similarly, d (S2, S5) = 13 and d (S2, S7) = 12, so d (S2, C1) = min (13, 12) = 12**

The distance between S6 and C1 will be (minimum of distance between S6-S5 and S7-S6, i.e. min (8, 11), i.e., 8***). By using the highlighted cells, the data in C1 is filled as shown in Table 7.29.

d(S8, S5) = 13 and d(S8, S7) = 14, so d(S8, C1) = min (13, 14) =13****

Similarly, d(S9, S5) = 43 and d(S9, S7) = 40, so d(S9, C1) = min (43, 40) =40*****

The instances with the smallest distance get clustered next.

Table 7.29 Input distance matrix, with m: 2

Instances	S1	S2	S3	S4	C1	S6	S8	S9	S10
S1	0								
S2	8	0							
S3	18	12	0						
S4	32	26	14	0					
C1	7*	12**	22	36	0				
S6	5	7	19	33	8***	0			
S8	8	6	12	26	13****	9	0		
S9	36	30	18	6	40*****	37	30	0	
S10	21	13	5	13	23	20	13	17	0

Step 3: Repeat Step 2 until all clusters are merged.

Combine S3 and S10 clusters into C2 as it has the minimum distance. The cells involved in merging record number 3 and 10 for the creation of C2 have been highlighted in Table 7.30.

Table 7.30 Cells involved in C2

Instances	S1	S2	S3	S4	C1	S6	S8	S9	S10
S1	0								
S2	8	0							
S3	18	12	0						
S4	32	26	14	0					
C1	7	12	22	36	0				
S6	5	7	19	33	8	0			
S8	8	6	12	26	13	9	0		
S9	36	30	18	6	43	37	30	0	
s10	21	13	5	13	23	20	13	17	0

The description of the some of the calculations to create a new distance matrix with m: 3 is given as follows.

For example, to create a new distance matrix after merging S3 and S10, calculations will be as:

$$d(S1, S3) = 18 \text{ and } d(S1, S10) = 21, \text{ so } d(S1, C2) = \min(18, 21) = 18^{*}$$

Similarly, $d(S2, S3) = 12$ and $d(S2, S10) = 13$, so $d(S2, C2) = \min(12, 13) = 12^{**}$

The distance between S4 and C2 will be (minimum of distance between S4-S3 and S10-S4, i.e. min(14,13), i.e., 13^{***}). By using the highlighted cells, the data in C2 is filled as shown in Table 7.30.

$$d(C1, S3) = 22 \text{ and } d(C1, S10) = 23, \text{ so } d(C1, C2) = \min(22, 23) = 22^{****}$$

Similarly, $d(S6, S3) = 19$ and $d(S6, S10) = 20$, so $d(S6, C2) = \min(19, 20) = 19^{*****}$

The input matrix at m: 3 is shown in Table 7.31.

Table 7.31 Input distance matrix, with m: 3

Instances	S1	S2	C2	S4	C1	S6	S8	S9
S1	0							
S2	8	0						
C2	18*	12**	0					
S4	32	26	13***	0				
C1	7	12	22****	36	0			
S6	5	7	19*****	33	8	0		
S8	8	6	12	26	13	9	0	
S9	36	30	17	6	40	37	30	0

At the level of m: 3, the minimum distance in the distance matrix is 5 between S6 and S1, so S6 and S1 will be merged into C3. The cells involved in the merging of S6 and S1 for the creation of C3 have been highlighted in Table 7.32.

Table 7.32 Cells involved in creating C3

Instances	S1	S2	C2	S4	C1	S6	S8	S9
S1	0							
S2	8	0						
C2	18*	12**	0					
S4	32	26	13***	0				
C1	7	12	22****	36	0			
S6	5	7	19	33	8	0		
S8	8	6	12	26	13	9	0	
S9	36	30	17	6	40	37	30	0

The description of calculation of some of new distances from compound object C3 to other objects is given below.

d(S2, S1) = 8 and d(S2, S6) = 7 so, d(S2, C3) = min (8, 7) = 7*

d (C2, S1) = 18 and d(C2, S6) = 19 so, d(C2, C3) = min (18, 19) =18**

Similarly, d(S8, S1) = 8 and d (S8, S6) = 9 so, min (S8, C3) = min (8, 9) = 8***
The input matrix at m: 4 is shown in Table 7.33.

Table 7.33 Input distance matrix, with m: 4

Instances	C3	S2	C2	S4	C1	S8	S9
C3	0						
S2	7*	0					
C2	18**	12	0				
S4	32	26	13	0			
C1	7	12	22	36	0		
S8	8***	6	12	26	13	0	
S9	36	30	17	6	40	30	0

At the level of m: 4, the minimum distance in the distance matrix is 6, so S9 and S4 will be merged into C4, and the cells involved in C4 are highlighted in Table 7.34.

Table 7.34 Cells involved in creating C4

Instances	C3	S2	C2	S4	C1	S8	S9
C3	0						
S2	7*	0					
C2	18	12	0				
S4	32	26	13	0			
C1	7	12	22	36	0		
S8	8	6	12	26	13	0	
S9	36***	30	17	6	40	30	0

The calculation of some of new distances from the compound object C4 to other objects is given below.

d(C3, S4) = 32 and d(C3, S9) = 36 so, d(C3, C4) = min(32, 36) = 32*

d (C2, S4) = 13 and d(C2, S9) = 17 so, d(C2, C4) = min(13, 17) =13**

Similarly, d(C1, S4) = 36 and d (C1, S9) = 40 so, min(C1, C4) = min(36, 40) = 36***

The input matrix at m: 5 is shown in Table 7.35.

Table 7.35 Input distance matrix, with m: 5

Instances	C3	S2	C2	C4	C1	S8
C3	0					
S2	7	0				
C2	18	12	0			
C4	32*	26	13**	0		
C1	7	12	22	36***	0	
S8	8	6	12	26	13	0

At the level of m: 5, the minimum distance in the distance matrix is 6, so S8 and S2 will be merged into C5, and the cells involved in C5 are highlighted in Table 7.36.

Table 7.36 Cells involved in creating C5

Instances	C3	S2	C2	C4	C1	S8
C3	0					
S2	7	0				
C2	18	12	0			
C4	32*	26	13**	0		
C1	7	12	22	36***	0	
S8	8	6	12	26	13	0

The description of calculation of new distances from compound object C5 to other objects is given below. The input matrix at m:6 is shown in Table 7.37.

*min(12,12) **min(26,26) ***min(12,13)

Table 7.37 Input distance matrix, with m: 6

Instances	C3	C5	C2	C4	C1
C3	0				
C5	7	0			
C2	18	12*	0		
C4	32	26**	13	0	
C1	7	12***	22	36	0

At the level of m: 6, the minimum distance in the distance matrix is 7, so C1 and C3 will be merged into C6, and the cells involved in C6 are highlighted in Table 7.38.

Table 7.38 Cells involved in creating C6

Instances	C3	C5	C2	C4	C1
C3	0				
C5	7	0			
C2	18	12*	0		
C4	32	26**	13	0	
C1	7	12***	22	36	0

The calculation of some of new distances from compound object C6 to other objects is given below. The input matrix at m: 7 is shown in Table 7.39.

*min(12,7) **min(22,18) ***min(36,32)

Table 7.39 Input distance matrix, with m: 7

Instances	C6	C5	C2	C4
C6	0			
C5	**7***	0		
C2	18**	12	0	
C4	32***	26	13	0

At the level of m: 7, the minimum distance in the distance matrix is 7, so C5 and C6 will be merged into C7, and the cells involved in C7 are highlighted in Table 7.40.

Table 7.34 Cells involved in creating C4

Instances	C6	C5	C2	C4
C6	0			
C5	**7***	0		
C2	18**	12	0	
C4	32***	26	13	0

The calculation of some of new distances from compound object C7 is given below. The input matrix at m: 8 is shown in Table 7.41.

*min(12, 18) **min(26,32)

Table 7.41 Input distance matrix, with m: 8

Instances	C7	C2	C4
C7	0		
C2	**12***	0	
C4	26**	13	0

At the level of m:8, the minimum distance in the distance matrix is 12, so C2 and C7 will be merged into C8, and the cells involved in C8 are highlighted in Table 7.42.

Table 7.42 Cells involved in creating C8

Instances	C7	C2	C4
C7	0		
C2	12*	0	
C4	26**	13	0

The input matrix at m: 9 is shown in Table 7.43.

Table 7.43 Input distance matrix, with m: 9

Instances	C8	C4
C8	0	
C4	13	0

Finally, the last two clusters are merged at level 8. The whole procedure is outlined by the hierarchical tree as shown in Figure 7.17. Thus, all clusters are eventually combined.

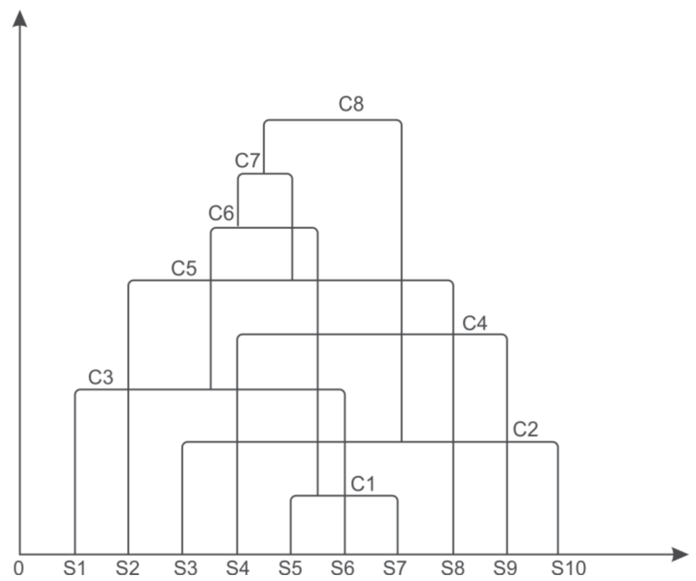


Figure 7.17 Hierarchical tree of clustering of students on the basis of their marks

7.7.1.2 *Weakness of agglomerative clustering methods*

The major weaknesses of agglomerative clustering methods are given as follows:

- Agglomerative clustering methods do not scale well as they have a high time complexity of at least $O(n^2)$ as the algorithm will have to compute $n*n$ distances, where n is the number of total objects.
- There is no undo mechanism to undo what was done previously.

7.7.2 Divisive clustering

As discussed earlier, in divisive clustering, all the objects or observations are assigned to a single cluster and then that single cluster is partitioned into two least similar clusters. This approach is repeatedly followed on each cluster until there is only one cluster for each object or observation.

Let us now apply a divisive algorithm on the same database discussed earlier for k-means and agglomerative clustering that contains the results of student examination in a given course. The students' performance on the basis of marks obtained out of 100 marks distributed across Quiz1, MSE, Quiz2, and ESE is given again in Table 7.44.

Table 7.44 Record of students' performance

Roll no	*Quiz1*	*MSE*	*Quiz2*	*ESE*
1	8	20	6	45
2	6	18	7	42

Contd.

Roll no	Quiz1	MSE	Quiz2	ESE
3	5	15	6	35
4	4	13	5	25
5	9	21	8	48
6	7	20	9	44
7	9	17	8	49
8	8	19	7	39
9	3	14	4	22
10	6	15	7	32

Step 1:

The distance matrix is calculated in the same way as we did in agglomerative clustering by using the centroid method to calculate the distance between the clusters. For representation, Roll numbers have been converted to Instances S1 to S10. Due to the symmetric nature of the distance matrix, we need to represent only the lower triangular matrix as shown in Table 7.45.

Table 7.45 Distance matrix at m: 0

Instances	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
S1	0									
S2	8	0								
S3	18	12	0							
S4	32	26	14	0						
S5	7	13	25	39	0					
S6	5	7	19	33	8	0				
S7	10	12	22	36	5	11	0			
S8	8	6	12	26	13	9	14	0		
S9	36	30	18	6	43	37	40	30	0	
S10	21	13	5	13	26	20	23	13	17	0

Step 2:

Pick up the two objects with the largest distance, i.e., 43 in this case between S5 and S9. Now, these two objects become the seed records of two new clusters. Split the entire group into two clusters based on the distances in the distance matrix as shown in Figure 7.18.

The details about some of the calculations of the distance matrix (referring to Table 7.45) for cluster C1 have been shown in Figure 7.18 (b) and similarly, the distances, for instance S9, i.e., C2 from other instances are computed.

(a)

Instances	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
S1	0									
S2	8	0								
S3	18	12	0							
S4	32	26	14	0						
S5	**7**	**13**	**25**	**39**	**0**					
S6	5	7	19	33	**8**	0				
S7	10	12	22	36	**5**	11	0			
S8	8	6	12	26	**13**	9	14	0		
S9	36	30	18	6	**43**	37	40	30	0	
S10	21	13	5	13	**26**	20	23	13	17	0

(a)

	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
S5, i.e., C1	7*	13	25**	39	0	8	5	13	43	26
S9, i.e., C2	36	30	18	6	43	37	40	30	0	17
Assigned Cluster	C1	C1	C2	C2	C1	C1	C1	C1	C2	C2

Figure 7.18 (a) Distance matrix at m: 0 (b) Data objects split after two clusters

In Figure 7.18 (b), the instances with minimum distance from cluster C1 and C2 are assigned to that cluster, For example,

*d(S1, C1) = 7 and d(S1, C2) = 36, so, min(7, 36) = 7, i.e., C1

**Similarly, d(S3, C1) = 25 and d(S3, C2) = 18, so, min(25, 18) = 18, i.e., C2

From the figure, it has been observed that

Cluster C1 includes – S1, S2, S5, S6, S7, S8, and

Cluster C2 includes – S3, S4, S9, S10

Since the stopping criterion hasn't been met, we will split up the largest cluster, i.e., C1. For this distance matrix will be computed for C1 as given in Table 7.46.

Table 7.46 Distance matrix for cluster C1

	S1	S2	S5	S6	S7	S8
S1	0					
S2	8	0				
S5	7	13	0			
S6	5	7	8	0		
S7	10	12	5	11	0	
S8	8	6	13	9	14	0

Since, the largest distance is 14, so C1 is split with S7 and S8 as seeds. They become the seeds of two new clusters. Splitting the entire group into these two clusters based on the distances in the distance matrix is shown in Table 7.47.

Table 7.47 Splitting of cluster C1 into two new clusters of S7 and S8

	S1	S2	S5	S6	S7	S8
S7,i.e, C3	10	12	5	11	0	14
S8, i.e., C4	8	6	13	9	14	0
Assigned Cluster	C4	C4	C3	C4	C3	C4

Here, Cluster C3 includes S5, S7 and cluster C4 includes S1, S2, S6, S8.

We already have another cluster as C2 (as described in Figure 7.18 (b)) which includes S3, S4, S9, S10. We can divide the cluster C3 further into two elementary clusters.

Now, the largest cluster, in this case is either C2 or C4 (both have the same number of objects). Here, we are considering cluster C2 as a candidate for a further split. For this the distance matrix is computed for cluster C2 as given in Table 7.48.

Table 7.48 Distance matrix for cluster C2

	S3	S4	S9	S10
S3	0			
S4	14	0		
S9	18	6	0	
S10	5	13	17	0

The largest distance is 18, so C2 has S3 and S9 as seeds for a further split. The result of splitting the entire group into these two clusters based on the distance in the distance matrix is given in Table 7.49.

Table 7.49 Splitting of cluster C2 into two new clusters of S3 and S9

	S3	S4	S9	S10
S3, i.e., C5	0	14	18	5
S9, i.e., C6	18	6	0	17
Assigned Cluster	C5	C6	C6	C5

Here, cluster C5 includes S3, S10 and cluster C6 includes S4, S9.

Both clusters C5 and C6 can be divided into two more elementary clusters. We already have other clusters namely cluster C3 that includes S5, S7 and cluster C4 that includes S1, S2, S6, S8. Now, the largest cluster is C4 thus, C4 is the candidate for the further split. For this the distance matrix is computed for C4 as given in Table 7.50.

Table 7.50 Distance matrix for cluster C4

	S1	S2	S6	S8
S1	0			
S2	8	0		
S6	5	7	0	
S8	8	6	9	0

The largest distance is 9, so C4 has S6 and S8 as seeds for a further split. The result of splitting the entire group into these two clusters based on the distance in the distance matrix is given in Table 7.51.

Table 7.51 Splitting of cluster C4 into two new clusters of S6 and S8

	S1	S2	S6	S8
S6, i.e., C7	5	7	0	9
S8, i.e., C8	8	6	9	0
Assigned Cluster	C7	C8	C7	C8

Here, Cluster C7 includes S1, S6 and cluster C8 includes S2, S8.

We already have other clusters as cluster C5 includes S3, S10 and cluster C6 includes S4, S9. At this stage each cluster can be divided into two clusters each of one object. Thus, the stopping criterion has been reached.

7.7.3 Density-based clustering

Density-based clustering algorithms perform the clustering of data by forming the cluster of nodes on the basis of the estimated density distribution of corresponding nodes in the region. In density-based clustering, clusters are dense regions in the data space. These clusters are separated by regions of lower object density. A cluster is defined as a maximal set of density-connected points.

In 1996, Martin Ester, Hans-Peter Kriegel, Jörg Sander and Xiaowei Xu proposed a Density-based clustering algorithm. The major strength of this approach is that it can identify clusters of arbitrary shapes. A Density-based clustering method is known as DBSCAN (Density-Based Spatial Clustering of Applications with Noise).

Familiarity with the following terms is a must in order to understand the workings of the DBSCAN algorithm.

Neighborhood (ε)

Neighborhood is an important term used in DBSCAN. It represents objects within a certain radius of from a centroid type object. The high-density neighborhood results if an object contains at least MinPts (minimum points) of objects in its neighborhood.

This concept is illustrated in Figure 7.19.

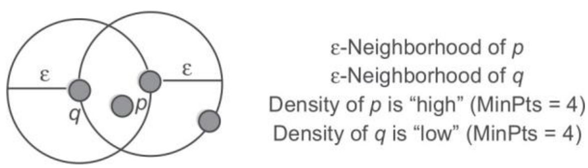


Figure 7.19 Concept of neighborhood and MinPts

Core, Border, and Outlier

A point is known as a core point if it has more than a specified number of points (MinPts) within neighborhood (ε). These points must lie at the interior of a cluster. A border point is a point if it has fewer than MinPts within neighborhood (ε), but it is in the neighborhood of a core point. A point is a noise or outlier point if it is neither a core point nor a border point. The concept of Core, Border, and Outlier is illustrated by considering Minpts = 4 in Figure 7.20.

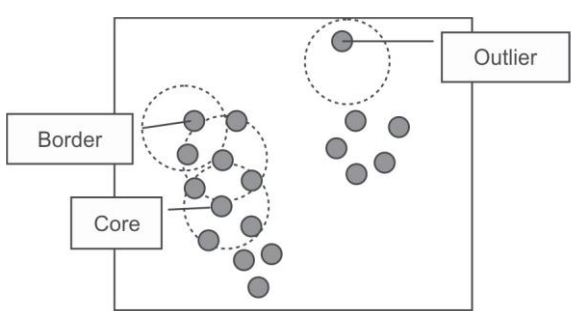


Figure 7.20 Concept of core, border and outlier

Density reachability; Directly and Indirectly density-reachable

DBSCAN's definition of a cluster is defined based on the concept of density reachability. Generally, a point *q* is directly density-reachable from a point *p* if it is not more distant than a given distance ε (i.e., is part of its ε-neighborhood). However, one may consider that *p* and *q* belong to the same cluster if point *p* is surrounded by necessarily many points.

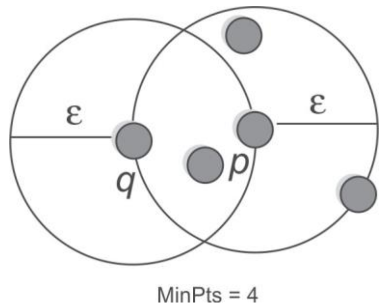


Figure 7.21 Concept of density reachability

As shown in Figure 7.21, if *p* is a core point and point *q* is in neighborhood ε of *p* then we can say that point *q* is directly density-reachable from point *p*.

Following are some important observations that can be drawn from data points given in Figure 7.21.

- Point *q* is directly density-reachable from point 'core point' *p*.
- Point *q* is not a core point as it has only 3 nodes in the neighborhood while MinPts is 4, therefore, *p* is not directly density-reachable from point *q*.
- Thus, Density-reachability is asymmetric

To further understand the concept of directly and indirectly density-reachable, let us consider Figure 7.22. Here, MinPts are 7.

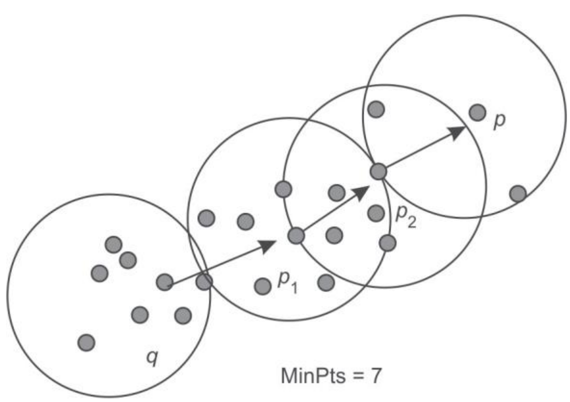


Figure 7.22 Concept of directly and indirectly density-reachable

As shown in Figure 7.22, following are some important observations:

- Point p_2 is a core point as it has more than 7 points within its neighborhood and point p is directly density-reachable from it.
- Point p_1 is also a core point and point p_2 is directly density-reachable from it.
- Point q is also a core point and point p_1 is directly density-reachable from it.
- Here, $p \rightarrow p_2 \rightarrow p_1 \rightarrow q$ form a chain.
- Point p is indirectly (through intermediate points p_1 and p_2) density-reachable from point q.
- Because p is not a core point as point q is not density-reachable from it. Point p has 6 nodes in the neighborhood while MinPts value is 7.

It is important to notice that the relation of density-reachable is not symmetric.

Let us consider another case given in Figure 7.23.

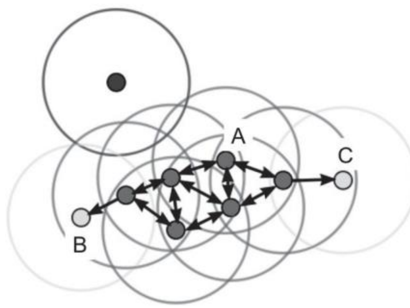


Figure 7.23 Another case of density reachability [see colour plate]

Here, in Figure 7.23, minPts = 4. Point A and the other red points are core points, because the area surrounding these points in an epsilon (ε) radius, contain at least 4 points (including the point itself). Because they are all reachable from one another, they form a single cluster. Points B and C are not core points, but are reachable from A (via other core points) and thus belong to the cluster as well. Point N is a noise point that is neither a core point nor directly-reachable.

Some more examples of DBSCAN are presented in Figure 7.24.

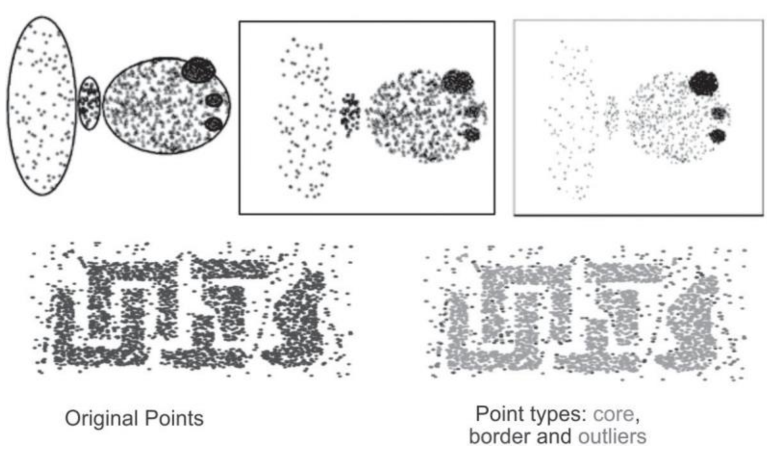


Figure 7.24 Some more examples of DBSCAN [see colour plate]

Thus, we can conclude that, a cluster then satisfies two properties:

- All points within the cluster are mutually density-connected.
- If a point is density-reachable from any point of the cluster, it is part of the cluster as well.

7.7.4 *DBSCAN algorithm*

After the understanding of concepts of neighborhood, core, border, outlier points and density reachability, DBSCAN algorithm can be summarized as given below.

Algorithm for DBSCAN

1. Find the ε (epsilon) neighbors of every point, and identify the core points with more than minPts neighbors.
2. Find the connected components of core points on the neighbourhood graph, ignoring all non-core points.
3. Assign each non-core point to a nearby cluster if the cluster is an ε (eps) neighbor, otherwise assign it to noise.

7.7.5 *Strengths of DBSCAN algorithm*

DBSCAN algorithm has following advantages.

- It is not required to specify the number of clusters in the data at the start in case of the DBSCAN algorithm.
- It requires only two parameters and does not depend on the ordering of the points in the database.
- It can identify clusters of arbitrary shape. It is also able to identify the clusters completely surrounded by a different cluster.
- DBSCAN is robust to outliers and has a notion of noise.

7.7.6 *Weakness of DBSCAN algorithm*

DBSCAN algorithm has following disadvantages.

- The DBSCAN algorithm is sensitive to the parameter, i.e., it is difficult to identify the correct set of parameters.
- The quality of the DBSCAN algorithm depends upon the distance measures like neighborhood (ε) and MinPts.
- The DBSCAN algorithm cannot cluster datasets accurately with varying densities or large differences in densities.

Association Mining

Chapter Objectives

- ✓ To comprehend the concept of association mining and its applications
- ✓ To understand the role of support, confidence and lift
- ✓ To comprehend the Naïve algorithm, Its limitations and improvements
- ✓ To learn about approaches for transaction database storage
- ✓ To understand and demonstrate the use of the apriori algorithm and the direct hashing and pruning algorithm
- ✓ To use dynamic Itemset counting to identify association rules
- ✓ To use FP growth for mining frequent patterns without candidate generation

9.1 Introduction to Association Rule Mining

Association rule mining often known as 'market basket' analysis is very effective technique to find the association of sale of item X with item Y. In simple words, market basket analysis consists of examining the items in the baskets of shoppers checking out at a market to see what types of items 'go together' as illustrated in Figure 9.1.

It would be useful to know, when people make a trip to the store, what kind of items do they tend to buy during that same shopping trip? For example, as shown in Figure 9.1, a database of customer transactions (i.e., shopping baskets) is given where each transaction consists of a set of items (i.e., products purchased during a visit). Association rule mining is used to identify groups of items which are frequently purchased together (customers' purchasing behavior). For example, 'IF one buys bread and milk, THEN he/she also buys eggs with high probability.' This information is useful for the store manager for better planning of stocking items in the store to improve its sale and efficiency.

Let us suppose that the store manager, receives customer complaints about heavy rush in his store and its consequent slow working. He may then decide to place associated items such as bread and milk together, so that customers can buy the items easier and faster than if these were at a distance. It also improves the sale of each product. In another scenario, let us suppose his store is new and the store manager wishes to display all its range of products to prospective customers. He may then decide to put the associated items, i.e., bread and milk, at opposite ends of the store and would place other new items and items being promoted in between them, thereby ensuring that many customers, on the way from bread to milk, would take notice of these and some would end up buying them.

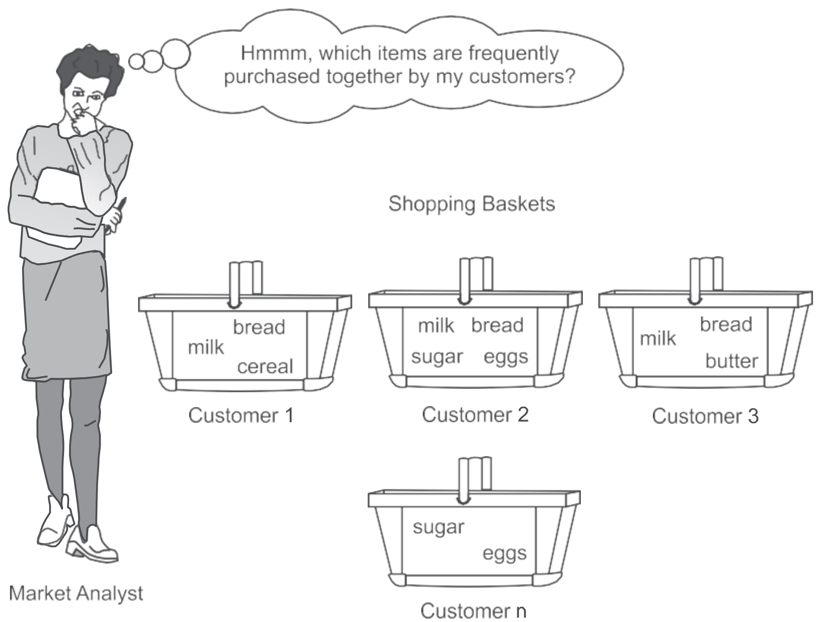


Figure 9.1 Need for association mining

Sometimes analysis of sale records leads to very interesting and unexpected results. In one very popular case study by Walmart USA, they had identified that people buying diapers often also bought beer. By putting the beer next to the diapers in their stores, it was found that the sales of each skyrocketed as depicted in Figures 9.2, 9.3 and 9.4.

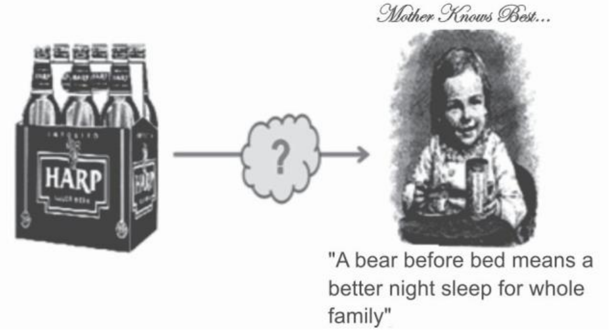


Figure 9.2 Association of sale of beer and diapers

One may make multiple conclusions about the association of bear and diapers. It may be funny to assume that 'A beer before bed means a better night's sleep for the whole family.' But serious analysis could lead to 'Young couples prepare for the weekend by stocking up on diapers for the infants and beer for dad'.

Figure 9.3 Association of sale of beer and diapers

Nevertheless, this story aptly illustrates the market basket concept of things 'going together', which is the basis of the data mining association function.

Figure 9.4 Association of sale of beer and diapers

Some other interesting findings of association mining may be trivial, for example, 'Customers who purchase maintenance agreements are very likely to purchase large appliances.' Or it may be unexplicable and unexpected, as in 'When a new restaurant opens, one of the most sold items is coffee'. This may be due to the reason that coffee is the cheapest item in the menu. So, to get the feel of the restaurant, people only order coffee. This type of finding suggests to the manager to increase the cost of coffee. It happens to be the reason why in many good restaurants, the price of coffee is alarming high.

Often, while conducting market basket analysis or association rule mining, the only source of data available on customers is the bills of purchase, which tells what items go together in a shopping cart, and can also show what items 'go together' at certain times of the day, days of the week, seasons of the year, credit versus cash versus check payment, geographical locations of stores, and so on. Discovering associations among these attributes can lead to fact-based marketing strategies for things like store f loor plans, special discounts, coupon offerings, product clustering, catalog design, identifying items that need to be put in combo packs and can be used to compare stores as shown in Figure 9.5.

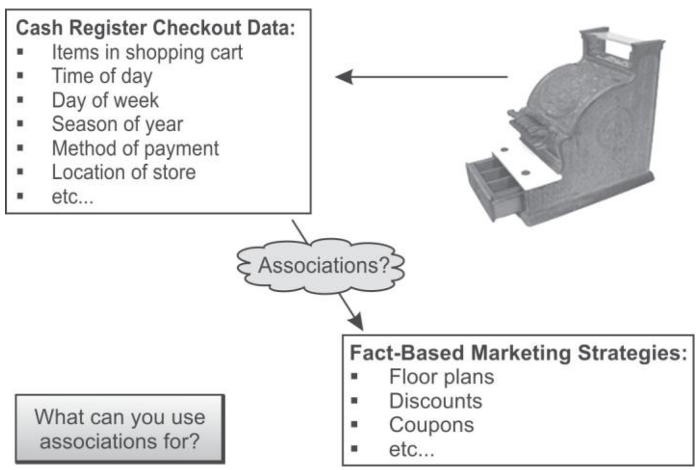


Figure 9.5 Association and customer purchase bills

Nowadays, recommendations given by online stores like Amazon and Flipkart also make use of association mining to recommend products related with your purchase and the system offers the list of products that others often buy together with the product you have just purchased. Besides the examples from market basket analysis given above, association rules are used today in many application areas such as intrusion detection, Web usage mining, bioinformatics and continuous production. Programmers use association rules to build programs capable of machine learning.

This is commonly known as market basket data analysis.

Association rule mining can also be used in applications like marketing, customer segmentation, web mining, medicine, adaptive learning, finance and bioinformatics, etc.

9.2 Defining Association Rule Mining

Association rule mining can be defined as identification of frequent patterns, correlations, associations, or causal structures among sets of objects or items in transactional databases, relational databases, and other information repositories.

Association rules are generally ***if/then*** statements that help in discovering relationships between seemingly unrelated data in a relational database or other information repository. For example, 'If a customer buys a dozen eggs, he is 80% likely to also purchase milk.'

An association rule consists of two parts, i.e., an antecedent (if) and a consequent (then). An antecedent is an object or item found in the data while a consequent is an object or item found in the combination with the antecedent.

Association rules are often written as X → Y meaning that whenever X appears Y also tends to appear. X and Y may be single items or sets of items. Here, X is referred to as the rule's antecedent and Y as its consequent.

For example, the rule found in the sales data of a supermarket could specify that if a customer buys onions and potatoes together, he or she will also like to buy burgers. This rule will be represented as onions, potatoes → burger.

The concept of association rules was formulated by two Indian scientists Dr Rakesh Agrawal and Dr R. Srikant. The detail about their brief profile is given in Section 9.7.1.

9.3 Representations of Items for Association Mining

Let us assume that the number of items in the shop stocks is n. In Table 9.1, there are 6 items in stock, namely, bread, milk, diapers, beer, eggs, cola, thus n = 6 for this shop.

The item list is represented by I and its items are represented by {i1,i2,…in}.

The number of transactions are represented by N transactions, i.e., N = 5 for the shop data as given in Table 9.1.

Table 9.1 Sale database

TID	*Items*
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

Each transaction is denoted by T {t1, t2, ...,tN} each with a unique identifier (TID) and each transaction consists of a subset of items (possibly a small subset) purchased by one customer.

Let each transaction of m items be {i1,i2, …, im}, where m <= n [number of items in a transaction should be less than or equal to total items in the shop]. Typically, transactions differ in the number of items.

As shown in Table 9.1, the first transaction is represented as T1 and it has two items, i.e., m = 2, with i1 = Bread and i2 = Milk. Similarly transaction T4, has four items, having m = 4, with i1 = Bread, i2 = Milk, i3 = Diapers and i4 = Beer.

Our task will be to find association relationships, given a large number of transactions, such that items that tend to occur together are identified. It is important to note that association rules mining does not take into account the quantities of items bought.

9.4 The Metrics to Evaluate the Strength of Association Rules

The metrics to judge the strength and accuracy of the rule are as follows:

- Support
- Confidence
- Lift

9.4.1 Support

Let N is the total number of transactions. Support of X is represented as the number of times X appears in the database divided by N, while the support for X and Y together is represented as the number of times they appear together divided by N as given below.

Support(X) = (Number of times X appears) / N = P(X)

Support(XY) = (Number of times X and Y appear together) / N = P(X ∩ Y)

Thus, Support of X is the probability of X while the support of XY is the probability of X ∩ Y.

Table 9.1, has been reproduced as Table 9.2 to calculate the supports for each item as given below:

Table 9.2 Sale database

TID	*Items*
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

Support(Bread) = Number of times Bread appears / total number of translations = 4/5 = P(Bread)

Support(Milk) = Number of times Milk appears / total number of translations = 4/5 = P(Milk)

Support(Diapers) = Number of times Diapers appears / total number of translations = 4/5 = P(Diapers)

Support(Beer) = Number of times Beer appears / total number of translations =3/5 = P(Beer)

Support(Eggs) = Number of times Eggs appears / total number of translations =1/5 = P(Eggs)

Support(Cola) = Number of times Cola appears / total number of translations = 2/5 = P(Cola)

Support(Bread, Milk) = Number of times Bread, Milk appear together / total number of translations = 3/5 = P(Bread ∩ Milk)

Support(Diapers, Beer) = Number of times Diapers, Beer appears together / total number of translations = 3/5 = P(Diapers ∩ Beer)

A high level of support indicates that the rule is frequent enough for the business to take interest in it.

Support is very important metric because if a rule has low support then it may be the case that the rule occurs by chance and it will not be logical to promote items that customers seldom buy together. But if a rule has high support then that association becomes very important and if implemented properly will result in increase in revenue, efficiency and customer satisfaction.

9.4.2 Confidence

To understand the concept of confidence, let us suppose that support for X→Y is 80%, then it means that X→Y is very frequent and there are 80% chances that X and Y will appear together in a transaction. This would be of interest to the sales manager.

Let us suppose we have another pairs of items (A and B) and support for A→B is 50%.

Of course it is not as frequent as X→Y, but if this was higher, such as whenever A appears there is 90% chance that B also appears, then of course it would be of great interest.

Thus, not only the probability that A and B appear together matters, but also the conditional probability of B when A has already occurred plays a significant role. This conditional probability that B will follow when A has already been occurred is considered during determining the confidence of the rule.

Thus, Support and Confidence are important metrics to judge the quality of the association mining rule.

A high level of confidence shows that the rule is true often enough to justify a decision based on it.

Confidence for X→Y is defined as the ratio of the support for X and Y together to the support for X (which is same as the conditional probability of Y when X has already been occurred). Therefore if X appears much more frequently than X and Y appearing together, the confidence will be low.

Confidence of (X→Y) = Support(XY) / Support(X) = P(X ∩ Y) / P(X) = P(Y|X)

P(Y|X) is the probability of Y once X has taken place, also called the conditional probability of Y.

Let us consider Table 9.3 and Ttable 9.4 to further understand the concept of support and confidence.

Table 9.3 Example of the support measure

TID	*Items*	*Support = Occurence / Total Support*
1	ABC	Total Support = 5 Support {AB} = 2/5 = 40% Support {BC} = 3/5 = 60% Support {ABC} = 1/5 = 20%
2	ABD	
3	BC	
4	AC	
5	BCD	

Table 9.4 Example of the confidence measure

TID	Items	Given X $\Rightarrow$ Y Confidence = Occurence {X and Y}/ Occurence of (X)
1	ABC	Confidence {A $\Rightarrow$ B} = 2/3 = 66% Confidence {B $\Rightarrow$ C} = 3/4 = 75% Confidence {AB $\Rightarrow$ C} = 1/2 = 50%
2	ABD	
3	BC	
4	AC	
5	BCD	

Let us consider the database given in Table 9.5 for further understanding support and confidence.

Table 9.5 Database for identification of association rules

Antecedent	*Consequent*
A	0
A	0
A	1
A	0
B	1
B	0
B	1

There are two rules derived from the association of these combinations:

Rule 1: A implies 0, i.e., A$\rightarrow$0

Rule 2: B implies 1, i.e., B$\rightarrow$1

The support for Rule 1 is the Number of times A and 0 appear together / Total Number of transactions.

Thus, Support of Rule 1 = 3/7

The Support for Rule 2 is the Number of times B and 1 appear together / Total Number of transactions.

Support of Rule 2 = 2/7

The Confidence for Rule 1 is Support of (A,0) / Support A, i.e.,

(Number of times A and 0 appear together / Total number of items) DIVIDED BY

(Number of times A appears / Total number of items)

Here, total number of items get cancelled.

Thus, the Confidence for Rule 1 is the Number of times A and 0 appear together / Number of times A appears.

So the Confidence for Rule 1 is 3/4

Similarly, the Confidence for Rule 2 is 2/3

Going back to our general example involving 'X' (the Antecedent) and 'Y' (the Consequence), the Confidence of the rule does not depend on the frequency of 'Y' appearing. But in order to identify the strength of the rule it is important to consider the frequency of X and Y; X only; and Y only. In simple words, it is important to consider the whole dataset.

TheLift of the rule however considers the whole dataset, by taking into account the probability of Y also, in deciding the strength of the rule. So, Lift is the third important metric to check the strength or power of the rule.

9.4.3 Lift

It is very important to consider the frequency of Y or probability of Y for the effectiveness of the association mining rule $X \rightarrow Y$.

As already explained, Confidence is the conditional probability of Y when X has already occurred. It is very important to consider how frequent Y is to guage the effectiveness of the confidence.

Thus, Lift is the ratio of conditional probability of Y when X is given to the unconditional probability of Y in the dataset. In simple words, it is Confidence of $X \rightarrow Y$ divided by the probability of Y.

Lift = $P(Y|X) / P(Y)$

Or

Lift = Confidence of $(X \rightarrow Y) / P(Y)$

Or

Lift = $(P(X \cap Y) / P(X))/P(Y)$

Thus, lift can be computed by dividing the confidence by the unconditional probability of consequent Y.

Let us suppose that Coke is a very common sales item in a store and that it usually appears in most of the transactions. Let us suppose that we have a rule of Candle→Coke which has a support of 20% and has a confidence of 90%. It is very logical to think that if coke is very popular and it appears in 95% of transactions, then obviously it also appears quite often with the candle as well. So, the rule for association of candle and coke will not be all that useful. But if we find that Candle→ Matchbox also has a support of 20% and a confidence of 90% then it is logical to suppose that the frequency of matchbox sales is very little as compared to the sale of coke. And the rule suggests that when we make a sale of candles, 90% chance indicates that a matchbox will also be sold in the same transaction. It is more effective and logical to conclude that when we sell a candle then we also sell a coke (coke is popular and will appear with every item not just with candle). As support and confidence are unable to handle this case, it is handled by the lift of the rule.

In this case, the probability of Y is very low in case of Candle→ Matchbox (Here, Y is matchbox) and will be very high in case of Candle→Coke (Here, Y is coke).

One can note that the low probability of Y, makes the $X \rightarrow Y$ rule more effective as compared to high probability of Y. Lift takes the note of this, and is defined as follows.

Lift = $P(Y|X) / P(Y)$

Or

Lift = Confidence of $(X \rightarrow Y) / P(Y)$

Or

Lift = (P(X ∩ Y) / P(X)) / P(Y)

Thus, the high probability of Y, i.e., P(Y) makes lift less effective and its low value makes it more effective.

Let us again consider the data given in Table 9.5, which has been reproduced in Table 9.6 for quick reference to calculate the lift of Rule 1 and Rule 2 as discussed earlier.

Table 9.6 Dataset

Antecedent	*Consequent*
A	0
A	0
A	1
A	0
B	1
B	0
B	1

Lift for Rule1, i.e., A→0 = P (0 | A) / P(0) = (P (A ∩ 0) / P(A))/ P(0) = (3/4) / (4/7)=1.3125

Lift for Rule2, i.e., B→1= P (1|B) / P(1) = (P (B ∩ 1) / P (B))/ P(1) = (2/3) / (3/7) =1.55

As discussed earlier, the confidence for Rule 1 is 3/4=0.75 and confidence for Rule 2 is 0.66.

It should be observed that although the Rule 1 has higher confidence as compared to Rule 2, but it has lower lift as compared to Rule 2.

Naturally, it would appear that Rule1 is more valuable because of having higher confidence; it appears more accurate (better supported). But, the accuracy of the rule can be misleading if it is independent of the dataset. Lift, as a metric is important because it considers both the confidence of the rule and the overall dataset.

In order to understand the importance of lift let us consider modified dataset given in Table 9.7.

In this modified database another five records have been added to the existing database (the five at the bottom). In these five records 0 appears four times, i.e., in the modified database 0 becomes more frequent; so, its probability has been increased, while the probability of 1 has been reduced.

Thus for first rule, i.e., A→0 the lift of the rule has been reduced in the modified database while it has same value of confidence as calculated below.

Support of A→0 = 3/12

Confidence of A→0 = Support(A,0) / Support(A) = 3/4 = 0.75

Lift = Confidence (A,0) / Support(0) = (3/4) / (8/12) = 36/32 = 1.125

Here, the lift of rule 1 has been reduced from 1.312 in the original dataset to 1.125 in the modified dataset, so the strength of rule 1 has been decreased in the modified dataset.

For Rule 2, i.e., B→1, the lift rule has been improved in the modified database while it has same value of confidence as calculated below.

Support of B→1 = 2/12

Confidence of B→1 = Support(B,1) / Support(B) = 2/3 = 0.66

Lift = Confidence / Support(1) = (2/3) / (4/12) =2

Here, the lift of Rule 2 has been increased from 1.555 in the original dataset to 2 in the modified dataset, so the strength of Rule 2 has been improved in the modified dataset. Thus, lift considers both the confidence of the rule and the overall dataset.

Table 9.7 Modified dataset

Antecedent	*Consequent*
A	0
A	0
A	1
A	0
B	1
B	0
B	1
C	0
D	0
C	0
C	1
E	0

Five new records inserted into the database

Another representation of association rules

Sometimes, the representation of association rules also includes support and confidence as shown in Figure 9.6.

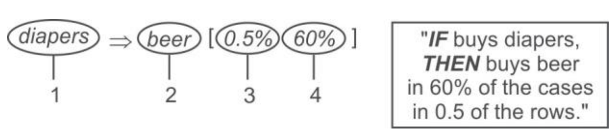


Figure 9.6 Representation of association rules

diapers ⇒ beer [0.5%, 60%]

Here,

1. Antecedent, left-hand side (LHS), body
2. Consequent, right-hand side (RHS), head
3. Support, frequency
4. Confidence, strength

9.5 The Naïve Algorithm for Finding Association Rules

To understand the Naïve Algorithm, let us consider the sales record of a grocery store. For simplicity, only four sale transactions of it are considered which deal with only four items for sale (Bread, Cornflakes, Jam and Milk) as shown in Table 9.8 and a naïve brute force algorithm is used to perform the task to identify the association of items in this sale record.

Let us find the association rules having minimum 'support' of 50% and minimum 'confidence' of 75%.

Table 9.8 Sale record of grocery store

Transaction ID	*Items*
100	Bread, Cornflakes
101	Bread, Cornflakes, Jam
102	Bread, Milk
103	Cornflakes, Jam, Milk

9.5.1 Working of the Naïve algorithm

In order to find association rules, the first step of the naïve algorithm is to list all the combinations of the items that are in stock and then identify the frequent combinations or combinations having frequency more than or equal to a specified support limit. Using this approach, the association rules that have the 'confidence' more than the threshold limit are identified.

Here, we have four items in stock, thus there are 2^4 possible combinations that we can create by considering null as one combination and if we ignore null then the following 15 combinations are possible as given in Table 9.9 along with their frequencies of occurrence in the transaction database.

Table 9.9 List of all itemsets and their frequencies

Itemsets	*Frequency*
Bread	**3**
Cornflakes	**3**
Jam	**2**
Milk	**2**
(Bread, Cornflakes)	**2**
(Bread, Jam)	1
(Bread, Milk)	1
(Cornflakes, Jam)	**2**

Contd.

Itemsets	*Frequency*
(Cornflakes, Milk)	1
(Jam, Milk)	1
(Bread, Cornflakes, Jam)	1
(Bread, Cornflakes, Milk)	0
(Bread, Jam, Milk)	0
(Cornflakes, Jam, Milk)	1
(Bread, Cornflakes, Jam, Milk)	0

Here, the minimum required support is 50% and we have to identify the frequent itemsets that appear in at least two transactions. The list of frequencies shows that all four items Bread, Cornflakes, Jam and Milk are frequent. It has been observed that the frequency goes down when we look at 2-itemsets and only two 2-itemsets such as (Bread, Cornflakes) and (Cornflakes, Jam) are frequent. All the frequent itemsets has been shown in bold in Table 9.8. There are no three or four itemsets that are frequent. The frequent itemsets are provided in Table 9.10.

Table 9.10 The set of all frequent items

Itemsets	*Frequency*
Bread	3
Cornflakes	3
Jam	2
Milk	2
(Bread, Cornflakes)	2
(Cornflakes, Jam)	2

As we are interested in association rules that can only occur with item pairs, thus individual frequent items Bread, Cornflakes, Jam and Milk are ignored, and item pairs (Bread, Cornflakes) and (Cornflakes, Jam) are considered for association rule mining.

Now, the association rules for the two 2-itemsets (Bread, Cornflakes) and (Cornflakes, Jam) are determined with a required confidence of 75%.

It is important to note that every 2-itemset (A, B) can lead to two rules A→B and B→A if both satisfy the required confidence. As stated earlier, confidence of A→B is given by dividing the support for A and B together, by the support for A.

Therefore, we have four possible rules which are given as follows along with their confidence:

Bread→Cornflakes

Confidence = Support of (Bread, Cornflakes) / Support of (Bread) = 2/3 = 67%

Cornflakes→Bread

Confidence = Support of (Cornflakes, Bread) / Support of (Cornflakes) = 2/3 = 67%
Cornflakes→Jam
Confidence = Support of (Cornflakes, Jam) / Support of (Cornflakes) = 2/3 = 67%
Jam→Cornflakes
Confidence = Support of (Jam, Cornflakes) / Support of (Jam) = 2/2 = 100%

Therefore, only the last rule Jam→Cornflakes has more than the minimum required confidence, *i.e.*, 75% and it qualifies. The rules having more than the user-specified minimum confidence are known as confident.

9.5.2 Limitations of the Naïve algorithm

We can observe that in Table 9.7, we had a small number of items. However, the number of all possible combinations for which we have to calculate the frequency grows enormously as the number of items increase in the dataset. When dealing with 4 itemsets, it produces 15 different combinations in Table 9.8 and if we include a null combination (a combination with no items) as well, we would obtain 16. If there are six items it would have produced 64 combinations and if the number of items were 10 then we would have a table with 1024 combinations.

This simple algorithm works well with four items but if the number of items is say 100, the number of combinations is much larger, in billions. The number of combinations becomes about a million with 20 items since the number of combinations is 2^n with n items. The naive algorithm can be improved to deal more effectively with larger datasets.

9.5.3 Improved Naïve algorithm to deal with larger datasets

Rather than counting all the possible item combinations in the stock it will be better to focus only on the items that are sold in transactions. Because, we may have hundreds of items in stock but we are concerned with finding associations only for the items that are being sold together. Thus, it is natural to to focus only on the items that are sold in transactions instead of all the items available in stock.

We can look at each transaction and count only the combinations that actually occur; with this approach we do not count itemsets with zero frequency. And that will be an improvement.

As an example, Table 9.11 lists all the actual combinations occurring within the transactions given in Table 9.8.

Table 9.11 All possible combinations with nonzero frequencies

Transaction ID	*Items*	*Combinations*
100	Bread, Cornflakes	(Bread, Cornflakes)
200	Bread, Cornflakes, Jam	(Bread, Cornflakes), (Bread, Jam), (Cornflakes, Jam), (Bread, Cornflakes, Jam)
300	Bread, Milk	(Bread, Milk)
400	Cornflakes, Jam, Milk	(Cornflakes, Jam), (Cornflakes, Milk), (Jam, Milk), (Cornflakes, Jam, Milk)

We therefore only need to look at the following combinations and their frequencies are given in Table 9.12.

Table 9.12 Frequencies of all itemsets with nonzero frequencies

Itemsets	*Frequency*
Bread	3
Cornflakes	3
Jam	2
Milk	2
(Bread, Cornflakes)	2
(Bread, Jam)	1
(Cornflakes, Jam)	2
(Bread, Cornflakes, Jam)	1
(Bread, Milk)	1
(Cornflakes, Milk)	1
(Jam, Milk)	1
(Cornflakes, Jam, Milk)	1

We can now proceed as before. This would work better since the list of item combinations is significantly reduced from 15 (as given in Table 9.8) to 12 (as given in Table 9.11) and this reduction is likely to be much larger for bigger problems. Regardless of the extent of the reduction, this list will also become very large for, say, 1000 items since it includes all the nonzero itemsets that exist in the transactions database whether they have minimum support or not. Therefore, the naïve algorithm or any improvement of it is not suitable for large problems. We will discuss a number of algorithms for more efficiently solving the association rule problem starting with the classical ***Apriori algorithm***.

Before starting with the Apriori algorithm, it is important to discuss different ways to store our transaction database in computer memory because, it will have an impact on performance if we are processing a large number of transactions. In the next section, implementation issues of transition storage have been discussed.

9.6 Approaches for Transaction Database Storage

It is important to understand different ways to store a dataset of transactions before processing it using algorithms, as storage is an important issue for its performance.

There are three ways to store datasets of transactions. These are as follows.

- Simple Storage
- Horizontal Storage
- Vertical Storage

Let us suppose the number of items be five; to demonstrate the different options. Let them be {I1, I2, I3, I4, I5}. Let there be only seven transactions with transaction IDs {T1, T2, T3, T4, T5, T6, T7}. This set of seven transactions with five items can be stored using three different methods given as follows.

9.6.1 Simple transaction storage

The first representation is commonly used and known as Simple Transaction Storage. Each row of the table shows the transaction ID and the purchased items as given in Table 9.13.

Table 9.13 A simple representation of transactions as an item list

Transaction ID	*Items*
T1	I1, I2, I4
T2	I4, I5
T3	I1, I3
T4	I2, I4, I5
T5	I4, I5
T6	I2, I3, I5
T7	I1, I3, I4

9.6.2 Horizontal storage

In this representation, each row is still a transaction, but columns have been created for each item. In the cell, 1 is filled against the item that occurs in a transaction and 0 against the rest as shown in Table 9.14.

Table 9.14 Horizontal storage representation

TID	*I1*	*I2*	*I3*	*I4*	*I5*
T1	1	1	0	1	0
T2	0	0	0	1	1
T3	1	0	1	0	0
T4	0	1	0	1	1
T5	0	0	0	1	1
T6	0	1	1	0	1
T7	1	0	1	1	0

The advantage of this storage system is that we can count the frequency of each item by counting the '1's in the given column. As we can easily calculate the frequency of item I1 as 3 and item I4 as

1. We can also easily calculate the frequency of item pairs by counting the 1's that result after an 'AND' operation on corresponding columns. For example, the frequency of item pairs I1 and I2 is 1 (By AND operation on column of I1 and I2 we will get only one 1, i.e., T1).

9.6.3 Vertical representation

In this representation as given in Table 9.15, the transaction list is turned around. Rather than using each row to represent a transaction of the items purchased, each row now represents an item and it indicates transactions in which the item appears. The columns now represent the transactions. This representation is also called a TID-list since for each item it provides a list of TIDs (Transition Ids).

Table 9.15 Vertical storage representation

Item	*TID*						
	T1	*T2*	*T3*	*T4*	*T5*	*T6*	*T7*
I1	1	0	1	0	0	0	1
I2	1	0	0	1	0	1	0
I3	0	0	1	0	0	1	1
I4	1	1	0	1	1	0	1
I5	0	1	0	1	1	1	0

A vertical representation also facilitates counting of items by counting the number of 1s in each row and in case of 2-itemsets, where you want to find out the frequency of occurrences of 2 items together in a transaction, you have to refer to the intersection of 2 rows corresponding to the items, and inspect one column at a time.

Example: If you want to find out frequency of item pairs (I1,I2) just look at rows I1 and I2 for each column if values of both I1 and I2 is 1 then this means I1 and I2 are occurring together in a transaction hence write their count as 1, move further in some way for each transaction. Finally you will get Table 9.16.

From the table it can be seen that the count for item pair {I1,I2}is 1; similarly the count of item pair {I1, I4} is 2. (Table 9.16)

Table 9.16 Frequency of item pairs

	T1	T2	T3	T4	T5	T6	T7
I1, I2	1	0	0	0	0	0	0
I1, I4	1	0	0	0	0	0	1

The vertical representation is not storage efficient in case of very large number of transactions. Also, if an algorithm needs data in the vertical representation then there would be a cost in transforming data from the horizontal representation to the vertical one.

With this we can move on to the concept of the Apriori algorithm.

9.7 The Apriori Algorithm

The Apriori algorithm was developed by two Indians Rakesh Agrawal and Ramakrishnan Srikant in 1994, to mine frequent itemsets for identifying association rules.

It should be a matter of pride and motivation to have two Indians as inventors of association mining. Their work is the base for commonly used recommendation systems for modern applications such as product recommendations for online purchases and video recommendation and so on.

A brief profile of these two scientists, based on Wikipedia, has been presented below to motivate readers and in appreciation of their efforts.

9.7.1 About the inventors of Apriori

Rakesh Agrawal is the President and Founder of the Data Insights Laboratories. He is also the President of the Professor Ram Kumar Memorial Foundation.

Rakesh is an innovator and thought leader driven by the desire to make the world better through scientific breakthroughs and by building practical working systems. He is the recipient of the ACM-SIGKDD Inaugural Innovation Award, ACM-SIGMOD Edgar F. Codd Innovations Award, VLDB 10-Yr Most Influential Paper Award. Scientific American has included him in its first list of 50 top scientists and technologists.

Until recently, Rakesh was a Microsoft Technical Fellow and headed the Search Labs in Microsoft Research. Prior to joining Microsoft in March 2006, Rakesh was an IBM Fellow and led the Quest group at the IBM Almaden Research Center. Earlier, he was with the Bell Laboratories, Murray Hill from 1983 to 1989. He also worked for three years at a leading Indian, namely Bharat Heavy Electricals Ltd. He received the M.S. and Ph.D. degrees in Computer Science from the University of Wisconsin-Madison in 1983. He also holds a B.E. degree in Electronics and Communication Engineering from IIT-Roorkee, and a two-year Post Graduate Diploma in Industrial Engineering from the National Institute of Industrial Engineering (NITIE), Bombay. Both IIT-Roorkee and NITIE have decorated him with their distinguished alumni awards.

Rakesh has been granted 83 patents. He has published more than 200 research papers, many of them considered seminal. He has written the 1st as well as the 2nd highest cited of all papers in the fields of databases and data mining (18th and 26th most cited across all computer science). Wikipedia lists one of his papers as one of the most influential database papers. His papers have been cited more than 80,000 times, with more than 25 of them receiving more than 500 citations each and three of them receiving 5,000 citations each (Google Scholar). He is the most cited author in the field of database systems and the 26th most cited author across all of Computer Science (Citeseer). His research has been featured in N.B.C., New York Times, and several other media channels.

It is rare that a researcher's work creates not only a product, but a whole new industry. IBM's data mining product, Intelligent Miner, grew straight out of Rakesh's research. IBM's introduction of Intelligent Miner and associated services created a new category of software and services. His research has been incorporated into many other commercial products, including DB2 Mining Extender, DB2 OLAP Server, WebSphere Commerce Server, and Microsoft Bing Search engine, as well as many research prototypes and applications.

To know more, please visit http://rakeshagrawal.org/

Ramakrishnan Srikant is a Distinguished Research Scientist at Google. His research interests include data mining, online advertising, and user modeling.

He previously managed the Privacy Research group, part of the Intelligent Information Systems Research department at IBM's Almaden Research Center.

Srikant has published more than 30 research papers that have been extensively cited. He has been granted 25 patents. Dr Srikant was a key architect for the IBM Intelligent Miner, and contributed the association rules and sequential patterns modules to the product.

Srikant received the ACM SIGKDD Innovation Award (2006) for his seminal work on mining association rules and privacy preserving data mining, the 2002 ACM Grace Murray Hopper Award for his work on mining association rules, the VLDB 2004 Ten Year Best Paper Award, and the ICDE 2008 Influential Paper Award. He was named an IBM Research Division Master Inventor in 1999. He has received 2 Outstanding Technical Achievement Awards for his contributions to the Intelligent Miner, and an Outstanding Innovation Award for his work on Privacy in Data Systems.

Srikant received his M.S. and Ph.D. from the University of Wisconsin, Madison and his B. Tech. from the Indian Institute of Technology, Madras.

To know more, please visit http://www.rsrikant.com/index.html

9.7.2 Working of the Apriori algorithm

The Apriori algorithm has been named so on the basis that it uses prior knowledge of frequent itemset properties. This algorithm consists of two phases. In the first phase, the frequent itemsets, i.e., the itemsets that exceed the minimum required support are identified. In the second phase, the association rules meeting the minimum required confidence are identified from the frequent itemsets. The second phase is comparatively straight forward – therefore, the major focus of research in this field is to improve the first phase.

To understand the Apriori algorithm, let us follow the learn by example approach. We will first apply this algorithm on a simple database to illustrate its working and then we will go into detail to learn the concept more deeply.

Example: Let us consider an example of only five transactions and six items. We want to identify association rules with 50% support and 75% confidence with the Apriori algorithm. The transactions are given in Table 9.17.

Table 9.17 Transactions database

Transaction ID	Items
T1	Bread, Cornflakes, Eggs, Jam
T2	Bread, Cornflakes, Jam
T3	Bread, Milk, Tea
T4	Bread, Jam, Milk
T5	Cornflakes, Jam, Milk

For 50% support each frequent item must appear in at least three transactions. The first phase of the Apriori algorithm will be to find frequent itemsets.

Phase 1: Identification of frequent itemsets

It starts with the identification of candidate 1 itemsets, represented by C1 (one itemsets that may be frequent) and it is always the items in the stock which the store deals with, Here, we have six items, so C1 will be as shown below.

Table 9.18 Candidate one itemsets C1

Item
Bread
Cornflakes
Eggs
Milk
Jam
Tea

From Candidate 1 itemsets, frequent one-itemsets are represented by L1 and found by calculating the count of each candidate item and selecting only those counts which are equal to or more than the threshold limit of support, i.e., 3 in this case. Thus, frequent single itemsets, L1, will be as shown in Table 9.19.

Table 9.19 Frequent items L1

Item	*Frequency*
Bread	4
Cornflakes	3
Milk	3
Jam	4

Then, the candidate 2-itemsets or C2 are determined per the process illustrated below:
C2 = L1 JOIN L1.
The joining of L1 with itself has been illustrated below.

Bread		Bread
Cornflakes	JOIN	Cornflakes
Milk		Jam
Jam		Milk

In the Join operation the first step will be to perform a Cartesian product, i.e., to make all possible pairs between L1 and L1 as shown below.

Bread, Bread
Bread, Cornflakes
Bread, Milk
Bread, Jam
Cornflakes, Bread
Cornflakes, Cornflakes
Cornflakes, Milk
Cornflakes, Jam
Milk, Bread
Milk, Cornflakes
Milk, Milk
Milk, Jam
Jam, Bread
Jam, Cornflakes
Jam, Milk
Jam, Jam

Here, we have 16 possible pairs. In association mining, we are interested in item pairs, so the pairs having same itemsets (like Bread, Bread) need to be considered for removal from the list. Similarly, if we already have an item pair of Milk, Jam then there is no need to list item pair Jam, Milk so it will also be removed from the list.

Thus, to create C2 the following steps are applied:

- Perform Cartesian product of L1 with itself
- Some item pairs may have identical items. Keep just one. Remove the extras.
- Select only those item pairs in which items are in lexical order (so that if we have Milk, Jam then Jam, Milk should not appear).

Thus, the final C2 will be as shown below:

Bread, Cornflakes
Bread, Milk
Bread, Jam
Cornflakes, Milk
Cornflakes, Jam
Milk, Jam

The frequency of each of these item pairs will be found as shown in the table given below.

Table 9.20 Candidate item pairs C2

Item pairs	*Frequency*
(Bread, Cornflakes)	2
(Bread, Milk)	2
(Bread, Jam)	3

Contd.

Item pairs	*Frequency*
(Cornflakes, Milk)	1
(Cornflakes, Jam)	3
(Milk, Jam)	2

We therefore have only two frequent item pairs in L2 as shown in Table 9.21.

Table 9.21 Frequent two item pairs L2

Item pairs	*Frequency*
(Bread, Jam)	3
(Cornflakes, Jam)	3

Generation of C3 from L2

C3 is generated from L2 by carrying out a JOIN operation over L2 as shown below.

C3 = L2 JOIN L2

It will involve the same steps as performed for C2, but it has one important pre-requisite for Join, i.e., two items are joinable if their first item is common. In a generalized case:

Ck = Lk-1 JOIN Lk-1

And they are joinable if their first k-2 items are the same. So, in case of C3, the first item should be the same in L2, while in case of C2 there is no requirement of first item similarity because k-2 in the C2 case is 0.

From {Bread, Jam} and {Cornflakes, Jam} two frequent 2-itemsets, we do not obtain a candidate 3-itemset since we do not have two 2-itemsets that have the same first item. This completes the first phase of the Apriori algorithm.

Phase 2: Generation of rules

The two frequent 2-itemsets given above lead to the following possible rules.

Bread→Jam

Jam→Bread

Cornflakes→Jam

Jam→Cornf lakes

The confidence of these rules is obtained by dividing the support for both items in the rule by the support for the item on the left-hand side of the rule. The confidence of the four rules therefore are 3/4 = 75%, 3/4 = 75%, 3/3 = 100%, and 3/4 = 75% respectively. Since all of them have a minimum 75% confidence, they all qualify.

In order to generalize the Apriori algorithm, let us define the representation of itemsets.

Representation of Itemsets

To generate C_k, let us first define L_{k-1}.

Let *l*1 and *l*2 be itemsets in list L_{k-1} and the notation *li*[*j*] refers to the *j*th item in *li*.

To generate C2 from L1, C2 is represented as C_k and L1 as L_{k-1}. Let us consider L1 as shown in Table 9.22, the description of each item list has been given below.

Table 9.22 L1 for generation of C2 having only one element in each list

L1	
1	represented as item list l1 and its item, i.e., 1 is represented as l1[1].
2	represented as item list l2 and its item, i.e., 2 is represented as l2[1].
3	represented as item list l3 and its item, i.e., 3 is represented as l3[1].
5	represented as item list l4 and its item, i.e., 5 is represented as l4[1].

Let us consider L2 given in Table 9.23.

Table 9.23 L2 for generation of C3 (i.e., K=3) having two elements in each list

L2	
1,2	represented as item list l1 and its item, 1 is represented as l1[1] or l1[k-2] and 2 is represented as l1[2] or l1[k-1]
4,5	represented as item list l2 and its item, 4 is represented as l2[1] or l2[k-2] and 5 is represented as l2[2] or l2[k-1]
2,6	represented as item list l3 and its item, 2 is represented as l3[1] or l3[k-2] and 6 is represented as l3[2] or l3[k-1]

Let us consider L3 given in Table 9.24.

Table 9.24 L3 for generation of C4 (i.e., K = 4) having three elements in each list

L3	
1,2,3	represented as item list l1 and its item, 1 is represented as l1[1] or l1[k-3] and 2 is represented as l1[2] or l1[k-2] and 3 is represented as l1[3] or l1[k-1]
1,2,5	represented as item list l2 and its item, 1 is represented as l2[1] or l2[k-3] and 2 is represented as l2[2] or l2[k-2] and 5 is represented as l2[3] or l2[k-1]
2,3,6	represented as item list l3 and its item, 2 is represented as l3[1] or l3[k-3] and 3 is represented as l3[2] or l3[k-2] and 6 is represented as l1[3] or l1[k-1]

Defining Phase 1: Identification of frequent itemsets

The Apriori algorithm uses an iterative approach and is also known as a level-wise search where k-itemsets are used to search (k+1) itemsets. First, the database is scanned to find the set of frequent1-itemsets to accumulate the count for each item denoted as C1, known as candidate 1-itemset, and gathering those items satisfying minimum required support denoted as L1, frequent 1-itemset.

Next, L1 is used to find C2, candidate 2-itemsets and collecting those items that satisfy minimum support denoted as L2, frequent 2-itemsets; which is used to find C3, candidate 3-itemsets; which is used further to identify L3, frequent 3-itemsets and so on, until no more frequent k-itemsets can be found. This process has been illustrated in Figure 9.7. The finding of each L_k requires one full scan of the database.

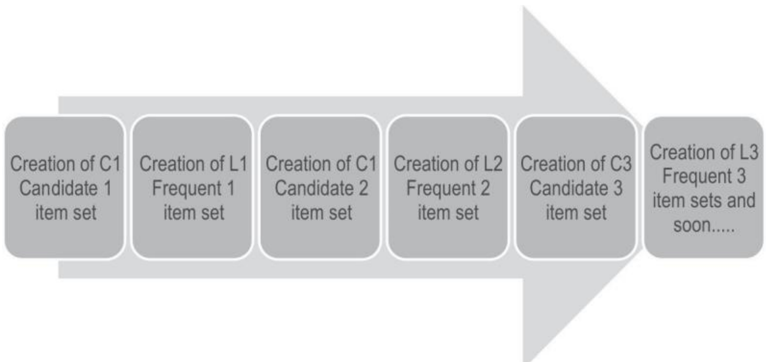


Figure 9.7 Process for identification of frequent itemsets

To proceed further, we have to generate C2 based on L1. To understand the generation of candidate itemsets, it is important to understand the representation of items in the item lists.

Step 3: Form candidate 2 itemset pair, i.e., C2.

C2 is created by joining L1 with itself by using following joining rule.

The join step

To find C_k, a set of candidate k-itemsets denoted as C_k is generated by joining L_{k-1} with itself. Let $l1$ and $l2$ be itemsets in L_{k-1}, the notation $li[j]$ refers to the jth item in li, e.g., $l1[k\text{-}2]$ refers to the second last item in $l1$. By convention, the prior assumption is that items within a transaction or itemset are sorted in lexicographic order. For the (k-1) itemset, li, this means that the items are sorted such that $li[1] < li[2] < : : : < li[k\text{-}1]$.

The join, L_{k-1} on L_{k-1}, is performed, where members of L_{k-1} are joinable if their first (k-2) items are in common.

That is, members $l1$ and $l2$ of L_{k-1} are joined if $(l1[1] = l2[1]) \wedge (l1[2] = l2[2]) \wedge : : : \wedge (l1[k\text{-}2] = l2[k\text{-}2]) \wedge (l1[k\text{-}1] < l2[k\text{-}1])$. The condition $l1[k\text{-}1] < l2[k\text{-}1]$ simply ensures that no duplicates are generated.

The resulting itemset formed by joining $l1$ and $l2$ is $l1[1], l1[2], : : : , l1[k\text{-}2], l1[k\text{-}1], l2[k\text{-}1]$.

Thus, there are three important points that make items of L_{k-1} joinable and these are as follows.

Point 1: The members of L_{k-1} are joinable if their first (k-2) items are in common

It means for C2 = L1 JOIN L1, with k = 2, there is no requirement for having the first common data item, because k-2 = 0.

For C3 = L2 JOIN L2, with k = 3, the first k-2, i.e., first 1 item should be the same in the itemset.

For C4 = L3 JOIN L3, having k = 4, the first k-2, i.e., first 2 items should be the same in the itemset.

For example, Let us verify whether L1 given in Table 9.25 is joinable or not, to create C2.

Table 9.25 L1

L1
1
2
3

C2 = L1 JOIN L1

*There is no requirement for first common item because k-2 = 0 in this case and all itemsets, i.e., 1, 2 and 3 are joinable.

Now, let us verify whether L2 given in Table 9.26 is joinable or not to create C3.

Table 9.26 L2

L2	
1,2	represented as item list l1 and its item, i.e.,1 is represented as l1[1] or l1[k-2] and 2 is represented as l1[2] or l1[k-1]
1,3	represented as item list l2 and its item, i.e.,1 is represented as l2[1] or l2[k-2] and 3 is represented as l2[2] or l2[k-1]
2,5	represented as item list l3 and its item, 2 is represented as l3[1] or l3[k-2] and 5 is represented as l3[2] or l3[k-1]

C3 = L2 JOIN L2 [Here, k = 3]

Itemsets are joinable only if their first k-2 itemsets, i.e., first 1 item is common. Thus, itemsets 1, 2 and 1, 3 are joinable as their first element is 1 in both the itemsets.

Now, let us identify that L3 given in Table 9.27 is joinable or not to create C4.

Table 9.27 L3

L3	
1,2,3	represented as item list l1 and its item, i.e.,1 is represented as l1[1] or l1[k-3] and 2 is represented as l1[2] or l1[k-2] and 3 is represented as l1[3] or l1[k-1]
1,2,5	represented as item list l2 and its item, i.e.,1 is represented as l2[1] or l2[k-3] and 2 is represented as l2[2] or l2[k-2] and 5 is represented as l2[5] or l2[k-1]
1,3,5	represented as item list l3 and its item, 1 is represented as l3[1] or l3[k-3] and 3 is represented as l3[2] or l3[k-2] and 5 is represented as l1[3] or l1[k-1]

C4 = L3 JOIN L3 [Here, k = 4]

Itemsets are joinable only if their first k-2 itemsets, i.e., first 2 items are common in this case. Thus, itemsets 1, 2, 3 and 1, 2, 5 are joinable as their first two elements, i.e., 1 and 2 are common in both the itemsets.

Point 2

In joinable item lists l1(k-1) should be less than l2(k-1), i.e., l1(k-1)<l2(k-1): this rule ensures that items in the itemset should be in lexicographic order and there is no repetition.

To understand this condition, let us find C2 for given L1 in Table 9.28.

Table 9.28 L1

L1
1
2
3
5

As discussed earlier, there is no requirement for the first common item, because k-2 = 0 in this case and all itemsets, i.e., 1, 2, 3 and 5 are joinable.

Thus, C2 = L1 JOIN L1 (the base for join is the Cartesian product of two itemsets) as illustrated in Table 9.29.

Table 9.29 Generation of C2

1	JOIN	1
2		2
3		3
5		5

Possible pairs for 1 are 11*, 12, 13&15

*11 will not appear in the final list because, here l1(1) = l2(1) and according to the joining rule l1(1) should be less than l2(1) [1 is not less than 1].

Thus, selected pairs for 1 are 12, 13 & 15

Possible pairs for 2 are 21*, 22*, 23 & 25

21 and 22 will not be selected because, l1(1) is not less than l2(1) [2 is not less than 1 and 2 is not less than 2].

Thus, selected pairs for 2 are 23 & 25

Possible pairs for 3 are 31*, 32*, 33* & 35

31, 32 and 33 will not be selected because , l1(1) is not less than l2(1) [3 is not less than 1; 3 is not less than 2; and 3 is not less than 3].

Thus, the selected pair for 3 is 35
Possible pairs for 5 are 51*, 52*, 53* & 55*
*No item will be selected because, l1(1) in not less than l2(1) [5 is not less than 1,2,3 and 5].
Thus, C2 will be as shown in Table 9.30.

Table 9.30 Generated C2

C2
12
13
15
23
25
35

Point 3

The resulting itemset formed by joining *l*1 and *l*2 is *l*1[1], *l*1[2], : : : , *l*1[*k*-2], *l*1[*k*-1], *l*2[*k*-1].

It means that, resulting itemset will contain all the items of the first item list, i.e., from item 1 to k-1 and last item of the second item list, i.e., k-1 item of second list, because other k-2 items of second list are already there, as they are common with first list.

To illustrate this point, let us identify C3 for given L2 in Table 9.31.

Table 9.31 L2

L2
1,2
1,3
2,3

Here, itemsets 1, 2 (first list) and 1, 3 (second list) are joinable. The final item list for C3 will be all items of first list and last item of second list.

Thus, C3 will be as shown in Table 9.32.

Table 9.32 C3

C3
1, 2, 3

Let us identify C4 for the given L3 in Table 9.33.

Table 9.33 L3

L3
1, 2, 4
1, 2, 5
1, 2, 7
1, 3, 6

Here, itemsets (1, 2, 4), (1, 2, 5) and (1, 2, 7) are joinable. The resultant C4 will be as shown in Table 9.34.

Table 9.34 C4

C4	
1, 2, 4, 5	Considering (1, 2, 4) as first list and (1, 2, 5) second list
1, 2, 4, 7	Considering (1, 2, 4) as first list and (1, 2, 7) second list
1, 2, 5, 7	Considering (1, 2, 5) as first list and (1, 2, 7) second list

Example

Let us understand the process for identification of frequent itemsets for the given transaction database of Table 9.35. The threshold value of support is 50% and confidence is 70%,

Table 9.35 Transaction database

T1	1,3,4
T2	2,3,5
T3	1,2,3,5
T4	2,5

Step 1

Create C1 candidate 1-itemsets. It is all the items in the transaction database as shown in Figure 9.11. Figure 9.8 also contains the frequency of each item.

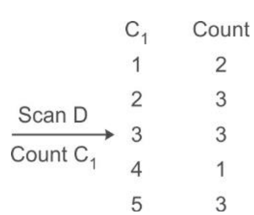


Figure 9.8 C1, candidate 1-itemset and their count

Step 2

Create frequent 1-itemset, i.e., list of 1-itemsets whose frequency is more than the threshold value of support, i.e., 2 as shown in Figure 9.9.

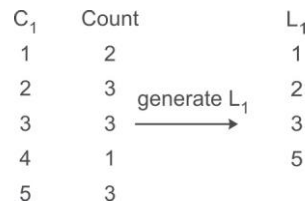


Figure 9.9 L1, frequent 1-itemset

Candidate two itemsets, C2 with its frequency count as given in Table 9.36.

Table 9.36 Generation of C2

C2	*Count*
1, 2	1
1, 3	2
1, 5	1
2, 3	2
2, 5	3
3, 5	2

L2 is created by selecting those candidate pairs having support of 2 or more as shown in Table 9.37.

Table 9.37 Generation L2

L2
1, 3
2, 3
2, 5
3, 5

From the given L2, the next step will be to generate C3 by using L2 JOIN L2
Thus, C3 will be as shown in Table 9.38.

Table 9.38 Generation of C3

C3
2, 3, 5

The frequency of candidate three itemset 2, 3, 5 is 2, so C3 is qualified as L3. Thus, frequent three itemset is 2, 3, 5 for given dataset.

This is the final frequent item list and it completes the first phase of the Aproiri algorithm to find the frequent itemsets.

Phase 2: Generating association rules from frequent itemsets

To understand the process of generation of association rules from the frequent itemset, let us consider frequent itemset *l*.

- For each frequent itemset *l* generates all non-empty subsets of *l*.
- For every non-empty subset *s* of *l*, output the rule *s*→*l-s* and if the confidence of the rule is more than the threshold value of the confidence, then, this rule will be selected as the final association rule.

Let us generate the rules for the frequent itemset (2, 3, 5) represented as *l*. Here, non-empty subsets are {{2},{3},{5},{(2, 3)}, {(2, 5)},{(3, 5)}.

For every non-empty subset, the rule will be generated as follows.

2→3, 5 [Here, (2) is *s* and (3, 5) is *l-s*]

3→2, 5

5→2, 3

2, 3→5

2, 5→3

3, 5→2

The next step will be to calculate the confidence for each rule as shown below.

2→3, 5; Confidence = S(2∩3∩5) / S(2) = 2/3 = 0.67

3→2, 5; Confidence = S(2∩3∩5) / S(3) = 2/3 = 0.67

5→2, 3; Confidence = S(2∩3∩5) / S(5) = 2/3 = 0.67

2, 3→5; Confidence = S(2∩3∩5) / S(2∩3) = 2/2 = 1.0

2, 5→3; Confidence = S(2∩3∩5) / S(2∩5) = 2/3 = 0.67

3, 5→2; Confidence = S(2∩3∩5) / S(3∩5) = 2/2 = 1.0

Here, the minimum threshold for confidence is 70%, thus selected association rules are as follows.

2, 3→5;

3, 5→2;

The possible rules from 2, 3→5

It is intuitive to create two new rules as 2→5 and 3→5 from the given rule. Since 2, 3→5 has confidence more than the threshold limit, in this case, it is not implicit or guaranteed that 2→3 and 3→5 will have confidence more than the threshold limit as there is no correlation between denominator and quotient of both the rules as shown below.

Confidence of 2, 3→5 = S(2∩3∩5) / S(2∩3)

Confidence of 2→5 = S(2∩5) / S(2)

Same is the case for 3→5 as shown below.

Confidence of 3→5 = S(3∩5) / S(3)

Thus, the given high confidence rule 2, 3→5 does not implicitly state that 2→5 and 3→5 will have its confidence more than the threshold limit. Thus, there is a need to calculate the confidence of these rules as shown in Table 9.39.

Similarly, the given high confidence rule 3, 5→2 does not implicitly state that 3→2 and 5→2 will have its confidence more than the threshold limit. Thus, again there is a need to calculate the confidence of these rules as shown in Table 9.39.

Table 9.39 Calculation of confidence

Rule	*Confidence*
2, 3→5	1.0
2→5	S(2∩5) / S(2) = 3/3 = 1.0
3→5	S(2∩5) / S(3) = 3/3 = 1.0
3, 5→2	1.0
3→2	S(3∩2) / S(3) = 2/3 = 0.67
5→2	S(5∩2) / S(5) = 3/3 = 1.0

Thus, final rules having confidence more than the threshold limit, i.e., 75% are as follows.

Selected Association rules are
2, 3→5
2→5
3→5
3, 5→2
5→2

Apriori property

Apriori property states that all nonempty subsets of a frequent itemset must also be frequent.

It means all the subsets of the candidate itemset should be frequent otherwise that itemset will be removed from the candidate itemset. This step is called pruning of the candidate itemset and it will help to reduce the search space and to improve the performance of the algorithm because now the frequency of fewer item pairs need to be computed.

Lattice structure of frequent itemsets

The Apriori property that defines an itemset can be frequent only if all the subsets of the itemset are frequent. This can be illustrated by a lattice structure. Let us suppose there are four items A, B, C and D in the dataset. Then, the four-itemset ABCD will be frequent only if three itemsets

ABC, ABD, ACD and BCD are frequent. These three itemsets are frequent only if their subsets, i.e., AB, AC, BC, BD, AD and CD are frequent. And these two itemsets are frequent only if their subsets, i.e., A, B, C and D items are also frequent as shown in Figure 9.10.

From the lattice structure we can conclude that the Apriori algorithm works bottom up one level at a time, i.e., it first computes frequent 1-itemsets, the candidate 2-itemsets and then frequent 2-temsets and so on. The number of scans of the transaction database is equal to the maximum number of items in the candidate itemsets.

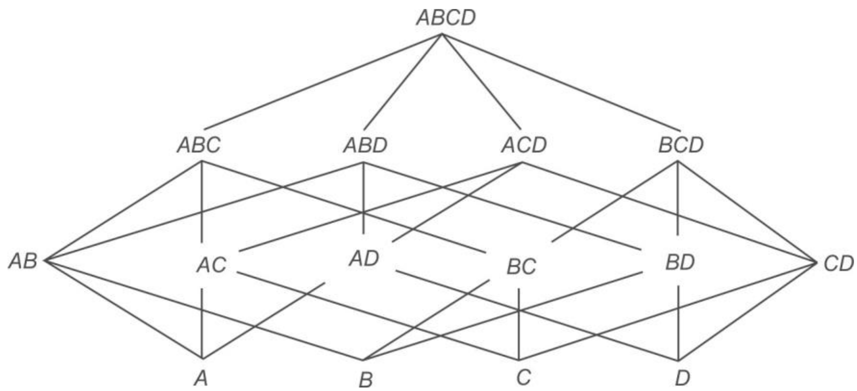


Figure 9.10 Lattice structure of frequent itemsets

For example, Apriori property states that for a frequent three itemset (1, 2, 3) all of its non empty subsets, i.e., (1, 2), (2, 3) and (1, 3) must be frequent, because, support of a superset is always less than or equal to support of its subset. It means that if (1, 3) is not frequent then (1, 2, 3) will also not be frequent.

This Apriori property also explains the significance of having first k-2 items common in a joining principle.

Let us suppose, we have L2 that is (1, 2) and (2, 3), then, one may suppose that it could produce a set (1,2,3) as C3. But as discussed earlier, (1, 2, 3) should be frequent only if all of its nonempty subsets, i.e., (1, 2), (2, 3) and (1, 3) are frequent. Here, as given in L2 (1, 3) is not frequent. So, itemsets given in L2 are considered as non joinable and the condition of having first k-2 items common is enforced to ensure it.

Now, what would happen if L2 has (1, 2) and (1, 3) but not (2, 3)? Then, according to the joining principle C3 will be generated as (1, 2, 3), but it will be discarded by the Apriori property because one of its nonempty subset, i.e., (2, 3) is not frequent. Thus, the combination of joining principle and apriori property make the whole process complete.

In conclusion, it is not the only joining principle that decides about the candidate itemsets, the joining principle can produce some extra items sets that can be removed further by the pruning or apriori property.

Example: Find association rules for the transaction data given in Table 9.40 for having support 15% and confidence 70%.

Table 9.40 Transaction database for identification of association rules

TID	*List of Items*
10	I1, I2, I5
20	I2, I4
30	I2, I3
40	I1, I2, I4
50	I1, I3
60	I2, I3
70	I1, I3
80	I1, I2, I3, I5
90	I1, I2, I4

The frequency count for each item, C1 is given in Table 9.41.

Table 9.41 C1

C1	*Count*
I1	6
I2	7
I3	5
I4	3
I5	2

All the items have frequency more than the specified support limit, thus all 1-item candidate sets given as C1 qualify as 1-item frequent itemset L1. The next step will be to generate C2.

C2 = L1 JOIN L1

The process of creation of C2 and L2 is illustrated in Figure 9.11.

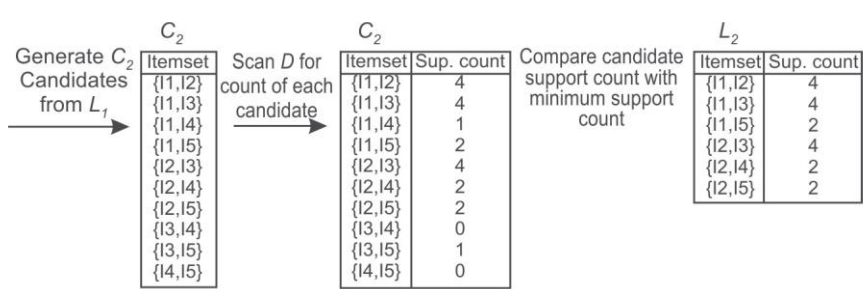


Figure 9.11 Generation of C2 and L2

From the given L2, C3 is generated by considering those item pairs whose first k-2 items, i.e. first 1 item is common as shown in Table 9.42.

Table 9.42 Generation of C3

C3
I1, I2, I3
I1, I2, I5
I1, I3, I5
I2, I3, I4
I2, I3, I5
I2, I4, I5

Before finding the frequency of each candidate item pair, the Apriori property will be applied to prune the candidate list.

Prune using the Apriori property: All nonempty subsets of a frequent itemset must also be frequent. Do any of the candidates have a subset that is not frequent? Let us check that as shown in Table 9.43.

Table 9.43 Pruning of candidate itemset C3

I1, I2, I3	The 2-item subsets of I1, I2 and I3 are (I1, I2), (I1, I3), and (I2, I3). All 2-item subsets are members of L2. Therefore, keep I1, I2 and I3 in C3.	Qualify
I1, I2, I5	The 2-item subsets of I1, I2 and I5 are (I1, I2), (I1, I5), and (I2, I5). All 2-item subsets are members of L2. Therefore, keep I1, I2 and I5 in C3.	Qualify
I1, I3, I5	The 2-item subsets of I1, I3 and I5 are (I1, I3), (I1, I5), and (I3, I5). Since, (I3, I5) is not a member of L2, and so it is not frequent. Therefore, remove I1, I3 and I5 from C3.	Does not qualify
I2, I3, I4	The 2-item subsets of I2, I3 and I4 are (I2, I3), (I2, I4), and (I3, I4). Since, (I3, I4) is not a member of L2, and so it is not frequent. Therefore, remove I2, I3 and I4 from C3.	Does not qualify
I2, I3, I5	The 2-item subsets of I2, I3 and I5 are (I2, I3), (I2, I5), and (I3, I5). Since, (I3, I5) is not a member of L2, so it is not frequent. Therefore, remove I2, I3 and I5 from C3.	Does not qualify
I2, I4, I5	The 2-item subsets of I2, I4 and I5 are (I2, I4), (I2, I5), and (I4, I5). Since, (I4, I5) is not a member of L2, it is not frequent. Therefore, remove I2, I4 and I5 from C3.	Does not qualify

Thus pruned C3 will be as shown in Table 9.44.

Table 9.44 Pruned C3

Pruned C3
I1, I2, I3
I1, I2, I5

It is important to note that pruning results into improving the efficiency of the algorithm, as in this case instead of finding the count of six three item pairs, there is a need to find the count for only 2 three item pairs (pruned list) which is a significant improvement in the efficiency of the system.

The process of creation of L3 from C3 by identifying those 3 itemsets that has a frequency more than or equal to the given value of threshold value of support, i.e., 2 has been shown below in Figure 9.12.

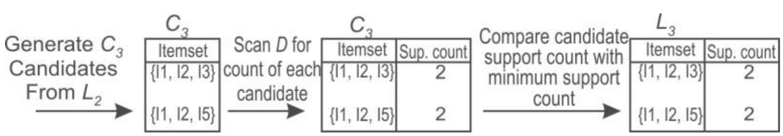


Figure 9.12 Generation of C3 and L3

The next step will be to generate C4 as shown below.

C4 = L3 JOIN L3

C4 will be I1, I2, I3 & I5, because 2 items are common as shown in Table 9.45.

Table 9.45 C4

C4
I1, I2, I3, I5

But this itemset is pruned by the Apriori property because its subset (I2, I3, I5) is not frequent as it is not present in L3. Thus, C4 is null and the algorithm terminates at this point, having found all of frequent itemsets as shown in Table 9.46.

Table 9.46 Pruned C4

C4
NULL

In this case, instead of finding the count of 1 four-item pair, pruning indicates that there is no need to find its count as all its subsets are not frequent.

With this, the first phase of the Apriori algorithm has been completed. Now the next phase to generate association rules from frequent itemsets will be performed.

Phase 2: Generating association rules from frequent itemsets

Let us apply this rule to frequent 3-itemsets (I1, I2, I3) and (I1, I2, I5) found in case of example given in Table 9.40.

For first frequent itemset (I1, I2, I3), non-empty subsets are {{I1},{I2},{I3},{(I1, I2)}, {(I1, I3)},{(I2, I3)}}.

For every non-empty set, the rule will be generated as follows:

I1→I2, I3 [Here, (I1) is *s* and I2 and I3 are *l-s*]

I2→I1, I3

I3→I1, I2

I1, I2→I3

I1, I3→I2

I2, I3→I1

The next will be to calculate the confidence for each rule as shown below.

I1→I2, I3; Confidence = S(I1∩I2∩I3) / S(I1) = 2/6 = 0.3

I2→I1, I3; Confidence = S(I1∩I2∩I3) / S(I2) = 2/7 = 0.28

I3→I1, I2; Confidence = S(I1∩I2∩I3) / S(I3) = 2/5 = 0.4

I1, I2→I3; Confidence = S(I1∩I2∩I3) / S(I1∩I2) = 2/4 = 0.5

I1, I3→I2; Confidence = S(I1∩I2∩I3) / S(I1∩I3) = 2/4 = 0.5

I2, I3→I1; Confidence = S(I1∩I2∩I3) / S(I2∩I3) = 2/4 = 0.5

Since, the minimum threshold is 70%, there are no rules that qualify from the frequent itemset {1, 2, 3}.

Now, let us apply this rule to second frequent 3-itemset (I1, I2, I5). For this frequent itemset, non-empty subsets are {I1}, {I2}, {I5}, {(I1, I2)}, {(I1, I5)}, {(I2, I5)}.

For every non-empty set the rule will be generated as follows:

I1→I2, I5 (Here, (I1) is *s* and I2 and I5 are *l-s*]

I2→I1, I5

I5→I1, I2

I1, I2→I5

I1, I5→I2

I2, I5→I1

The next step will be to calculate the confidence for each rule as shown below.

I1→I2, I5; Confidence = S(I1∩I2∩I5) / S(I1) = 2/6 = 0.3

I2→I1, I5; Confidence = S(I1∩I2∩I5) / S(I2) = 2/7 = 0.28

I5→I1, I2; Confidence = S(I1∩I2∩I5) / S(I5) = 2/2 = 1

I1, I2→I5; Confidence = S(I1∩I2∩I5) / S(I1∩I2) = 2/4 = 0.5

I1, I5→I2; Confidence = S(I1∩I2∩I5) / S(I1∩I5) = 2/2 = 1

I2, I5→I1; Confidence = S(I1∩I2∩I5) / S(I2∩I5) = 2/2 = 1

Now, there are three rules whose confidence is more than minimum threshold value of 70%, and these rules are as follows:

I5→I1, I2

I1, I5→I2

I2, I5→I1

The possible rules from I5→I1, I2

It is intuitive to create two new rules as I5→I1 and I5→I2 from the given rule.

Since, I5→I1, I2 has confidence more than threshold limit, so it is implicit that I5→I1 and I5→I2 will also have their confidence more than the threshold limit as justified below.

Confidence of I5→I1,I2 = S(I5∩I1∩I2) / S(I5)

Confidence of I5→I1 = S(I5∩I1) / S(I5)

Since, Support of subset (I5, I1) will be more than or equal to Support of superset (I5, I1, I2), it is implicit that Confidence of I5→I1 will be more than equal to Confidence of I5→I1, I2.

Similarly, Confidence of I5→I2 = S(I5∩I2) / S(I5) will be more than equal to Confidence of I5→I1,I2.

Thus, I5→I1 and I5→I2 will also have their confidence more than the threshold limit.

Discussing possible rules from I2, I5→I1

It is intuitive to create two new rules as I2→I1 and I5→I1 from the given rule. I2, I5→I1 has confidence more than the threshold limit, but it is not implicit or guaranteed that I2→I1 and I5→I1 will have their confidence more than the threshold limit as explained below.

Confidence of I2, I5→I1 = S(I2∩I5∩I1) / S(I2∩I5)

Confidence of I2→I1 = S(I2∩I1) / S(I2)

Here, there is no correlation between denominator and quotient of both the rules, so it is not implicit that this rule will also have its confidence more than the threshold.

Similarly, the Confidence of I5→I1 = S(I5∩I1) / S(I5)

Again, there is no correlation between denominator and quotient of this rule with I2, I5→I1 rules, so it is not implicit that this rule will also have its confidence more than the threshold.

The given high confidence rule I2, I5→I1 does not implicitly state that I2→I1 and I5→I1 will have their confidence more than the threshold limit. Thus, there is a need to calculate the confidence of these rules.

Similarly, the given high confidence rule I1, I5→I2 does not implicitly state that I1→I2 and I5→I2 will have their confidence more than the threshold limit. Thus, again there is a need to calculate the confidence of these rules.

The process of generation of all association rules from I5→I1, I2; I2, I5→I1 and I1, I5→I2 rules has been illustrated in Table 9.47.

Table 9.47 Generation of association rules

Association Rule	*Discussion*	*Confidence*	*More than threshold limit Or Not*
I5→I1, I2	Already identified	1.0	Yes
I5→I1	Implicit	No need to calculate it will be more than or equal to I5→I1, I2	Yes
I5→I2	Implicit	No need to calculate it will be more than or equal to I5→I1, I2	Yes

Contd.

Association Rule	*Discussion*	*Confidence*	*More than threshold limit Or Not*
I2, I5→I1	Already identified	1.0	Yes
I2→I1	Not found earlier, confidence need to be calculated	Confidence of I2→I1 = S(I2∩I1) / S(I1) = 4/6 = 67%	No
I5→I1	Already found from I5→I1, I2 given in row 2	No need to calculate it will be more than or equal to I5→I1, I2	Yes (Already listed)
I1, I5→I2	Already identified	1.0	Yes
I1→I2	Not found earlier, so confidence needs to be calculated	Confidence of I1→I2 = S(I2∩I1) / S(I2) = 4/7 = 57%	No
I5→I2	Already found from I5→I1, I2 given in row 3		Yes (Already listed)

During generation of the final rules, it is important to look at L2 because there may be some frequent 2-itemsets that have not appeared in three itemsets and may be left out. The frequent 2 itemsets generated earlier have been reproduced in Table 9.48 from Figure 9.10.

Table 9.48 Frequent 2-itemsets, i.e., L2

L2	*Discussion*	*New Rules Identified*
I1, I2	This item pair has already been covered as it is a sub set of I5→I1, I2; I2, I5→I1 and I1, I5→I2 rules discussed in Table 9.47.	NIL
I1, I3	This item pair has not already been covered. So, confidence needs to be calculated for possible rules of this item pair I1, I3. I1→I3, Confidence = S(I1∩I3) / S(I1) = 4/6 = 66% I3→I1, Confidence = S(I3∩I1) / S(I3) = 4/5 = 80%	Since, I3→I1 has confidence more than the threshold limit. So, the new rule identified is I3→I1.
I1, I5	This item pair has already been covered as it is a sub set of I5→I1, I2; I2, I5→I1 and I1, I5→I2 rules.	NIL
I2, I3	This item pair has not already been covered. So, confidence needs to be calculated for possible rules of this item pair I2, I3. I2→I3, Confidence = S(I2∩I3) / S(I2) = 4/7 = 57% I3→I2, Confidence = S(I2∩I3) / S(I3) = 4/5 = 80%	I3→I2
I2, I4	This item pair has not already been covered. So, confidence needs to be calculated for possible rules of this item pair I2, I4. I2→I4, Confidence = S(I2 I4) / S(I2) = 2/7 = 28% I4→I2, Confidence = S(I4 I2) / S(I4) = 2/3 = 66%	NIL, because the confidence of both the rules is less than threshold limit; so, no new rule has been found.
I2, I5	This item pair has already been covered as it is a sub set of I5→I1, I2; I2, I5→I1 and I1, I5→I2 rules.	NIL

Here, we have identified two new rules by considering L2 and these are I3→I1 and I3→I2, both of which has its confidence more than threshold limit of 70%.

The final association rules having support more than 20% and confidence more than 70% are given below.

I5→I1, I2
I2, I5→I1
I1, I5→I2
I5→I1
I5→I2
I3→I1
I3→I2.

Example: Consider a store having 16 items for sale as listed in Table 9.49. Now consider the 25 transactions on the sale of these items as given in Table 9.50. As usual, each row in the table represents one transaction, that is, the items bought by one customer. In this example, we want to find association rules that satisfy the requirement of 25% support and 70% confidence.

Table 9.49 List of grocery items

Item number	Item name
1	Biscuit
2	Bournvita
3	Bread
4	Butter
5	Coffee
6	Cornflakes
7	Chocolate
8	Curd
9	Eggs
10	Jam
11	Juice
12	Milk
13	Rice
14	Soap
15	Sugar
16	Tape

In Table 9.50 showing 25 transactions, we list the names of items of interest but it would be more storage efficient to use item numbers.

Table 9.50 Transaction data

TID	Items
1	Biscuit, Bournvita, Butter, Cornflakes, Tape
2	Bournvita, Bread, Butter, Cornflakes
3	Butter, Coffee, Chocolate, Eggs, Jam
4	Bournvita, Butter, Cornflakes, Bread, Eggs
5	Bournvita, Bread, Coffee, Chocolate, Eggs
6	Jam, Sugar
7	Biscuit, Bournvita, Butter, Cornflakes, Jam
8	Curd, Jam, Sugar
9	Bournvita, Bread, Butter, Coffee, Cornflakes
10	Bournvita, Bread, Coffee, Chocolate, Eggs
11	Bournvita, Butter, Eggs
12	Bournvita, Butter, Cornflakes, Chocolate, Eggs
13	Biscuit, Bournvita, Bread
14	Bread, Butter, Coffee, Chocolate, Eggs
15	Coffee, Cornflakes
16	Chocolate
17	Chocolate, Curd, Eggs
18	Biscuit, Bournvita, Butter, Cornflakes
19	Bournvita, Bread, Coffee, Chocolate, Eggs
20	Butter, Coffee, Chocolate, Eggs
21	Jam, Sugar, Tape
22	Bournvita, Bread, Butter, Cornflakes
23	Coffee, Chocolate, Eggs, Jam, Juice
24	Juice, Milk, Rice
25	Rice, Soap, Sugar

Computing C1

All 16 items are candidate items for a frequent single-itemset. So, all the items listed in Table 9.49 will act as C1.

Computing L1

To find the frequent single itemset, we have to scan the whole database to count the number of times each of the 16 items has been sold. Scanning transaction database only once, we first set up the list of items and one transaction at a time, update the count of every item that appears in that transaction as shown in Table 9.51.

Table 9.51 Frequency count for all items

Item no.	*Item name*	*Frequency*
1	Biscuit	4
2	Bournvita	13
3	Bread	9
4	Butter	12
5	Coffee	9
6	Cornflakes	9
7	Chocolate	10
8	Curd	2
9	Eggs	11
10	Jam	6
11	Juice	2
12	Milk	1
13	Rice	2
14	Soap	1
15	Sugar	4
16	Tape	2

The items that have the necessary support (25% support in 25 transactions) must occur in at least 7 transactions. The frequent 1-itemset or L1 is now given in Table 9.52.

Table 9.52 The frequent 1-itemset or L1

Item	*Frequency*
Bournvita	13
Bread	9
Butter	12

Contd.

Item	*Frequency*
Coffee	9
Cornflakes	9
Chocolate	10
Eggs	11

Computing C2

C2 = L1 JOIN L1

This will produce 21 pairs as listed in Table 9.53. These 21 pairs are the only ones worth investigating since they are the only ones that could be frequent.

Table 9.53 The 21 candidate 2-itemsets or C2

{Bournvita, Bread}
{Bournvita, Butter}
{Bournvita, Coffee}
{Bournvita, Cornflakes}
{Bournvita, Chocolate}
{Bournvita, Eggs}
{Bread, Butter}
{Bread, Coffee}
{Bread, Cornflakes}
{Bread, Chocolate}
{Bread, Eggs}
{Butter, Coffee}
{Butter, Cornflakes}
{Butter, Chocolate}
{Butter, Eggs}
{Coffee, Cornflakes}
{Coffee, Chocolate}
{Coffee, Eggs}
{Cornflakes, Chocolate}
{Cornflakes, Eggs}
{Chocolate, Eggs}

Computing L2

This step is the most resource intensive step. We scan the transaction database once and look at each transaction and find out which of the candidate pairs occur in that transaction. This then requires scanning the list C2 for every pair of items that appears in the transaction that is being scanned. Table 9.54 presents the number of times each candidate 2-itemset appears in the transaction database example given in Table 9.50.

Table 9.54 Frequency count of candidate 2-itemsets

Candidate 2-itemset	*Frequency*
{Bournvita, Bread}	8
{Bournvita, Butter}	9
{Bournvita, Coffee}	4
{Bournvita, Cornflakes}	8
{Bournvita, Chocolate}	4
{Bournvita, Eggs}	6
{Bread, Butter}	5
{Bread, Coffee}	5
{Bread, Cornflakes}	3
{Bread, Chocolate}	4
{Bread, Eggs}	5
{Butter, Coffee}	4
{Butter, Cornflakes}	8
{Butter, Chocolate}	4
{Butter, Eggs}	6
{Coffee, Cornflakes}	2
{Coffee, Chocolate}	7
{Coffee, Eggs}	7
{Cornflakes, Chocolate}	1
{Cornflakes, Eggs}	2
{Chocolate, Eggs}	9

From this list we can find the frequent 2-itemsets or the set L2 by selecting only those item pairs from Table 9.54 given above, having a frequency equal to or more than 7. Therefore the list of frequent-2 itemsets or L2 is given in Table 9.55. Usually L2 is smaller than L1 but in this example they both have seven itemsets and each member of L1 appears as a subset of some member of L2. This is unlikely to be true when the number of items is large.

Table 9.55 The frequent 2-itemsets or L2

Frequent 2-itemset	Frequency
{Bournvita, Bread}	8
{Bournvita, Butter}	9
{Bournvita, Cornflakes}	8
{Butter, Cornflakes}	8
{Coffee, Chocolate}	7
{Coffee, Eggs}	7
{Chocolate, Eggs}	9

Computing C3

C3 = L2 JOIN L2 (First item should be common)

To find the candidate 3-itemsets or C3, we combine the appropriate frequent 2-itemsets from L2 (these must have the same first item) and obtain four such itemsets as given in Table 9.56.

Table 9.56 Candidate 3-itemsets or C3

Candidate 3-itemset
{Bournvita, Bread, Butter}
{Bournvita, Bread, Cornflakes}
{Bournvita, Butter, Cornflakes}
{Coffee, Chocolate, Eggs}

Pruning C3

Before processing further to calculate the frequency of each 3-item pairs, it is important to apply the Apriori property to prune the candidate 3-itemsets as shown in Table 9.57.

Table 9.57 Pruning of candidate itemset C3

Candidate Item Pair	*Discussion*	*Qualify or Not*
{Bournvita, Bread, Butter}	Its subsets are (Bournvita, Bread), (Bournvita, Butter) and (Bread, Butter). Out of these subsets (Bread, Butter) is not frequent as it is not in L2 given in Table 9.55. Thus, {Bournvita, Bread, Butter} will be removed from C3.	Not Qualified, so will be pruned
{Bournvita, Bread, Cornflakes}	Its subsets are (Bournvita, Bread), (Bournvita, Cornflakes) and (Bread, Cornflakes). Out of these subsets, (Bread, Cornflakes) is not frequent as it is not in L2. Thus, {Bournvita, Bread, Cornflakes} will be removed from C3.	Not Qualified, so will be pruned
{Bournvita, Butter, Cornflakes}	Its subsets are (Bournvita, Butter), (Bournvita, Cornflakes) and (Butter, Cornflakes). Here all subsets are frequent as they are in L2. Thus, {Bournvita, Butter, Cornflakes} will be retained in C3.	Qualified
{Coffee, Chocolate, Eggs}	Its subsets are (Coffee, Chocolate), (Coffee, Eggs) and (Chocolate, Eggs). Here, all subsets are frequent as they are in L2. Thus, {Coffee, Chocolate, Eggs} will be retained in C3.	Qualified

Thus, pruned C3 will be as shown in Table 9.58.

Table 9.58 Pruned candidate itemset C3

{Bournvita, Butter, Cornflakes}
{Coffee, Chocolate, Eggs}

As observed, this pruning step has reduced the candidate items from 4 to 2 and it improves the performance of the algorithm. Table 9.59 shows candidate 3-itemsets or C3 and their frequencies.

Table 9.59 Candidate 3-itemsets or C3 and their frequencies

Candidate 3-itemset	*Frequency*
{Bournvita, Butter, Cornflakes}	8
{Coffee, Chocolate, Eggs}	7

Computing L3

Table 9.60 now presents the frequent 3-itemsets or L3. Note that only one member of L2 is not a subset of any itemset in L3. That 2-itemset is {Bournvita, Bread}.

Table 9.60 The frequent 3-itemsets or L3

Frequent 3-itemset	*Frequency*
{Bournvita, Butter, Cornflakes}	8
{Coffee, Chocolate, Eggs}	7

Computing C4

C4 = L3 JOIN L3 (First two items should be common)

Since, no first two items are common, thus, C4 is null.

This is the end of frequent itemset generation since there cannot be any candidate 4-itemsets.

Finding the rules

The rules obtained from frequent 3-itemsets would always include some rules that we could obtain from L2 embedded in them, but some additional rules will be obtainable from itemsets in L2 which are not subsets of itemsets in L3, for example, {Bournvita, Bread} in this case.

Here, the frequent itemset is {Bournvita, Butter, Cornflakes} whose resulting subsets are as followed:

{Bournvita},{Butter},{Cornf lakes},{Bournvita, Butter},{Butter, Cornf lakes},{Bournvita, Cornflakes} [represented by *s*, and the superset or whole list {Bournvita, Butter, Cornflakes} is represented as *l*, and the rules will be generated in the form of *s*→*l*-*s*].

The rules generated from {Bournvita, Butter, Cornflakes} are as follows.

Bournvita→Butter, Cornflakes

Butter→Bournvita, Cornflakes

Cornflakes→Bournvita, Butter

Bournvita, Butter→Cornflakes

Bournvita, Cornflakes→Butter

Butter, Cornflakes→Bournvita

Now, let us calculate the confidence of these rules as shown in Table 9.61.

Table 9.61 Confidence of association rules from {Bournvita, Butter, Cornflakes}

Rule	*Support of Both together*	*Frequency of Antecedent LHS (Refer to Table 9.52 and 9.55)*	*Confidence*	*Whether Confidence is more than threshold limit or not*
Bournvita→Butter, Cornflakes	8	13	8/13 = 0.61	No
Butter→Bournvita, Cornflakes	8	12	8/12 = 0.67	No
Cornflakes→Bournvita, Butter	8	9	8/9 = 0.89	Yes
Bournvita, Butter→Cornflakes	8	9	8/9 = 0.89	Yes
Bournvita, Cornflakes→Butter	8	8	8/8 = 1.0	Yes
Butter, Cornflakes→Bournvita	8	8	8/8 = 1.0	Yes

We also have two rules from L2 for a frequent item pair {Bournvita, Bread} let us calculate its confidence as shown in Table 9.62.

Table 9.62 Confidence of association rules from {Bournvita, Bread}

Rule	*Support of Both together*	*Frequency of Antecedent LHS (Refer to Tables 9.52 and 9.55)*	*Confidence*	*Whether Confidence is more than threshold limit or not*
Bournvita →Bread	8	13	8/13 = 0.61	No
Bread→Bournvita	8	9	8/9 = 0.89	Yes

Thus, from this discussion we have found out five rules having confidence more than 70% as shown below in Table 9.63.

Table 9.63 Identified rules from {Bournvita, Butter, Cornflakes} having confidence more than 70%

Cornflakes→Bournvita, Butter
Bournvita, Butter→Cornflakes
Bournvita, Cornflakes→Butter
Butter, Cornflakes→Bournvita
Bread→Bournvita

Now, let us find all possible rules from these identified rules as shown in Table 9.64.

Table 9.64 List of all possible rules from rules given in Table 9.61

Rules	*Discussion*	*Rules having confidence more than the threshold value*
Cornflakes→ Bournvita, Butter	It has confidence 0.89 as discussed in Table 9.61. [Formula = S (Cornflakes, Bournvita, Butter) / S(Cornflakes)]	Cornflakes→ Bournvita, Butter
Cornflakes→ Bournvita	It is intuitive that this rule will have confidence greater than the threshold limit as its formula is S (Cornflakes, Bournvita) / S(Cornflakes). Since, support of subset, i.e, Cornflakes, Bournvita, should be equal to or more than superset, Cornflakes, Bournvita, Butter so this rule qualifies. (Resulting confidence = 0.89)	Cornflakes→ Bournvita

Contd.

Rules	*Discussion*	*Rules having confidence more than the threshold value*
Cornflakes→Butter	It is also intuitive that this rule will have confidence greater than the threshold limit as its formula is S (Cornflakes, Butter) / S(Cornflakes). Since, support of subset, i.e, Cornflakes, Butter should be equal to or more than superset, Cornflakes, Bournvita, Butter so this rule qualifies. (Resulting confidence = 0.89)	Cornflakes→ Butter
Bournvita, Butter→ Cornflakes	It has confidence 0.89 as shown in Table 9.61. [Formula = S (Bournvita, Butter, Cornflakes) / S(Bournvita, Butter)]	Bournvita, Butter→ Cornflakes
Bournvita→ Cornflakes	It is not intuitive that this rule will have confidence more than the threshold limit as its formula is S (Bournvita, Cornflakes) / S(Bournvita). So, let us calculate its confidence. Confidence= S (Bournvita, Cornflakes) / S(Bournvita) = 8/13 = 0.61. This rule has confidence less than threshold limit, so it will be discarded.	Discarded
Butter→ Cornflakes	It is not intuitive that this rule will have confidence more than the threshold limit as its formula is S (Butter, Cornflakes)/S(Butter). So, let us calculate its confidence. Confidence = S (Butter, Cornflakes) / S(Butter) = 8/12 = 0.67 This rule has confidence less than threshold limit, so it will be discarded.	Discarded
Bournvita, Cornflakes→Butter	It has confidence 1.0 as shown in Table 9.61.	Bournvita, Cornflakes→ Butter
Bournvita→Butter	It is not intuitive that this rule will have confidence more than the threshold limit. So, let us calculate its confidence. Confidence = S (Bournvita, Butter) / S(Bournvita) = 9/13 = 0.69 This rule has confidence less than threshold limit, so it will be discarded.	Discarded
Cornflakes→Butter	It is not intuitive that this rule will have confidence more than the threshold limit. So, let us calculate its confidence. Confidence = S (Cornflakes, Butter) / S(Cornflakes) = 8/9 = 0.88 This rule has confidence more than threshold limit, so it will be selected.	Cornflakes→ Butter

Contd.

Rules	*Discussion*	*Rules having confidence more than the threshold value*
Butter, Cornflakes→ Bournvita	It has confidence 1.0 as shown in Table 9.61.	Butter, Cornflakes→ Bournvita
Butter→ Bournvita	It is not intuitive that this rule will have confidence more than the threshold limit. So, let us calculate its confidence. Confidence = S (Butter, Bournvita) / S(Butter) = 8/12 = 0.67 This rule has confidence less than threshold limit, so it will be discarded.	Discarded
Cornflakes→ Bournvita	It is not intuitive that this rule will have confidence more than the threshold limit. So, let us calculate its confidence. Confidence = S (Cornflakes, Bournvita) / S(Cornflakes) = 8/9 = 0.89 This rule has confidence more than threshold limit, so it will be selected.	Cornflakes→ Bournvita

Similarly, we will examine the other 3-itemset {Coffee, Chocolate, Eggs}.

Table 9.65 Confidence of association rules from {Coffee, Chocolate, Eggs}

Rule	*Support*	*Frequency of LHS (Antecedent)*	*Confidence*
Coffee→Chocolate, Eggs	7	9	0.78
Chocolate→Coffee, Eggs	7	10	0.70
Eggs→Coffee, Chocolate	7	11	0.64
Chocolate, Eggs→Coffee	7	9	0.78
Coffee, Eggs→Chocolate	7	7	1.0
Coffee, Chocolate→Eggs	7	7	1.0

Again, the confidence of all these rules, except the third, is higher than 70%, and so five of them also qualify.

List of all possible rules for three item pairs {Coffee, Chocolate, Eggs} has been explained in Table 9.66, given below.

Table 9.66 List of all possible rules from rules given in Table 9.65

Rules	*Discussion*	*Rules having confidence more than the threshold value*
Coffee→Chocolate, Eggs	It has confidence of 0.78 as shown in Table 9.65.	Coffee→Chocolate, Eggs
Coffee→Eggs	It is intuitive from rule Coffee→Chocolate, Eggs	Coffee→Eggs
Chocolate→Coffee, Eggs	It has confidence of 0.70 as shown in Table 9.65.	Chocolate→Coffee, Eggs
Chocolate→Coffee	It is intuitive from rule Chocolate→Coffee, Eggs	Chocolate→Coffee
Chocolate→Eggs	It is intuitive from rule Chocolate→Coffee, Eggs	Chocolate→Eggs
Chocolate, Eggs→Coffee	It has confidence of 0.78 as shown in Table 9.65.	Chocolate, Eggs→Coffee
Chocolate→Coffee	It is non intuitive, so let us calculate its confidence. Confidence = S(Chocolate, Coffee) / S(Coffee) = 7/9 = 0.77 Since, confidence is more than the threshold so this rule will be selected.	Chocolate→Coffee
Eggs→Coffee	It is non intuitive, so let us calculate its confidence. Confidence = S(Eggs, Coffee) / S(Eggs) = 7/11 = 0.64 Since, confidence is less than the threshold so this rule will be discarded.	Discarded
Coffee, Eggs→Chocolate	It has confidence 1.0 as discussed in Table 9.65.	Coffee, Eggs→Chocolate
Coffee→Chocolate	It is non intuitive, so let us calculate its confidence. Confidence = S(Coffee, Chocolate) / S(Coffee) =7/9 = 0.77 Since, confidence is more than the threshold so this rule will be selected.	Coffee→Chocolate
Eggs→Chocolate	It is non intuitive, so let us calculate its confidence. Confidence = S(Eggs, Chocolate) / S(Eggs) = 9/11= 0.81 Since, confidence is more than the threshold so this rule will be selected.	Eggs→Chocolate

Contd.

Rules	*Discussion*	*Rules having confidence more than the threshold value*
Coffee, Chocolate→Eggs	It has confidence 1.0 as shown in Table 9.65.	Coffee, Chocolate→Eggs
Coffee→Eggs	This rule is non intuitive, but it is already selected as given in row 2 from rule Coffee→Chocolate, Eggs	Coffee→Eggs
Chocolate→Eggs	This rule is non intuitive, but it is already selected as given in row 5 from rule Chocolate→Coffee, Eggs	Chocolate→Eggs

Thus, all association mining rules for the given database and having confidence of more than 70% are listed in Table 9.67.

Table 9.67 All association rules for the given database

Rule No.	*Rule*	*Confidence*
1	Bread→Bournvita	0.89
2	Cornflakes→ Bournvita, Butter	0.89
3	Cornflakes→ Bournvita	0.89
4	Cornflakes→ Butter	0.89
5	Bournvita, Butter→ Cornflakes	0.89
6	Bournvita, Cornflakes→ Butter	1
7	Cornflakes→ Butter	0.89
8	Butter, Cornflakes→ Bournvita	1
9	Coffee→Chocolate, Eggs	0.78
10	Coffee→Chocolate	0.78
11	Coffee→Eggs	0.78
12	Chocolate→Coffee, Eggs	0.7
13	Chocolate→Coffee	0.7
14	Chocolate→Eggs	0.9
15	Chocolate, Eggs→Coffee	1
16	Coffee, Eggs→Chocolate	1
17	Eggs→Chocolate	0.82
18	Coffee, Chocolate→Eggs	1

9.8 Closed and Maximal Itemsets

To further improve the efficiency of the association mining process, frequent itemsets can be divided into closed and maximal itemsets.

A frequent closed itemset is a frequent itemset X such that there exists no superset of X with the same support count as X.

A frequent itemset Y is maximal if it is not a proper subset of any other frequent itemset. Therefore a maximal itemset is a closed itemset, but a closed itemset is not necessarily a maximal itemset as shown in Figure 9.13.

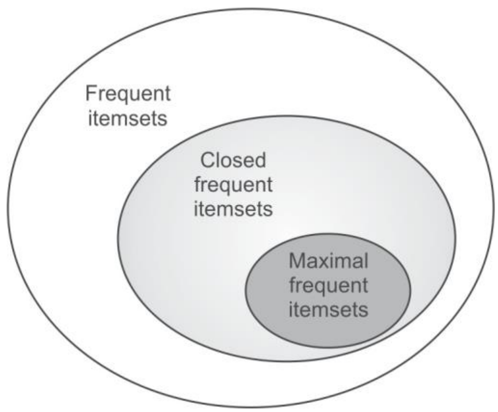


Figure 9.13 Illustration of closed and maximal frequent itemsets

The concept of Closed and maximal itemsets have been explained with an example database.

Example: Consider the transaction database in Table 9.68 with minimum support required to be 40%.

Table 9.68 A transaction database to illustrate closed and maximal itemsets

100	Butter, Curd, Jam
200	Butter, Curd, Jam, Muffin
300	Curd, Jam, Nuts
400	Butter, Jam, Muffin, Nuts
500	Muffin, Nuts

There are a total of 14 frequent itemsets as shown in Table 9.69 and the closed and maximal itemsets for the given database have also been presented there.

Table 9.69 Frequent itemsets for the database in Table 9.68

Itemset	*Support*	*Closed?*	*Maximal?*	*Both?*
{Butter}	3	No, there exists a superset of Butter with the same support 3, i.e., {Butter, Jam}	No, it is a proper subset of another frequent itemset, i.e, {Butter, Curd, Jam}	No
{Curd}	3	No, there exists a superset of Curd with the same support 3, {Curd, Jam}	No, it is a proper subset of another frequent itemset, i.e, {Butter, Curd, Jam}	No
{Jam}	4	Yes, there exists no superset of Jam with the same support 4	No, it is a proper subset of another frequent itemset, i.e, {Butter, Curd, Jam}	No
{Muffin}	3	Yes, there exists no superset of Muffin with the same support 3	No, it is a proper subset of another frequent itemset, i.e, {Muffin, Nuts}	No
{Nuts}	3	Yes, there exists no superset of Nuts with the same support 3	No, it is a proper subset of another frequent itemset, i.e, {Muffin, Nuts}	No
{Butter, Curd}	2	No, there exists a superset of Butter, Jam with the same support 2, i.e., {Butter, Curd, Jam}	No, it is a proper subset of another frequent itemset, i.e, {Butter, Curd, Jam}	No
{Butter, Jam}	3	Yes, there exists no superset of Butter, Jam with the same support 3	No, it is a proper subset of another frequent itemset, i.e, {Butter, Curd, Jam}	No
{Butter, Muffin}	2	No, there exists a superset of Butter, Muffin with the same support 2, i.e., {Butter, Jam, Muffin}	No, it is a proper subset of another frequent itemset, i.e, {Butter, Jam, Muffin}	No
{Curd, Jam}	3	Yes, there exists no superset of Curd, Jam with the same support 3	No, it is a proper subset of another frequent itemset, i.e, {Butter, Curd, Jam}	No
{Jam, Muffin}	2	No, there exists a superset of Jam, Muffin with the same support 2, i.e., {Butter, Jam, Muffin}	No, it is a proper subset of another frequent itemset, i.e, {Butter, Jam, Muffin}	No
{Jam, Nuts}	2	Yes, there exists no superset of Jam, Nuts with the same support 2	Yes, it is not proper subset of any other frequent itemset	Yes

Contd.

Itemset	Support	Closed?	Maximal?	Both?
{Muffin, Nuts}	2	Yes, there exists no superset of Muffin, Nuts with the same support 2	Yes, it is not proper subset of any other frequent itemset	Yes
{Butter, Curd, Jam}	2	Yes, there exists no superset of Butter, Curd, Jam with the same support 2	Yes, it is not proper subset of any other frequent itemset	Yes
{Butter, Jam, Muffin}	2	Yes, there exists no superset of Butter, Jam, Muffin with the same support 2	Yes, it is not proper subset of any other frequent itemset	Yes

One can observe that there are 9 closed frequent itemsets and 4 maximal frequent itemsets as shown in Table 9.69.

It is important to note that closed and maximal frequent itemsets are often smaller than all frequent itemsets and association rules can be generated directly from them. Thus, it is desirable that instead of mining every frequent itemset, an efficient algorithm should mine only maximal frequent itemsets. The association of frequent itemsets, closed frequent itemsets and maximal frequent itemsets has been illustrated in Figure 9.16 for better understanding. From this one can observe that a maximal itemset is a closed itemset, but a closed itemset is not necessarily a maximal itemset.

9.9 The Apriori-TID Algorithm for Generating Association Mining Rules

Apriori–TID uses the Apriori algorithm to generate the candidate and frequent itemsets, but the difference is that it uses the TID or vertical storage approach (discussed in Section 9.6.3) for its implementation.

The working of the Apriori–TID algorithm is given as follows.

1. Scan the whole transaction database to convert it into TID storage and represent it as T1 (i.e., each entry of T1 consists of all items in the transaction along with the corresponding TID).
2. Calculate the frequent 1-itemset L1 using T1.
3. Obtain C2 by applying the joining principle discussed under the Apriori algorithm.
4. Calculate the support for the candidates in C2 with the help of T1 by intersecting corresponding rows and a new TID dataset is created and represent it as T2 containing two item pairs.
5. Generate L2 from C2 by the usual means and then generate C3 from L2, again by using the joining principle.
6. Generate T3 using T2 and C3. This process is repeated until the set of candidate k-itemsets is an empty set.

Example of Apriori–TID: Let us understand this concept with an example database of transactions given in Table 9.70 for finding frequent itemsets with the Apriori–TID algorithm. Here, minimum support is 50% and minimum confidence is 75%.

Table 9.70 Transaction database

100	Butter, Curd, Eggs, Jam
200	Butter, Curd, Jam
300	Butter, Muffin, Nuts
400	Butter, Jam, Muffin
500	Curd, Jam, Muffin

Step 1:

Scan the entire transaction database to convert it into TID storage represented as T1 as given in Table 9.71.

Table 9.71 Transaction database T1

	100	*200*	*300*	*400*	*500*
Butter	1	1	1	1	0
Curd	1	1	0	0	1
Eggs	1	0	0	0	0
Jam	1	1	0	1	1
Muffin	0	0	1	1	1
Nut	0	0	1	0	0

Step 2:

Frequent 1-itemset L1 is calculated using T1.

Since, the minimum support is 3, all items have been selected as frequent items having support equal or more than threshold limit of support, i.e., 3 with the help of T1.

Thus, L1 is as shown in Table 9.72.

Table 9.72 L1

Itemset	Support
Butter	4
Curd	3
Jam	4
Muffin	3

Step 3:

C2 is obtained by applying the joining principle, i.e., L1 JOIN L1

Thus, C2 will be as given in Table 9.73.

Table 9.73 C2

Itemsets
Butter, Curd
Butter, Jam
Butter, Muffin
Curd, Jam
Curd, Muffin
Jam, Muffin

Step 4:

Support for the candidates in C2 is then calculated with the help of T1 by intersecting corresponding rows and a new TID dataset is created represented as T2 containing two item pairs as illustrated in Table 9.74.

Table 9.74 Transaction database T2

	100	200	300	400	500
Butter, Curd	1	1	0	0	0
Butter, Jam	1	1	0	1	0
Butter, Muffin	0	0	1	1	0
Curd, Jam	1	1	0	0	1
Curd, Muffin	0	0	0	0	1
Jam, Muffin	0	0	0	1	1

Thus, support for C2 is given in Table 9.75.

Table 9.75 Support for C2

Itemsets	*Support*
Butter, Curd	2
Butter, Jam	3
Butter, Muffin	2
Curd, Jam	3
Curd, Muffin	1
Jam, Muffin	2

Step 5:

L2 is then generated from C2 as shown in Table 9.76.

Table 9.76 L2

Itemsets	*Support*
Butter, Jam	3
Curd, Jam	3

Thus, {Butter, Jam} and {Curd, Jam} are the frequent pairs and they generate L2. C3 may also be generated but it is observed that C3 is empty because the first item is not common in the L2 item .pairs. If C3 was not empty then it would be used to identify T3 using the transaction set T2. That would result in a smaller T3 and it might have resulted in the removal of a transaction or two. In the example database, all the records of T2 have been removed and T3 is null.

The generation of association rules from the derived frequent set can be done in the usual way.

Advantages of the Apriori-TID algorithm

The advantage of Apriori–TID algorithm is in the storage structure because it facilitates counting of items by counting the number of 1s in each row and the number of 2-itemsets can be counted by finding the intersection of the two rows.

The other main advantage of the Apriori–TID algorithm is that the size of T_k is generally smaller than the transaction database.

Because the support for each candidate k-itemset is counted using the corresponding T_k, therefore this algorithm runs faster than the basic Apriori algorithm. It is important to note that both Apriori and Apriori–TID use the same candidate generation algorithm, and therefore they count the same itemsets.

9.10 Direct Hashing and Pruning (DHP)

A major shortcoming of the Apriori algorithm is that it cannot prune 2 itemset candidate items. As C2 is derived from L1 by using following formula:

C2 = L1 Join L1.

Since, every subset of C2 is a member of L1, C2 cannot be pruned by the Apriori property, i.e., subset of frequent itemsets should also be frequent. In case of C2, all the subsets are frequent, i.e., member of L1, so it cannot be pruned. However, Apriori property is very helpful in pruning higher order candidate itemsets like C3 or C4.

The DHP algorithm is useful as it overcomes this limitation since it has the capability to prune C2; this is important to improve performance.

Working of the DHP algorithm

This algorithm employs a hash-based technique for reducing the count of candidate itemsets generated in the first pass (i.e., a significantly smaller C2 is generated). The number of itemsets in C2 generated using DHP can be smaller in magnitude; therefore the scan to identify L2 is more efficient.

The steps to be taken to operate DHP are:

1. Find all frequent 1-itemsets and all the candidate 2-itemsets by using the same steps as the Apriori algorithm, i.e., C2 is generated by L1 JOIN L1.
2. Generate all possible 2-itemsets in each transaction and assign a code to each item and itemsets.
3. All the possible 2-itemsets are hashed to a hash table by using the code of each itemset. For hashing, a hash function is applied on each code and a bucket is assigned to each itemset based on the output of the hash function. The count of each bucket in the hash table is increased by one, each time when an itemset is hashed to that bucket. When different itemsets are hashed to the same bucket then collisions can occur. To assign a f lag for each bucket, a bit vector is associated with the hash table. If the bucket count is equal or above the minimum required support count, then accordingly flag in the bit vector is set to 1, otherwise it is set to 0.
4. In this step C2 is pruned. Each item pair of C2 generated in step 1 is checked for the bit vector of its corresponding bucket to which it was assigned in step 3. If its bit vector is 0, then the item pair will be pruned and removed from C2, while all those item pairs having a bit vector of 1 are retained in C2. This step helps to prune C2 and this pruning of C2 is the major objective of the DHP algorithm.
   It is important to note that, having the corresponding bit vector or bit set does not guarantee that the itemset is frequent; this is due to collisions. The hash table filtering does reduce C2, significantly.
5. Then, from the pruned C2, frequent item pairs, L2 is generated and process continues as in Aproiroi property.
6. In this step, the hash table for the next step is generated. Only those itemsets that are frequent, i.e. in L2, are kept in the database. The pruning not only trims each transaction by removing the unwanted itemsets but also removes transactions that have no itemsets that could be frequent.
7. The algorithm is continued till no more candidate itemsets are identified. The whole transaction database is scanned to identify the support count for each itemset despite using a hash table to identify the frequent itemsets. Now, the database seems relatively smaller because of the pruning. The algorithm identifies the frequent itemsets as earlier by checking against the minimum support when the support count is calculated. The algorithm then produces candidate itemsets similar to what the Apriori algorithm does.

To get a clear idea about the working of DHP algorithm, let us consider some case studies by way of examples. After going through these examples, this algorithm will become clear to you.

Example: Using the DHP algorithm

To understand this concept, let us identify the association rules satisfying 50% support and 75% confidence for the transaction database given in Table 9.77 with the DHP algorithm.

Table 9.77 Transaction database

100	Butter, Curd, Eggs, Jam
200	Butter, Curd, Jam
300	Butter, Muffin, Nuts
400	Butter, Jam, Muffin
500	Curd, Jam, Muffin

Let us first find all frequent 1-itemsets and in this case it will all those items that appear 3 or more times in the database. The L1 is given in Table 9.78.

Table 9.78 Frequent 1-itemset L1

Butter
Curd
Jam
Muffin

Then, generate C2 from L1, i.e., L1 JOIN L1. This is given in Table 9.78. Here, for simplicity we are representing each item by its first alphabet.

Table 9.79 Candidate 2 itemsets C2

BC ('B' for Butter and 'C' for Curd)
BJ
BM
CJ
CM
JM

Since, the size of C2 is 6 and the major advantage of DHP is to prune C2 so hashing is applied on each item pair as discussed below.

To apply hashing, the next step is to generate all possible 2-itemsets for each transaction as shown in Table 9.80.

Table 9.80 Possible 2-itemsets for each transaction

100	(Butter, Curd), (Butter, Eggs), (Butter, Jam), (Curd, Eggs), (Curd, Jam), (Eggs, Jam)
200	(Butter, Curd), (Butter, Jam), (Curd, Jam)
300	(Butter, Muffin), (Butter, Nuts), (Muffin, Nuts)
400	(Butter, Jam), (Butter, Muffin), (Jam, Muffin)
500	(Curd, Jam), (Curd, Muffin), (Jam, Muffin)

For a support of 50%, the frequent items are Butter, Curd, Jam, and Muffin as shown in Table 9.81. Now, to hash the possible 2-itemsets, a code is assigned to each item as given in Table 9.82.

Table 9.81 Frequent itemsets for a support of 50%

Bread
Curd
Jam
Muffin

Table 9.82 Code for each item

Code	*Item Name*
1	Butter
2	Curd
3	Eggs
4	Jam
5	Muffin
6	Nuts

For each pair, a numeric value is obtained by representing Butter by 1, Curd by 2 and so on as shown in the above table. Then, each pair is represented by a two-digit number, for example, (Butter, Curd) is represented by 12; (Curd, Jam) is represented by 24 and so on.

The coded value for each item pair is given in Table 9.83.

Table 9.83 Coded representation for each item pair

100	12, 13, 14, 23, 24, 34
200	12, 14, 24
300	15, 16, 56
400	14, 15, 45
500	24, 25, 45

Now, the hash function is applied on coded values so that these items can be assigned to different buckets. Here, we are using modulo 8 number, i.e., dividing the code by 8 and using the remainder as hash function for the given dataset.

Hash function: Modulo 8, i.e., divide the concerned item pair code by 8 and use the remainder as its bucket number.

The guidelines to decide about hash function have been discussed later. For the moment, let's apply hash function to each item pair and assign the item pairs to each bucket on the basis of modulo 8 as shown in Table 9.60.

Applying the hash function to each item pair in transaction 100, the first item pair is 12, and when modulo 8 hash function is applied on 12, it gives a remainder of 4, so 12 is assigned to bucket 4. The next item pair in transaction 100 is 13, and gives a remainder of 5 after dividing it by 8, so 13 is assigned to bucket 5. Similarly, 14 has a remainder of 6, 23 has a remainder of 7, 24 has the remainder 0 and 34 has a remainder of 2, so these item pairs are assigned corresponding bucket numbers as shown in Table 9.84. This process is repeated for each transaction and every item pair is assigned to its corresponding bucket.

Table 9.84 Assigning item pairs to buckets based on hash function modulo 8

0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*Bucket address*
24 24 16 56 24	25	34		12 12	13 45 45	14 14 14	23 15 15	Item pairs
5	1	1	0	2	3	3	3	Support of bucket
1	0	0	0	0	1	1	1	Bit Vector (it refers to a sequence of numbers which are either 0 or 1 and stored contiguously in memory)

After assigning every item pair to its corresponding bucket, the support or count for each bucket has been calculated and if its support is equal to the threshold value then the corresponding bit vector is set 1 as shown in Table 9.84.

Now, C2 is pruned by checking the bit vector of the corresponding bucket for each candidate item pair. And only those item pairs are retained in C2 which have the corresponding bit vector as 1. The process of pruning of C2 has been illustrated in Table 9.85.

It has been noted that pair BC, i.e, 12 belong to the bucket with support of 2, since its support is less than threshold value of support so its corresponding bit vector is 0 and this item pair has been removed from C2; in other words, C2 has been pruned.

Similarly, this analysis is performed for each item pair of C2 as shown in Table 9.85.

Table 9.85 Pruning of C2

Item Pairs in C2	*Bucket*	*Bit Vector*	*Remarks*	*Pruned C2*
BC, i.e., 12	4	0	Removed from C2	
BJ, i.e., 14	6	1	Will be retained	BJ
BM, i.e., 15	7	1	Will be retained	BM
CJ, i.e., 24	0	1	Will be retained	CJ
CM, i.e., 25	1	0	Removed from C2	
JM, i.e., 45	5	1	Will be retained	JM

These candidate pairs are then hashed to the hash table and those candidate pairs are removed that are hashed to locations where the bit vector bit is not set. Table 9.85 illustrates that (B, C) and (C, M) have been eliminated from C2. Now, only four candidate item pairs are left instead of six item pairs in C2 and thus, C2 has been pruned as given in the last column of Table 9.85. After that, analyze the transaction database and modify it to include only these candidate pairs as given in Table 9.86.

Now, from the pruned C2, let us find L2 as shown in Table 9.86.

Table 9.86 Finding L2

Itemset	*Count*
BJ	3
BM	2
CJ	3
JM	2

Thus, L2 has two item pairs, i.e, BJ and CJ. Now, let us use L2 by selecting only those item pairs of the transaction database that qualify by being above the threshold value of support. The method of finding three itemsets is shown in Table 9.87.

Table 9.87 Finding three itemsets

TID	*Original Item groups*	*Selected Item pairs according to L2*	*Three Item groupings (sets)*
100	(Butter, Curd), (Butter, Eggs), (Butter, Jam), (Curd, Eggs), (Curd, Jam), (Eggs, Jam)	(Butter, Jam), (Curd, Jam)	NIL
200	(Butter, Curd), (Butter, Jam), (Curd, Jam)	(Butter, Jam), (Curd, Jam)	NIL
300	(Butter, Muffin), (Butter, Nuts), (Muffin, Nuts)	NIL	NIL
400	(Butter, Jam), (Butter, Muffin), (Jam, Muffin)	(Butter, Jam)	NIL
500	(Curd, Jam), (Curd, Muffin), (Jam, Muffin)	(Curd, Jam)	NIL

As shown above in Table 9.87, there are no 3-Item groups, and this complete the process of identifying frequent item pairs .So, in this case, there are two item pairs (Butter, Jam) and (Curd, Jam).We will find out the rules for selected item pairs in the same way as we did in the Apriori algorithm, previously. Proceed as follow:

These item pairs produce four rules as shown below.

Butter → Jam
Jam → Butter
Curd → Jam
Jam → Curd

Now, let us calculate their confidence as shown below.

Confidence of Butter → Jam = S (Butter, Jam) / S(Butter) = 3/4 = 0.75

Jam → Butter = S (Jam, Butter) / S(Jam) = 3/4 = 0.75

Curd → Jam = S (Curd, Jam) / S(Curd) = 3/3 = 1.0

Jam → Curd = S (Jam, Curd) / S(Jam) = 3/4 =1.0

Based on given threshold value of confidence we can select the qualified rules.

It is important to note that, the DHP algorithm usually performs better than the Apriori algorithm during the initial stages, especially during L2 generation. Added to this improvement is also the feature that trims and prunes the transaction set progressively, thereby improving the overall efficiency of the algorithm. The startup phase of DHP does consume some extra resources when the hash table is constructed but this is outweighed by the advantages gained during the later phases when both the number of candidate itemsets and the transaction set are reduced (pruned better).

Guidelines for selection of the hash function

As one of the major objectives of the DHP algorithm is to reduce the size of C2, it is essential that the hash table is large enough so that the collisions are low, i.e., different item pairs should not be hashed into the same bucket. In case of collisions, the bit vector is not very effective because in this case more than one item pair is contributing to support for the bucket and in ideal case it should be the support of only one item pair that should contribute, so that candidate item pairs are close to frequent item pairs.

If the hash table is too small and the collisions are high then more than one itemset will hash to most of the buckets. It will result into loss of effectiveness of the hash table in reducing the number of candidate itemsets C2. This is what happened in the example above in which we had collisions in three of the eight buckets. If the minimum support count is low, it is even more important that all collisions should be avoided.

If the hash table is too large then calculation will increase as we have to calculate the values for each bucket. Thus, the hash function should be selected in such a manner that it results in the least collisions.

Increasing the efficiency of the DHP algorithm

For efficient working of DHP, it is best to ensure that the hash table resides in the main memory. If the hash table does not fit in the main memory, additional disk I/O costs will apply which may again reduce the efficiency. This algorithm is most efficient when the number of itemsets in L2 is lot smaller than C2. This reduction in database size will have a positive impact on the efficiency of the algorithm.

The efficient pruning of the database, particularly C2, helps DHP to achieve a significantly shorter execution time than the Apriori algorithm.

Some more examples of DHP and Apriori algorithms

Another example will help in comparing Apriori with the DHP algorithm. In this example, we will first apply the Apriori algorithm to identify frequent item pairs; then we will apply the DHP algorithm for the same identification to illustrate differences in the process. Here, we have to find frequent item pairs with support more than 40% for the database given in Table 9.88.

By applying both the algorithms on the same database, comparison of the workings of both algorithms will be easy.

Table 9.88 Transaction database for Apriori and DHP

TID	*Items*
100	A C D
200	B C E
300	A B C E
400	B E

The process to identify frequent item pairs through the Apriori algorithm has been illustrated in Figure 9.14.

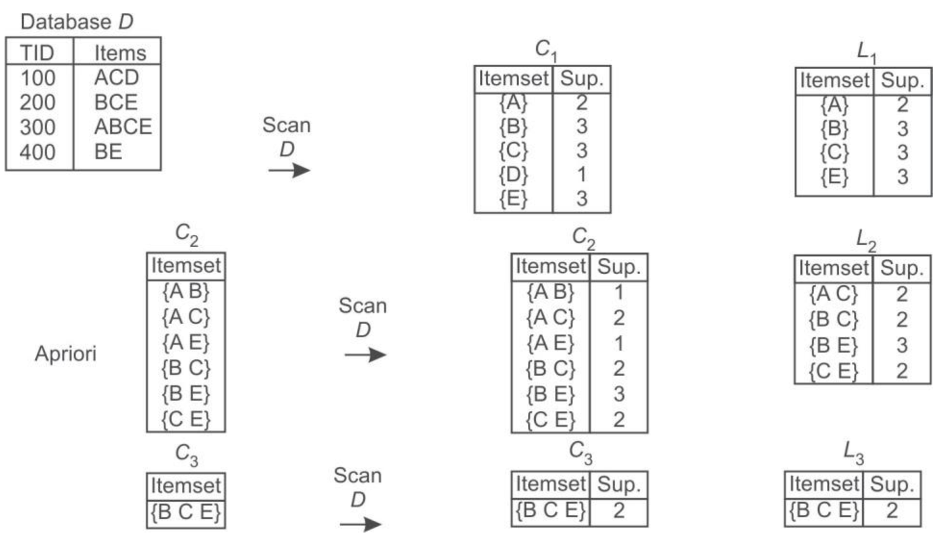


Figure 9.14 Process by the Aprioiri algorithm method

The process to identify frequent item pairs through DHP algorithm has been illustrated in Figure 9.15.

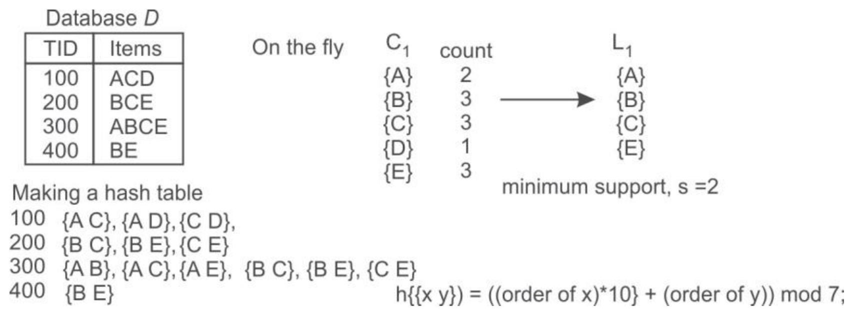


Figure 9.15 Process by the DHP algorithm method

Now, to hash the possible 2-itemsets, the code is assigned to each item. Let us assign code to each item as given in Table 9.89.

Table 9.89 Code for each item

Item	*Code*
A	1
B	2
C	3
D	4
E	5
F	6

A numeric value is attained for each pair by representing A by 1, B by 2 and so on as given in above table. Then each pair is represented by a two-digit number, for example, AC by 13 and AD by 14. The coded value for each item pair is given in Table 9.90.

Table 9.90 Coded representation for each item pair

TID	*Item pairs code*
100	13 (AC), 14(AD), 34 (CD)
200	23 (BC), 25 (BE), 35 (CE)
300	12 (AB), 13 (AC), 15 (AE), 23 (BC), 25 (BE), 35 (CE)
400	25 (BE)

Now, hash function is applied on the coded values so that these items can be assigned to different buckets. Here, we are using modulo 7 number, i.e., dividing by 7 and using the remainder as hash function for the given dataset as given in Table 9.91.

Table 9.91 Assigning of item pairs to buckets based on hash function modulo 7

0	1	2	3	4	5	6	Bucket address
14 35 35	15	23 23		25 25 25	12	13 34 13	Item pairs
3	1	2	0	3	1	3	Support of bucket
1	0	1	0	1	0	1	Bit Vector

The process of pruning C2 based on bit vector has been shown in Table 9.92.

Table 9.92 Pruning of C2

Item Pairs in C2	*Bucket*	*Bit Vector*	*Remarks*	*Pruned C2*
AB, i.e., 12	5	0	Removed from C2	
AC, i.e., 13	6	1	Will be retained	AC
AE, i.e., 15	1	0	Removed from C2	
BC, i.e., 23	2	1	Will be retained	BC
BE, i.e., 25	4	1	Will be retained	BE
CE, i.e., 35	0	1	Will be retained	CE

This pruned C2 is used to find L2 as shown in Table 9.93.

Table 9.93 Finding L2

Item pairs	*Count*
AC	2
BC	2
BE	3
CE	2

All these item pairs have support more than threshold limit. This L2 will be used for identifying three item pairs by selecting only those item pairs from Table 9.90 which are having support more than the threshold limit, as shown below in Table 9.94

Table 9.94 Identifying three itemsets

TID	*Original Item Pairs*	*Selected Item Pairs according to L2*	*Three Itemsets*
100	AC, AD, CD	AC	NIL
200	BC, BE, CE	BC, BE, CE	BCE
300	AB, AC, AE, BC, BE, CE	AC, BC, BE, CE	BCE
400	BE	BE	NIL

Here, BCE has a support of 2; thus, the frequent item group is BCE.

In next phase, association rules for three item group BCE can be found by using the same approach as discussed in the Apriori algorithm.

Example: Let us consider, the database given in Table 9.95 for identification of frequent item pairs with the Apriori algorithm and the DHP algorithm for minimum support of 50%.

Table 9.95 Transaction database

TID	*Item Bought*
100	1, 2, 3, 4
200	5, 6, 7, 8, 9
300	6, 8, 9, 10
400	2, 3, 9

The process to identify frequent item pairs through Apriori algorithm has been illustrated in Figure 9.16.

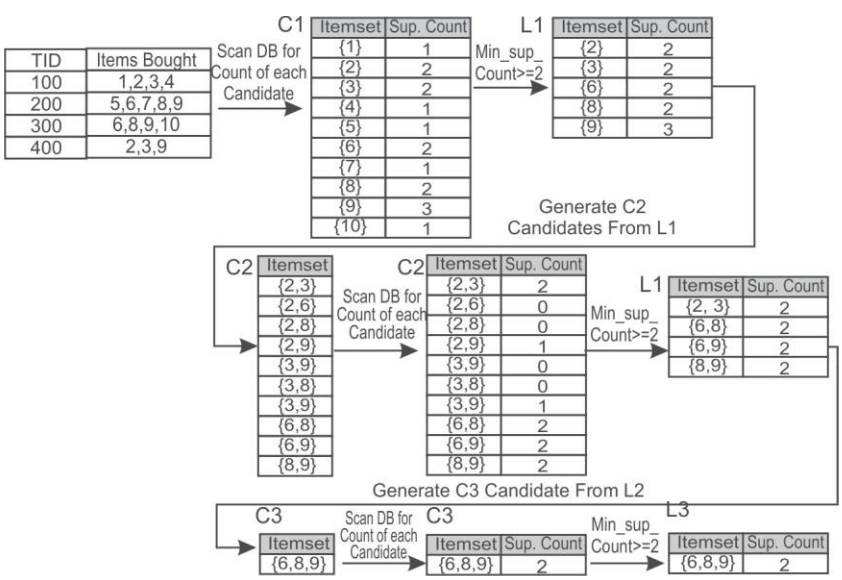


Figure 9.16 Identifying frequent item pairs and groups by Apriori algorithm

Thus, the process of the Apriori algorithm has produced a frequent three item group {6, 8, 9}.

The process to identify frequent item pairs and groups through the DHP algorithm has been illustrated in Table 9.96.

Table 9.96 Coded item pairs for DHP

TID	*Items bought*	*Item pairs code*
100	1,2,3,4	12,13,14,23,24,34
200	5,6,7,8,9	56,57,58,59,67,68,69,78,79,89
300	6,8,9,10	68,69,610,89,810,910
400	2,3,9	23,29,39

Now, the hash function is applied on coded values so that these items can be assigned to different buckets. Here, we are going to use Modulo 11 number, i.e., dividing by 11 and using the remainder as hash function to assign buckets for the given dataset.

Let's apply hash function to each item pair and assign the item pairs to each bucket on the basis of modulo 11 as shown in Table 9.97.

Table 9.97 Assigning of item pairs to buckets based on hash function modulo 11

0	1	2	3	4	5	6	7	8	9	10	Bucket address
	12 23 34 56 67 89 23	13 24 57 68 68	14 58 69 69	59	610	39	29 810	910			Item pairs
0	7	5	4	1	1	1	2	1	0	0	Support of bucket
0	1	1	1	0	0	0	1	0	0	0	Bit Vector

The pruning of C2 has been illustrated in Table 9.98.

Table 9.98 Pruning of C2

Item Pairs in C2	*Bucket*	*Bit Vector*	*Remarks*	*Pruned C2*
23	1	1	Will be retained	23
26	-	-	Removed from C2	
28	-	-	Removed from C2	
29	7	1	Will be retained	29
36	-	-	Removed from C2	

Contd.

Item Pairs in C2	*Bucket*	*Bit Vector*	*Remarks*	*Pruned C2*
38	-	-	Removed from C2	
39	6	0	Removed from C2	
68	2	1	Will be retained	68
69	3	1	Will be retained	69
89	1	1	Will be retained	89

This pruned C2 is used to find L2 as shown in Table 9.99.

Table 9.99 Finding L2

Item pair	*Count*
23	2
29	1
68	2
69	2
89	2

Thus, L2 is 23, 68, 69 and 89, which is used to find three item groups as shown in Table 9.100.

Table 9.100 Finding three itemsets

TID	*Original Item pairs*	*Selected Item pairs according to L2*	*Three Itemsets*
100	12,13,14,23,24,34	23	NIL
200	56,57,58,59,67,68,69,78,79,89	68,69,89	689
300	68,69,610,89,810,910	68,69,89	689
400	23,39	23	NIL

Here, 689 has a support of 2 and so the frequent item grouping is 689.

We have found the identical three item grouping by using both the Apriori and the DHP algorithms. Now, the same procedure that was discussed in the section on the Apriori algorithm can be applied to determine rules for the three item group {6, 8, 9}.

9.11 Dynamic Itemset Counting (DIC)

The major issue with the Apriori algorithm is that it requires multiple scans of the transaction database which is equal to the number of items in the last candidate itemset that was checked for its support.

For example, to identify candidate 3-itemset groupings, it will require a minimum of 3 scans of the database and to identify candidate 4-itemset groupings, it will require four scans of the database.

The major advantage of the Dynamic Itemset Counting (DIC) algorithm is that it requires only 2 scans of the database to identify frequent item pairs in the whole database. It reduces the number of scans required by not just doing one scan for the frequent 1-itemset and another for the frequent 2-itemset but combining the counting for a number of itemsets as soon as it appears that it might be necessary to count it.

The basic algorithm is as follows.

1. Divide the transaction database into *n* number of partitions.
2. In the first partition of the transaction database, count the 1-itemsets.
3. Count the 1-itemsets at the beginning of the second partition of the transaction database and also start counting the 2-itemsets using the frequent 1-itemsets from the first partition.
4. Count the 1-itemsets and the 2-itemsets at the beginning of the third partition and also start counting the 3-itemsets using results from the first two partitions.
5. Continue this process until the whole database is scanned once. There will be final set of frequent 1-itemsets once you scan the database fully.
6. Go back to the starting of the transaction database and continue counting the 2-itemsets and the 3-itemsets, etc.
7. After the end of the first partition in the second scan of the database, the whole database has been scanned for 2-itemsets. Therefore, we have the final set of frequent 2-itemsets.
8. After the end of the second partition in the second scan of the database, the whole database has been scanned for 3-itemsets. Therefore, we have the final set of frequent 3-itemsets.
9. This process is continued in a similar manner until no frequent k-itemsets are identified.

Illustration on working of the DIC algorithm

Let us consider, a dataset having 100 records which is divided into a number of, say 5 equal partitions. Now, the major assumption in DIC is that the data should be homogeneous throughout the file. This means that all the items should appear in partitions and the percentage of each item in the partition should be same as percentage of each item in whole dataset. In simple words, if there are 5 items A, B, C, D and E in the dataset then each of these items should appear in the each partition and if the percentage of item A in the whole transaction dataset is 60%, then it should be the same in each partition, i.e, if the count of data item A is 50 in whole dataset of 100 transactions, then the item should have the count of 10 in each partition of 20 records i.e. 50% both ways.

On the assumption that data is homogenous throughout in each partition, data is processed for the DIC algorithm.

Here, we have a database of 100 transactions which is divided into five equal partitions of 20 records each. During counting of the 1-itemsets in the first partition of the transaction database, we have identified data items A, B, C and D in the first partition. The count of these items is indicated as C_A, C_B and so on. Count the 1-itemsets at the beginning of the second partition, and also starts counting the 2-itemsets using the frequent 1-itemsets from the first partition, i.e. we also start counting two item pairs, i.e., AB, AC, AD, BC, BD and CD. The working of the DIC algorithm has been illustrated in Table 9.101.

It has been observed the counting of 1-itemsets, i.e., A, B, C and D has been completed after

Table 9.101 Working of the DIC algorithm for the example database

Total dataset of 100 records	Partition-1 20 records	PHASE-1 1-iemsets A-C_A B-C_B C-C_C D-C_D				PHASE-2 2-itemsets AB-C_{AB} AC-C_{AC} AD-C_{AD} BC-C_{BC} BD-C_{BD} CD-C_{CD}	PHASE-2 3-itemsets ABC-C_{ABC} ABD-C_{ABD} ACD-C_{ACD} BCD-C_{BCD}	PHASE-2 4-itemsets ABCD- C_{ABCD}	**Count of 2-itemset has been completed after first partition of second pass**
	Partition-2 20 records	1-iemsets A-C_A B-C_B C-C_C D-C_D	2-itemsets AB-C_{AB} AC-C_{AC} AD-C_{AD} BC-C_{BC} BD-C_{BD} CD-C_{CD}			PHASE-2 3-itemsets ABC-C_{ABC} ABD-C_{ABD} ACD-C_{ACD} BCD-C_{BCD}	PHASE-2 4-itemsets ABCD-C_{ABCD}	**Count of 3-itemset has been completed after second partition of second pass**	
	Partition-3 20 records	1-iemsets A-C_A B-C_B C-C_C D-C_D	2-itemsets AB-C_{AB} AC-C_{AC} AD-C_{AD} BC-C_{BC} BD-C_{BD} CD-C_{CD}	3-itemsets ABC-C_{ABC} ABD-C_{ABD} ACD-C_{ACD} BCD-C_{BCD}		PHASE-2 4-itemsets ABCD-C_{ABCD}	Count of 4-itemsets has been completed after third partition of second pass		

Contd.

	Partition-4 20 records	1-iemsets A-C_A B-C_B C-C_C D-C_D	2-itemsets AB-C_{AB} AC-C_{AC} AD-C_{AD} BC-C_{BC} BD-C_{BD} CD-C_{CD}	3-itemsets ABC-C_{ABC} ABD-C_{ABD} ACD-C_{ACD} BCD-C_{BCD}	4-itemsets ABCD-C_{ABCD}		
	Partition-5 20 records	1-iemsets A-C_A B-C_B C-C_C D-C_D	2-itemsets AB-C_{AB} AC-C_{AC} AD-C_{AD} BC-C_{BC} BD-C_{BD} CD-C_{CD}	3-itemsets ABC-C_{ABC} ABD-C_{ABD} ACD-C_{ACD} BCD-C_{BCD}	4-itemsets ABCD-C_{ABCD}	Count of 1-itemset has been completed after one pass	

one pass, the counting of 2-itemset has been completed after the first partition of the second pass, counting of 3-itemsets has been completed after the second partition of the second pass and counting of 4-itemsets has been completed after the third partition of the second pass. It is clear from above illustration that DIC will require only two scans of database while Apriori will require 4 scans for the identification of frequent 4-itemsets.

Major condition to make DIC effective

The DIC algorithm works well when the data throughout the file is relatively homogeneous as it starts counting of the 2-itemset before having a count of final 1-itemset. The algorithm is not able to identify the large itemset till the whole database is scanned if the data distribution is not homogeneous. The major benefit of DIC is that it finishes the counting of the itemset in two scans of the database while Apriori often takes three or more scans.

9.12 Mining Frequent Patterns without Candidate Generation (FP Growth)

As only frequent items are required for identifying the association rules, so it is best to identify the frequent items in the dataset and ignore all others. The FP growth works on the same principle and instead of generating candidate itemsets as in the case of the Aprioiri algorithm, frequent pattern-growth (FP growth) only tests and generates frequent itemsets. Thus, the major difference between the Apriori algorithm and FP growth is that FP growth does not generate candidate itemsets, it only tests, whereas the Apriori algorithm generates candidate itemsets and then tests them.

To improve the performance of the algorithm, it uses the principle to store frequent items in a compact structure which does not require the need to use the original transaction database repeatedly.

The working of the FP growth algorithm is as follows:

1. Like the Apriori algorithm, the transaction database is scanned once to identify all the 1-itemset frequent items and their supports are noted down.
2. Then 1-itemset frequent items are sorted in descending order of their support.
3. After that the FP-tree is created with a 'NULL' root.
4. Then the first transaction is obtained from the transaction database and all the non-frequent items are removed. And the remaining items are listed according to the order in the sorted frequent items.
5. After this, the first branch of the tree is constructed using the transaction with each node corresponding to a frequent item and by setting the frequency of each item as 1 for the first transaction.
6. Then the next transaction is retrieved from the transaction database and all the non-frequent items are removed. And the remaining items are listed according to the order in the sorted frequent items as done in step 4.
7. Now, the transaction is inserted in the tree using any common prefix that may appear and

count of items is increased by 1. If no common prefix is found then the branch of the tree is constructed using the transaction; with each node corresponding to a frequent item and by setting the frequency of each item as 1.

8. Step 6 is repeated until all transactions in the database have been processed.

To understand this algorithm let us find frequent item pairs by building FP-trees for the following database.

Example: Let us consider a transaction database shown in Table 9.102. The minimum support required is 50% and confidence is 75%.

Table 9.102 Transaction database

100	Butter, Curd, Eggs, Jam
200	Butter, Curd, Jam
300	Butter, Eggs, Muffin, Nuts
400	Butter, Eggs, Jam, Muffin
500	Curd, Jam, Muffin

According to the algorithm, the first step will be to scan the transaction database to identify all the frequent items in the 1-itemsets and sort them in descending order of their support.

The frequent 1-itemset database sorted on the basis of frequency is shown in Table 9.103.

Table 9.103 Frequency of each item in sorted order

Item	*Frequency*
Butter	4
Jam	4
Curd	3
Eggs	3
Muffin	3

Here, Nuts has been removed as it has a support of 1 only, which is less than the threshold limit of support (which is 3 in our case).

To create the FP-tree, the next step will be to exclude all the items that are not frequent from the transactions and sort the remaining frequent items in descending order of their frequency count. The example transaction database will be processed accordingly. The updated database after eliminating the non-frequent items and reorganising the frequent items on the basis of frequency, is shown in Table 9.104.

Table 9.104 Updated database after eliminating the non-frequent items and reorganising it according to support

Transaction ID	*Items*
100	Butter, Jam, Curd, Eggs
200	Butter, Jam, Curd
300	Butter, Eggs, Muffin
400	Butter, Jam, Eggs, Muffin
500	Jam, Curd, Muffin

It is important to note that in case of identical support, items are arranged according to lexicographic or sorted order. Now, we create the FP-tree by using the algorithm discussed above. For simplicity of discussion, we are referencing items with their first letter, i.e, Butter by B, Jam by J and so on.

An FP-tree consists of nodes. Here, each node contains three fields, i.e, an item name, a count, and a node link as given in Figure 9.17. To built the FP-tree let us consider first transaction, i.e., 100 having Butter, Jam, Curd, Eggs as items. Each item node will be created from the root node, i.e., NULL as shown in Figure 9.17. The tree is built by making a root node labeled NULL. A node is made for each frequent item in the first transaction and the count is set to 1. For example, to build the FP-tree of Figure 9.17, the first transaction {B, J, C, E} is inserted in the empty tree with the root node labeled NULL. Each of these items is given a frequency count of 1.

Next, the tree is traversed for the next transaction, i.e., 200. If a path already exists then it will follow the same path and corresponding count of item is increased by 1. If a path does not exist then a new path will be created. Next the second transaction, which is {B, J, C} having all the items common with first, is inserted. It changes the frequency of each item, i.e., {B, J, C} to 2. The FP for first two transactions is shown in Figure 9.18.

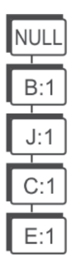


Figure 9.17 FP-tree for first transaction, i.e., 100 only

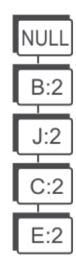


Figure 9.18 FP-tree for the first two transactions

Similarly, third transaction is considered and {B, E, M} is inserted. This requires that nodes for E and M be created. The counter for B goes to 3 and the counter for E and M is set to 1.The FP-tree for first three transactions is shown in Figure 9.19.

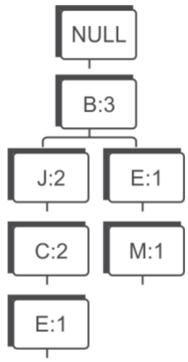


Figure 9.19 FP-tree for first three transactions

The next transaction {B, J, E, M} results in counters for B and J going up to 4 and 3 respectively and a new nodes for E and M have been inserted with count of 1 under node J as shown in Figure 9.20.

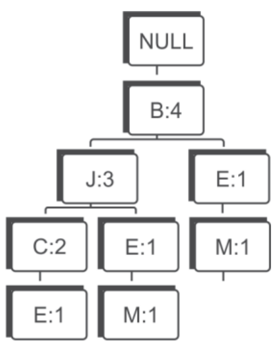


Figure 9.20 FP-tree for first four transactions

The last transaction {J, C, M} results in a brand new branch for the tree which is shown on the right-hand side in Figure 9.21 with a counter of 1 for each item.

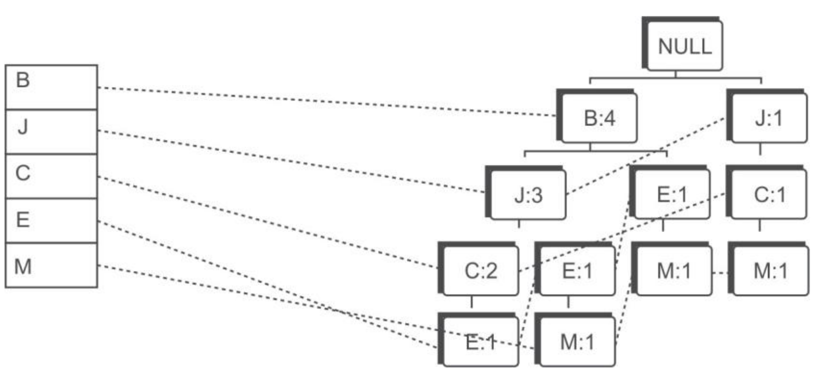


Figure 9.21 The final FP-tree for the example database after five transactions

The count tells the number of occurrences the path (constructed from the root node to this node) has in the transaction database. The node link is a link to the next node in the FP-tree containing the same item name or a null pointer if this node is the last one with this name. The FP-tree also consists of a header table with an entry for each itemset and a link to the first item in the tree with the same name. This linking is done to make traversal of the tree more efficient. Nodes with the same item name in the tree are linked via the dotted node-links.

The nodes near the root in the tree are more frequent than those further down the tree. Each identical path is only stored in the FP-tree once, which often makes the size of an FP-tree smaller than the corresponding transactional database. The height of an FP-tree is always equal to the maximum number of itemsets in a transaction excluding the root node. The FP-tree is compact and often orders of magnitude smaller than the transaction database. Once the FP-tree is constructed, the transaction database is not required.

Identification of frequent itemsets from FP-tree

The mining on the FP-tree structure is performed using the Frequent Pattern growth (FP growth) algorithm. This algorithm starts with the least frequent item, *i.e.*, the last item in the header table. Then the algorithm identifies all the paths from the root to this least frequent item and adjusts the count on the basis of the item's support count.

First, look at Figure 9.21 built using the FP-tree to identify the frequent itemsets in the example.

Start with the item M and identify the patterns given as follows:

Identification of patterns for item M

BEM(1)
BJEM(1)
JCM(1)

From this we can identify that, the support for BM is 2 (1 from BEM and 1 from BJEM), but its support is less than the threshold limit of 3, so this item pair will be discarded and we have identified no frequent item pairs by using item M.

Next consider the item E and identify the patterns given as follows:

Identification of patterns for item E

BJCE(1)
BJE(1)
BE(1)

From this we can identify that the support for BE is 3 (1 from BJCE, 1 from BJE and 1 from BE). However there is also a three item pair BJE having a support of 2 (1 from BJCE and 1 BJE), but its support is less than threshold value so will be discarded.

So, we have identified BE has two item pairs having support more than the threshold limit.

Next consider the item C and identify the following patterns:

Identification of patterns for item C

BJC(2)
JC(1)

From this we can identify that, the support for JC is 3 (2 from BJC and 1 from JC). There is no other item pair having support equal to or more than 3.

So, we have identified JC has two item pairs having supports more than the threshold limit.

Next consider the item J to identify its patterns.

Identification of patterns for item J

BJ(3)
J(1)

From this we can identify that, the support for BJ is 3. There is no other item pair having support equal to or more than 3.

So, we have identified that BJ has two item pairs having support more than the threshold limit.

Since, B is at top of the tree and cannot have any item prior to it to make a pair, so there is no need to follow links for item B. Thus, when the tree only contains one path, the algorithm will find all the possible combinations of the items in the itemset that support the minimum support count. It is therefore important to start with the least frequent item the first time, otherwise the items with the higher frequency will not be considered.

Thus, the final list of frequent item pairs for the given database of transactions is shown in Table 9.105.

Table 9.105 Frequent item pairs for database example given in table

Frequent Item pairs	*Count*
BE	3
JC	3
BJ	3

Finding association rules

To find the association mining rules, the same approach is followed as in Apriori algorithm.

For item pair BE, the possible rules are B→E and E→B. The confidence for each rule will be calculated and if its value is more than given threshold value of confidence then corresponding rule will be selected otherwise it will be discarded. The same approach will be followed for other frequent item pairs JC and BJ to find association rules for the given dataset.

The readers are advised to identify association mining rules that have confidence of more than 80%.

I hope that, you have identified it correctly and answer is E→B and C→J as explained below.

Confidence of $(B \rightarrow E) = S(B \cap E) / S(B) = 3/4 = 0.75$ [support of individual items is given in Table 9.101]

Confidence of $(E \rightarrow B) = S(E \cap B) / S(E) = 3/3 = 1.0$

Confidence of $(J \rightarrow C) = S(J \cap C) / S(J) = 3/4 = 0.75$

Confidence of $(C \rightarrow J) = S(C \cap J) / S(C) = 3/3 = 1$

Confidence of $(B \rightarrow J) = S(B \cap J) / S(B) = 3/4 = 0.75$

Confidence of $(J \rightarrow B) = S(J \cap B) / S(J) = 3/4 = 0.75$

Example: Let us consider another database given in Table 9.106, to identify frequent item pairs having support more than 50%.

Table 9.106 Transaction database

TID	Items
100	A C D
200	B C E
300	A B C E
400	B E

The count for each item is given in Table 9.107.

Table 9.107 Count for each data item

Item	Count
A	2
B	3
C	3
D	1
E	3

On the basis of the frequent items given in Table 9.108, those items are selected that have support equal to or more than 2 and these frequent items are sorted on the basis of their frequency as shown in Table 9.108.

Table 9.108 Frequency of each item in sorted order

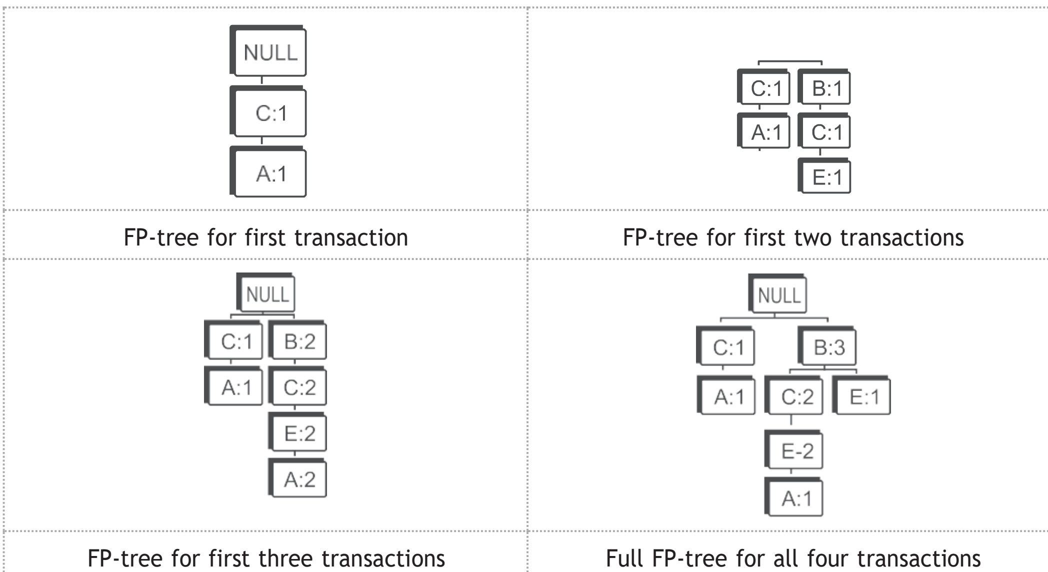
Item	Count
B	3
C	3
E	3
A	2

Now, the non-frequent items are removed from the transaction database and items are ordered on the basis of their frequency as given in the above table. The database after eliminating the items that are not frequent and reorganising them on the basis of their frequency is shown in Table 9.109.

Table 9.109 Modified database after eliminating the non-frequent items and reorganising

TID	Items
100	C A
200	B C E
300	B C E A
400	B E

The process of creating the FP-tree for the database given above has been shown in Figure 9.22 on a step-by-step basis.



FP-tree for first transaction | FP-tree for first two transactions

FP-tree for first three transactions | Full FP-tree for all four transactions

Figure 9.22 Illustrating the step-by-step creation of the FP-tree

Thus, the final FP-tree with nodes will be as shown in Figure 9.23.

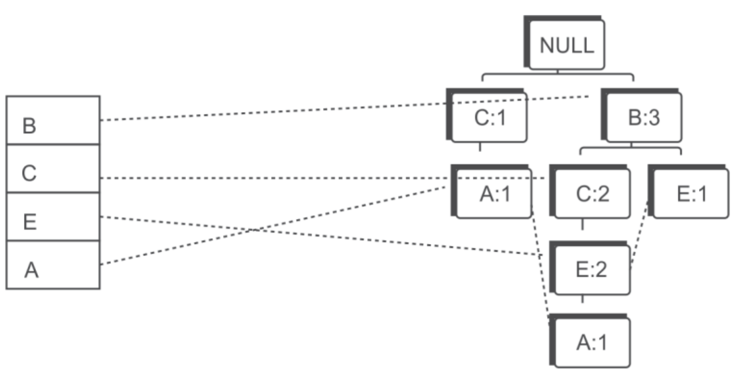


Figure 9.23 Final FP-tree for database given in Table 9.105

Identification of Frequent Itemsets from the FP-tree

Start with the item A and identify the patterns given as follows:

Identification of patterns for item A

CA(1),
BCEA(1),

The support for CA is 2 (1 from CA and 1 from BCEA) and it is equal to the threshold limit thus CA is identified as a frequent item pair.

Next consider the item E and identify the following patterns:

Identification of patterns for item E

BCE(2)
BE(1)

The support for BCE is 2 and it is equal to the threshold limit thus BCE is identified as a frequent 3-itemset.

Also from E, BE is identified as a frequent 2-item pair, since it has a support of 3 (2 from BCE and 1 from BE).

Thus, we have selected two frequent item pairs from E, i.e., BCE and BE.

Next consider the item C and examine the following patterns:

Identification of patterns for item C

BC(2)

The support for BC is 2 and it is equal to the threshold limit thus BC is identified as a frequent item pair.

Since, B is at top of the tree and cannot have any item prior to it to make a pair, so there is no need to follow links for item B.

The final list of frequent item pairs for the given database of transactions is as shown in Table 9.110.

Table 9.110 Frequent item pairs for the example database

Frequent Item pairs	*Count*
CA	2
BCE	2
BE	2
BC	2

Finding association rules

The identification process of association rules having confidence more than 70% has been shown in Table 9.111.

Table 9.111 Calculation of confidence for identification of association rules

CA	C→A = S(C∩A) / S(C) = 2/3 = 0.67 A→C = S(A∩C) / S(A) = 2/2 = 1.0	Selected rule is A→C
BCE	B→CE = S(B∩C∩E) / S(B) = 2/3 = 0.67	Confidence is less then threshold value, so it will be discarded.
	CE→B = S(B∩C∩E) / S(CE) = 2/2 = 1.0 Possible other non-implied rules are: C→E, E→B C→E = S(C∩E) / S(C) = 2/3 = 0.67 E→B = S(E∩B) / S(E) = 2/3 = 0.67	Selected rule is CE→B. Others are discarded
BE	B→E = S(B∩E) / S(B) =2/3 = 0.67 E→B = S(E∩B) / S(E) =2/3 = 0.67	Both the rules are discarded.
BC	B→C = S(B∩C) / S(B) = 2/3 = 0.67 C→B = S(C∩B) / S(C) = 2/3 = 0.67	Both the rules are discarded.

Thus, final association rules having confidence more than 70% are CE→B and A→C.

Example: Let us consider another database given in Table 9.112, to identify frequent item pairs having support more than 50%.

Table 9.112 Transaction database

Tr.	Itemsets
T1	{1,2,3,4}
T2	{1,2,4}
T3	{1,2}
T4	{2,3,4}
T5	{2,3}
T6	{3,4}
T7	{2,4}

The count for each item is given in Table 9.113.

Table 9.113 Frequency of each item

Item	Support
{1}	3
{2}	6
{3}	4
{4}	5

On the basis of the frequent items given in Table 9.114, those items are selected that have support equal to or more than 2 and these frequent items are sorted on the basis of their frequency as shown in Table 9.114.

Table 9.114 Frequency of each item in sorted order

Item	Support
{2}	6
{4}	5
{3}	4
{1}	3

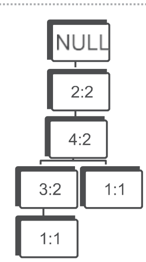
Now, the non-frequent items are removed from the transactions and items are organized according to their frequency as given in the above Table 9.114. Since, all items are frequent, transaction items are reorganized according to their support as shown in Table 9.115.

Table 9.115 Modified database after eliminating the non-frequent items and reorganizing

T1	2 4 3 1
T2	2 4 1
T3	2 1
T4	2 4 3
T5	2 3
T6	4 3
T7	2 4

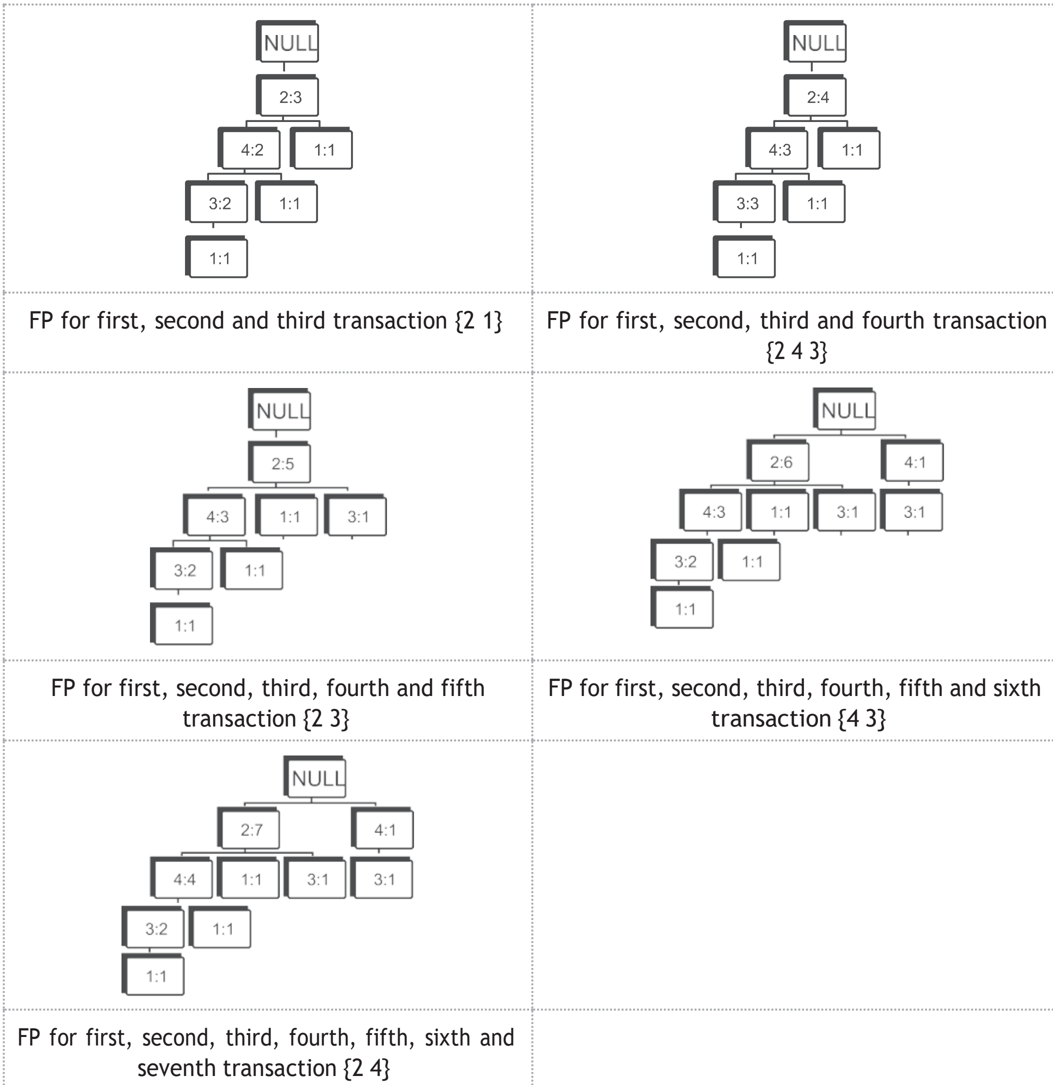
The process of creating the FP-tree for this database has been shown in Figure 9.24 on a step-by-step basis.

FP-tree for first transaction: {2 4 3 1}



FP for first and second transaction {2 4 1}

Contd.



FP for first, second and third transaction {2 1}

FP for first, second, third and fourth transaction {2 4 3}

FP for first, second, third, fourth and fifth transaction {2 3}

FP for first, second, third, fourth, fifth and sixth transaction {4 3}

FP for first, second, third, fourth, fifth, sixth and seventh transaction {2 4}

Figure 9.24 Illustrating the step-by-step creation of the FP-tree

Thus, the final FP-tree with nodes will be as shown in Figure 9.25.

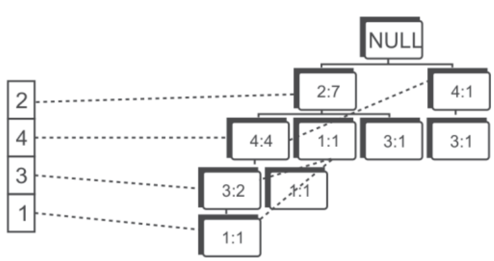


Figure 9.25 FP-tree for the example database

Identification of frequent itemsets from the FP-tree

Start with the item 1 and identify the patterns given as follows.

Identification of patterns for item 1

2431(1)
241(1)
21 (1)

The support for 21 is 3 (1 from 2431, 1 from 241 and 1 from 21) and it is equal to the threshold limit thus 21 is a identified as frequent item pair.

Next consider the item 3 and identify its patterns.

Identification of patterns for item 3

243(2)
23(1)
43(1)

The support for 43 is 3 (2 from 243 and 1 from 43) and it is equal to the threshold limit thus 43 is identified as a frequent item pair.

The support for 23 is also 3 (2 from 243 and 1 from 23) and it is equal to the threshold limit thus 23 is also identified as a frequent item pair.

Next consider the item 4 and identify its patterns.

Identifications of patterns for item 4

24(4)

The support for 24 is 4 and it is more than threshold limit, thus 24 is identified as a frequent item pair.

The final list of frequent item pairs for given database of transactions is shown in Table 9.116.

Table 9.116 Frequent item pairs for example database

Item	Support
{1,2}	3
{3,4}	3
{2,3}	3
{2,4}	4

It is important to note that the order of data items does not matter in listing frequent item pairs.

Finding association rules

The identification process of association rules having confidence more than 70% has been shown in Table 9.117.

Table 9.117 Association rules for database given in Table 9.76

{1, 2}	1→2 = S(1∩2) / S(1) = 3/3 =1.0 2→1 = S(2∩1) / S(1) = 3/3 = 1.0	Both the rules, i.e., 1→2 and 2→1 are selected
{3, 4}	3→4 = S(3∩4) / S(3) = 3/4 = 0.75 4→3 = S(4∩3) / S(3) = 3/4 = 0.75	Both the rules, i.e., 3→4 and 4→3 are selected
{2, 3}	2→3 = S(2∩3) / S(2) = 3/6 = 0.50 3→2 = S(3∩2) / S(3) = 3/4 = 0.75	3→2 rule has been selected
{2, 4}	2→4 = S(2∩4) / S(2) = 4/6 = 0.67 4→2 = S(4∩2) / S(4) = 4/5 = 0.80	4→2 rule has been selected

Thus, final association rules having confidence more than 70% are 1→2, 2→1, 3→4, 4→3, 3→2 and 4→2.

9.12.1 Advantages of the FP-tree approach

The advantages of the FP-tree algorithm over the Apriori algorithm are as follows:

- The FP-tree algorithm avoids the need to scan the database more than twice to identify the counts of support.
- The FP-tree completely eliminates the requirement of costly candidate generation, which can be expensive in case of the Apriori algorithm for the candidate set C2.
- The FP growth algorithm is better than the Apriori algorithm when the transaction database is huge and the minimum support count is low. Because in case of low minimum support count, there will be more items that will satisfy the support count and hence the size of the candidate sets for Apriori will be large. As the database grows, FP growth uses an efficient structure to mine patterns.

9.12.2 Further improvements of FP growth

In spite of the advantages there are possibilities to make the algorithm even more efficient and scalable. Important observations to further improve the performance of FP growth are as follows:

- It works best if the FP-tree fits in the main memory. However, in large databases this will not always be possible, and then it is better to first divide the database and then constructs FP-trees for each of the databases.
- Use ***B+-tree*** to index the FP-tree for faster access to items in the FP-tree.
- If the same data is used multiple times with different minimum confidence thresholds in order to yield different association rule mining then, it could be useful to use the same FP-tree results. This will save the initial cost of generating the FP-tree and make the algorithm overall more efficient.
- Sometimes the minimum support count will be different in different tasks, so to overcome this problem, the lowest minimum support count is used in the materialization of the FP-tree. When a higher support count is wanted, only the top of the FP-tree is used. Because of the structure of the FP-tree, it is easy to have different support counts.